Prof. Dr.-Ing. A. Siggelkow

Specification - **RWU-RV64I**



Hochschule Ravensburg-Weingarten
University of Applied Sciences

B

Chapter 1

Requirements

Table 1.1: Functional Requirements

Requirement	ID	Importance	Verifiable	Description	Remarks
General					
Gen.: #persons	G01	High	VHDL-TB	The number of persons in a room must be counted.	
Gen.: max	G02	High	VHDL-TB	The number of persons in a room must not exceed a given limit.	
Gen.: only one pers.	G03	High	?	Only one person can either enter or leave the room at a time.	Check test
Gen.: three light sensors	G04	Medium	VHDL-TB	Along the doorway, there are three light-curtains to allow direction-tracking of possible visitors.	Why not only two?

Table **??** lists the functional requirements for the **ICU0** peripheral (Section 6.5), derived from `ip_icu/README_REQ.txt`. Each requirement is atomic (one single, independently verifiable statement) and carries its own ID, Importance, and Verification Method, per IEEE 29148. The full traceability record (original requester wording, clarifying-question decisions, and every gap identified during implementation) is kept in `ip_icu/README_REQ.txt` and `ip_icu/README_IMPL.txt`; this table is the IEEE-formatted summary of the same content, and the ID column cross-references the identically-numbered requirement in Section 6.5.1.

Table 1.2: ICU0 Functional Requirements

Requirement	ID	Importance	Verifiable	Description	Remarks
Source Management					
ICU-SR: spare sources	SR-001	High	Testbench	The ICU shall reserve input slots for peripherals that do not exist yet: at least 2 spare sources.	Explicitly requested by A. Siggelkow.
ICU-SR: existing peripherals	SR-002	High	Testbench	The ICU shall accept one interrupt input per existing interrupt-capable peripheral (GPIO0, UART0, and – on *RV_PipelineCacheFwd* – QSPI0), plus the spares from SR-001.	Explicitly requested.
ICU-SR: slot count	SR-003	High	RTL Review	The register file shall provision exactly 8 source slots (1 byte per bitmap register): 3 wired to real peripherals, 2 requested spares, 3 reserved.	Decided via clarifying question (option: exactly 5 slots, minimal, was rejected in favour of headroom + a chip-independent register map).
Priority Resolution					

continued on next page

continued from previous page

Requirement	ID	Importance	Verifiable	Description	Remarks
ICU-PR: configurable priority	PR-001	High	Testbench	The ICU shall resolve priority in hardware across all simultaneously pending, unmasked sources, via a software-writable 4-bit priority value per source (ICU0_PRIO).	Explicitly requested (“Priorität ist wichtig”).
ICU-PR: tie-break	PR-002	Medium	Testbench	On equal priority between two or more pending, unmasked sources, the lowest source index shall win.	Gap identified during implementation; left undefined this would be simulator/synthesis-dependent.
Masking					
ICU-MA: per-source mask	MA-001	High	Testbench	The ICU shall provide an individually maskable enable bit per source (ICU0_IMSC).	Explicitly requested (“Masking ist wichtig”).
ICU-MA: global enable	MA-002	Medium	Testbench	The ICU shall additionally provide one global interrupt enable bit (ICU0_CTRL.GIE) gating the aggregated output line independent of per-source masking.	Gap identified during implementation: atomic “disable everything” primitive not achievable with IMSC alone.
Pending/Clear Semantics					

continued on next page

continued from previous page

Requirement	ID	Importance	Verifiable	Description	Remarks
ICU-PE: sticky pending	PE-001	High	Testbench	A source's pending bit (ICU0_RIS) shall be edge-triggered and sticky: it latches the source line's 0→1 transition and remains set until explicitly cleared by software.	Decided via clarifying question (option: level pass-through, closer to the graded reference submission's design, was rejected).
ICU-PE: race-free clear	PE-002	High	Testbench	A pending bit shall not be silently cleared by a software CLEAR that coincides, in the same clock cycle, with a new hardware edge (or software-forced SET) on the same bit: SET shall take priority over CLEAR.	Root-caused against a documented defect in `ip_icu/DUMMY_ICU/asInte` (the graded reference submission this peripheral replaces): a same-cycle set+clear silently favoured the clear.
Cause Identification					
ICU-ID: cause register	ID-001	High	Testbench	The ICU shall expose a register (ICU0_IRQID) identifying the current highest-priority pending, unmasked source.	Not named explicitly by the requester; the single biggest practical benefit of a real ICU over the OR-combine placeholder it replaces.

continued on next page

continued from previous page

Requirement	ID	Importance	Verifiable	Description	Remarks
ICU-ID: valid bit	ID-002	Medium	Testbench	ICU0_IRQID shall carry an explicit VALID bit, separate from the source-id field.	Source index 0 (GPIO0) is a real, valid id; collapsing "no interrupt" onto id = 0 would make the register ambiguous.
ICU-ID: GIE-independence	ID-003	Medium	Testbench	ICU0_IRQID shall reflect pending/priority state independent of ICU0_CTRL.GIE.	Keeps IRQID useful for diagnostics even while interrupts are globally disabled.
Software Interface					
ICU-SW: register control	SW-001	High	Testbench	The ICU shall be controllable exclusively through memory-mapped, 64-bit registers on a Wishbone B4 Classic slave interface (as_slave_bpi).	Explicitly requested ("Controlle über memory mapped Register").
ICU-SW: alignment	SW-002	Medium	RTL Review	Registers shall be 8-byte aligned.	Project-wide convention.
ICU-SW: address range	SW-003	High	RTL Review	The base address of ICU0 shall be 0x0000_0001_0000_0800 (cs[5]).	Explicitly requested ("Adressbereich festlegen"); placed at the first free 512 B slot after UART0.

continued on next page

continued from previous page

Requirement	ID	Importance	Verifiable	Description	Remarks
ICU-SW: ID register	SW-004	Low	Testbench	The ICU shall expose a read-only peripheral ID register at offset 0x00.	Consistency with every other peripheral in this SoC.
ICU-SW: functional reset	SW-005	Medium	Testbench	Reset values shall keep the system interrupt-functional without any firmware setup: ICU0_IMSC resets to all-enabled, ICU0_CTRL.GIE resets to 1.	Matches pre-ICU behaviour, where interrupts already worked without firmware configuration.
ICU-SW: bus timing	SW-006	Low	Testbench	The Wishbone interface shall respond with zero wait states (combinational ACK).	Consistent with all other Wishbone slaves in the system.
Hardware Interface					
ICU-HW: aggregated output	HW-001	High	Testbench	The ICU shall expose exactly one aggregated interrupt output line (icu_irq_o) to the CPU boundary.	Explicitly requested.
ICU-HW: level semantics	HW-002	Medium	RTL Review	icu_irq_o shall be a level output, asserted while at least one unmasked, pending, globally-enabled source exists.	Matches mip[11]'s existing read-only, level-mirroring semantics; no CPU changes required.

continued on next page

continued from previous page

Requirement	ID	Importance	Verifiable	Description	Remarks
ICU-HW: shared design	HW-003	High	Regression	ICU0 shall be a single, shared design instantiated identically on both chip variants, with a fixed, chip-independent source-index assignment.	Explicitly requested (“für beide Chips”); verified by re-running both chips’ full existing regression suites after integration (64/64+59/59 and 70/70+59/59, zero regressions).
ICU-HW: no internal CDC	HW-004	Medium	RTL Review	The ICU shall perform no cross-clock-domain synchronization; all connected sources shall already be synchronous to clk_i.	True for GPIO0/QSPI0/UART0/CP today (all on clk_div_s on both chips); documented as an explicit assumption for future sources.

Chapter 2

Document Overview

2.1 Validity of this document

This document is valid for **RWU-RV64I** version:

- V1.0: First samples
- V1.x: Revision

2.2 Target Specification Status

This **RWU-RV64I** target specification is initially derived from the **RV64I** specification. So readers being familiar with the **RV64I** architecture concepts and hardware building blocks will find most of them unchanged within this document.

This **RWU-RV64I** document represents and describes all features of the **RWU-RV64I** and is adequate as a standalone reference.

This release of the specification has the status of an target specification.

2.3 Major Changes since last Revision

Table 2.1: Spec Changes as compared to the previous **RWU-RV64I** Spec Revision

History		
Target Spec.	Current version : 1.1, 2026-06-05 Previous version : 1.0, 2022-11-22	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2022-11-22		
	2022-11-22	First draft (AS)
Current Chapter Version: C0.2; Date: 2026-06-05		
	2026-06-05	UART0 integration: `ip_uart_v2` IP, UART TX/RX simulation testbenches, RX FIFO pop bug fix in `as_uart_top.sv` (AS, Claude)

Contents

1 Requirements **1**

2 Document Overview **9**
2.1 Validity of this document 9
2.2 Target Specification Status 9
2.3 Major Changes since last Revision 10
2.4 Glossary 24
2.5 Naming Conventions 25
2.6 Document List 25

3 Product Overview **27**
3.1 Top Level View 27
3.1.1 Introduction 27
3.1.2 Key Features 28
3.1.3 Functional Block Diagram 29
3.1.4 Package 30
3.2 System View 31
3.3 Architecture Concepts Overview 31
3.3.1 Technology 31
3.3.2 System Memory Concept 32
3.3.3 Software Architecture 32
3.3.4 Interprocessor Communication Concept 32
3.3.5 Bus Concept 32
3.3.6 Clock Concept 33
3.3.7 System Control Concept 33
3.3.8 Interrupt Concept 33
3.3.9 Debug Concept 35
3.3.10 Power Management 35
3.3.11 Protection and Security Concept 35
3.4 Functional Block Overview 36
3.4.1 RISC-V Core 36

3.4.2 Instruction Memory 36
3.4.3 Data Memory 36
3.4.4 Wishbone Peripherals 37
3.4.5 Debug 37

4 Architecture Concepts **39**
4.1 Technology 40
4.2 CPU Concept 40
4.2.1 Pipeline Hazard Handling (`RV_PipelineCacheFwd` only) 44
4.3 Bus Concept 47
4.3.1 Overview 47
4.3.2 Bus Topology 48
4.3.3 Bus Selection Rationale 49
4.3.4 Wishbone B4 Classic – Peripheral Bus 50
4.3.5 AMBA AXI4 – Cache Memory Bus 53
4.3.6 Wishbone Slave BPI (`as_slave_bpi`) 55
4.3.7 Wishbone Master BPI (`as_master_bpi`) 60
4.3.8 Bus Interconnect 64
4.3.9 Comparison: Wishbone vs. AXI4 66
4.4 Interrupt Concept 66
4.5 Clock Concept 67
4.5.1 Clock Generation Unit (CGU) Overview 67
4.5.2 Clock Requirements – Variant A (RV_NoPipeline) 69
4.6 System Control Concept 71
4.7 Power Concept 71
4.8 Memory Concept 71
4.8.1 Chip Variants 72
4.8.2 Variant A – No-Cache Memory Subsystem 73
4.8.3 Variant B – Pipeline-Cache Memory Subsystem: Overview 73
4.8.4 System Memory Architecture 74
4.8.5 Cache Configuration Parameters 75
4.8.6 Physical Address Decomposition 76
4.8.7 Cache Tag and Metadata Arrays 77
4.8.8 SRAM Macro Organisation 79
4.8.9 SRAM Macro Selection – XH035 SPRAM 80
4.8.10 Instruction Cache (I-Cache) 83
4.8.11 Data Cache (D-Cache) 84
4.8.12 CPU-to-Cache Interface 87
4.8.13 AXI4 Memory Bus (Cache Controllers to Memory Arbiter) 90
4.8.14 Memory Arbiter 96

4.8.15 QSPI Controller Interface . . . 97
4.8.16 Cache Flush and Invalidation – FENCE.I . . . 97
4.8.17 DMA Coherency Protocol . . . 99
4.8.18 AXI4 Error Handling . . . 102
4.8.19 QSPI Operating Mode – Cache-Miss-Driven Read . . . 106
4.8.20 Memory Hierarchy Performance Analysis . . . 109
4.8.21 SystemVerilog Module Architecture . . . 111
4.8.22 Open Items and Refinements . . . 113
4.9 System Protection and Security Concept . . . 114
4.10 Debug Concept . . . 114
4.11 Register Concept . . . 114

5 RWU-RV64I Core 115
5.1 ALU . . . 115
5.1.1 History of ALU . . . 118
5.1.2 Functional Overview . . . 118
5.1.3 Structural Overview . . . 119
5.1.4 Service Requests of the ALU . . . 125
5.1.5 The ALU Peripheral Kernel . . . 125
5.1.6 Registers . . . 127
5.2 Instruction Decoder . . . 128
5.2.1 History of Instruction Decoder . . . 131
5.2.2 Functional Overview . . . 131
5.2.3 Structural Overview . . . 133
5.2.4 Service Requests of the Instruction Decoder . . . 141
5.2.5 The InstrDec Peripheral Kernel . . . 141
5.2.6 Registers . . . 144
5.3 Memory System . . . 145
5.3.1 System Integration of MemSys0 . . . 146
5.3.2 History of MemSys . . . 148
5.3.3 Functional Overview . . . 148
5.3.4 Structural Overview . . . 149
5.3.5 Service Requests of the MemSys . . . 173
5.3.6 The MemSys Peripheral Kernel . . . 173
5.3.7 Registers . . . 173

6 Peripherals 175
6.1 GPIO . . . 175
6.1.1 System Integration of GPIO0 . . . 175
6.1.2 History of GPIO . . . 179
6.1.3 Functional Overview . . . 179

6.1.4 Structural Overview . . . 180
6.1.5 Service Requests of the GPIO . . . 187
6.1.6 The GPIO Peripheral Kernel . . . 187
6.1.7 Registers . . . 188
6.2 Instruction Memory (I-Mem) . . . 190
6.2.1 System Integration of **I-Mem0** . . . 190
6.2.2 History of **I-Mem** . . . 193
6.2.3 Functional Overview . . . 193
6.2.4 Structural Overview . . . 194
6.2.5 Service Requests of the I-Mem . . . 194
6.2.6 The I-Mem Peripheral Kernel . . . 194
6.2.7 Registers . . . 194
6.3 QSPI . . . 195
6.3.1 System Integration of QSPI0 . . . 195
6.3.2 History of QSPI . . . 199
6.3.3 Functional Overview . . . 199
6.3.4 Structural Overview . . . 207
6.3.5 Service Requests of the QSPI . . . 216
6.3.6 The QSPI Peripheral Kernel . . . 216
6.3.7 Registers . . . 217
6.3.8 Software Programming Sequence . . . 233
6.4 UART . . . 234
6.4.1 Requirements . . . 234
6.4.2 System Integration of UART0 . . . 243
6.4.3 History of UART . . . 248
6.4.4 Functional Overview . . . 248
6.4.5 Structural Overview . . . 249
6.4.6 Service Requests of the UART . . . 249
6.4.7 The UART Peripheral Kernel . . . 250
6.4.8 Registers . . . 250
6.5 ICU . . . 254
6.5.1 Requirements . . . 254
6.5.2 System Integration of ICU0 . . . 260
6.5.3 History of ICU . . . 264
6.5.4 Functional Overview . . . 264
6.5.5 Structural Overview . . . 265
6.5.6 Service Requests of the ICU . . . 267
6.5.7 The ICU Peripheral Kernel . . . 268
6.5.8 Registers . . . 269

7 Test and Debug **273**
7.1 Simulation Test Suite 273
7.1.1 UART0 Simulation Tests 273
7.1.2 `RV_PipelineCacheFwd` Simulation Tests 274

8 Electrical Characteristics **277**

9 Memory Map and Registers **279**
9.1 System Memory Map 279
9.2 Interrupt Map 282

List of Figures

3.1 **RWU-RV64I** Overview 28
3.2 Simplified Block diagram 29
3.3 CPU Core – Top-Level Functional Block Diagram 30

4.1 Enable with a Pulse 39
4.2 On-Chip Bus Topology – Dual-Bus Architecture 49
4.3 Wishbone Classic Single Transfers: Write – Read – Write 52
4.4 as_slave_bpi – Internal Signal Flow 58
4.5 as_master_bpi – Internal Signal Flow 63
4.6 System Memory Architecture Block Diagram 75
4.7 Physical Address Decomposition (32-bit PA, 4 KB cache, 32 B line) 76
4.8 Pseudo-LRU Binary Tree for 4 Ways 79
4.9 AXI4 Cache-Line Fill: 4-Beat INCR Read Burst (ARLEN=3) . . 95
4.10 AXI4 Cache Write-Back: 4-Beat INCR Write Burst (AWLEN=3) 96

5.1 Moore-FSM: `asICache` Instruction Cache Controller (`ip_icache/src/asICache.sv`). **State nodes** show active outputs only; inactive signals are zero. cnt is the 5-bit set counter `flush_cnt_r` (counts 0…31 during cache-line invalidation). The self-loop label **1** on RESPOND and ERR denotes an unconditional one-cycle transition. 160
5.2 Moore-FSM: `asDCache` Data Cache Controller (`ip_dcache/src/asDCache.sv`). **Abbreviations on edges:** $dirty_victim$: victim way is valid and dirty; fm (`flush_mode_r`): eviction was triggered by a flush, not a normal miss; $more$: flush iterator has not yet scanned all sets/ways; $done$: flush complete (all sets/ways iterated); ok: AXI4 response is OKAY ($bresp$=0 or $rresp$=0); wr: the CPU request was a store (`lk_wr_r`). **Outputs** $rvalid$=$\neg wr$ in FILL_RESP: only a load returns data to the CPU. Self-loop label **1** denotes an unconditional single-cycle transition. 166

5.3 Moore-FSM: `asScratch` Scratchpad Controller (`ip_scratch/src/asScratch.s` **Note:** The `dc_stall_o` output is a Mealy signal (*stall* $= req \wedge \neg wr \wedge \neg busy_r$) and therefore depends on both state and inputs; it is not shown in the state node. Stores (*req* $\wedge$ *wr*) are single-cycle and never cause a stall; the SRAM write completes in the same cycle. Only loads (*req* $\wedge \neg$*wr*) require one extra cycle for the registered SRAM read output. 169

6.1 GPIO asSlaveBPI Peripheral 181
6.2 GPIO Kernel 188
6.3 QSPI Kernel Moore-FSM. Doppelter Rahmen = Reset-Zustand IDLE (grau). Grüner Rahmen = DONE (1 Zyklus, bedingungslos zu IDLE). Gestrichelter Pfad: ADDR $\rightarrow$ DATA wenn `DUMMY_CYC` $= 0$. 204
6.4 QSPI Dual-Port Peripheral (Wishbone config + AXI4 data) 207
6.5 Interrupt Request Flow 228

List of Tables

1.1 Functional Requirements 2
1.2 ICU0 Functional Requirements 3

2.1 Spec Changes as compared to the previous **RWU-RV64I** Spec Revision 10
2.2 Glossary Type Descriptions 25
2.3 Alias Names for Cores and other Objects in the Spec 25
2.4 History 25
2.5 Document List 26

3.1 History 27
3.2 History 31
3.3 History 31
3.4 History 36

4.1 History 40
4.2 History 40
4.3 Notations for Instructions 41
4.4 RV32I Base Integer Instruction Set [2], [3], [1] 41
4.5 RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1] 43
4.6 Pipeline Hazard Test Suite 46
4.7 History 47
4.8 On-Chip Bus Summary 48
4.9 Bus Selection Rationale 50
4.10 Wishbone Signal Naming 51
4.11 AXI4 Bus Participants 53
4.12 AXI4 Cache Bus Parameters 54
4.13 History of as_slave_bpi 55
4.14 Functional Configuration of as_slave_bpi 55
4.15 I/O of as_slave_bpi 56

4.16 as_slave_bpi Internal Signals 58
4.17 History of as_master_bpi 60
4.18 Functional Configuration of as_master_bpi 60
4.19 I/O of as_master_bpi 61
4.20 as_master_bpi Internal Signals 63
4.21 Wishbone B4 Classic vs. AXI4 – Key Differences 66
4.22 History 66
4.23 History 67
4.24 CGU Clock Domains 68
4.25 RV_NoPipeline – Recommended Clock Frequencies 69
4.26 History 71
4.27 History 71
4.28 History 71
4.29 Chip Variant Comparison 72
4.30 Memory Hierarchy Summary 74
4.31 Cache Configuration Parameters 76
4.32 Physical Address Fields 77
4.33 I-Cache – Metadata Bits per Set 78
4.34 D-Cache – Metadata Bits per Set 78
4.35 PLRU Bit Semantics 79
4.36 PLRU Victim Selection and Update on Cache Access 79
4.37 SRAM Macro Logical Requirements 80
4.38 XH035 SPRAM Instances 81
4.39 XH035 SPRAM Port Mapping 82
4.40 XH035 SPRAM Timing Parameters (worst-case SS/125 °C/3.0 V) 82
4.41 I-Cache Controller States 84
4.42 Software Segments in Scratchpad SRAM 86
4.43 D-Cache Controller States 87
4.44 Instruction Bus Interface (`as_icache_if`) 88
4.45 Data Bus Interface (`as_dcache_if`) 89
4.46 AXI4 Memory Bus Parameters 91
4.47 AXI4 AR Channel (Read Address) – I/D Cache Master 91
4.48 AXI4 R Channel (Read Data) – I/D Cache Master 92
4.49 AXI4 AW Channel (Write Address) – **NOT USED**; retained for reference only 92
4.50 AXI4 W Channel (Write Data) – **NOT USED**; retained for reference only 93
4.51 AXI4 B Channel (Write Response) – **NOT USED**; retained for reference only 93
4.52 Memory Arbiter Priority 96
4.53 Memory Arbiter Ports 97

4.54 FLUSH State – Operation per Clock Cycle 98
4.55 DMA Coherency Requirements by Memory Region 100
4.56 Minimum DMA Engine Signals for Cache Coherency 101
4.57 AXI4 RRESP Codes Handled by Cache Controllers 102
4.58 AXI4 Error to RISC-V Exception Mapping 103
4.59 ERR State – I-Cache and D-Cache (identical behaviour) 103
4.60 Error Signals Added to Cache Interfaces 104
4.61 Supported QSPI Read Commands 107
4.62 Cache-Line Fill Latency Estimate (Quad Fast Read, `0x6B`) 108
4.63 AMAT and Speed-Up for a Hypothetical L2 SRAM ($t_{\mathrm{L2}} = 2$ cycles, $t_{\mathrm{miss}} = 42$ cycles) 110
4.64 Miss Penalty vs. System Clock (QSPI fixed at 25 MHz) 110
4.65 Open Items – Memory Concept v0.1 113
4.66 History 114
4.67 History 114
4.68 History 114

5.1 Authors 115
5.2 History 116
5.3 ALU0 Feature Set 116
5.4 HW Interfaces of Module ALU0 117
5.5 History 118
5.6 ALU0 I/O 120
5.7 RV32I Base Integer Instruction Set [2], [3], [1] 125
5.8 RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1] 127
5.9 Authors 128
5.10 History 128
5.11 InstrDec0 Feature Set 129
5.12 HW Interfaces of Module InstrDec0 130
5.13 History 131
5.14 InstrDec0 I/O 134
5.15 Instruction Decoder Service Requests 141
5.16 RV32I/RV64I Base Integer Instruction Set, incl. FENCE/SYSTEM [2], [3], [1] 143
5.17 Memory Subsystem – SystemVerilog Module Overview 145
5.18 Authors 146
5.19 History 146
5.20 Memory System Feature Set 147
5.21 HW Interfaces of Module MemSys0 147
5.22 History 148

5.23 asMemTop Port List (1) . . . 150
5.24 asMemTop Port List (2) . . . 151
5.25 asICacheBlock Port List . . . 152
5.26 asDCacheBlock Port List . . . 154
5.27 asSpBlock Port List . . . 155
5.28 asICacheCtrl Port List (1) . . . 157
5.29 asICacheCtrl Port List (2) . . . 158
5.30 asDCacheCtrl Port List (1) . . . 163
5.31 asDCacheCtrl Port List (2) . . . 164
5.32 asMemArb Port List (1) . . . 170
5.33 asMemArb Port List (2) . . . 171
5.34 asSpramModel Port List (matches XH035 SPRAM interface) . 172

6.1 Authors . . . 175
6.2 History . . . 176
6.3 GPIO0 Feature Set . . . 176
6.4 HW Interfaces of Module GPIO0 . . . 177
6.5 Register List of Module GPIO0 . . . 178
6.6 History . . . 179
6.7 GPIO Kernel I/O . . . 183
6.8 GPIO BPI I/O . . . 184
6.9 GPIO BPI Internals . . . 185
6.10 GPIO Top I/O . . . 185
6.11 GPIO Top Signals . . . 186
6.12 GPIO Service Requests . . . 187
6.13 GPIO Register List . . . 189
6.14 Authors . . . 190
6.15 History . . . 191
6.16 **I-Mem0** Feature Set . . . 191
6.17 HW Interfaces of Module **I-Mem0** . . . 192
6.18 Register List of Module **I-Mem0** . . . 192
6.19 History . . . 193
6.20 **I-Mem** Kernel I/O . . . 194
6.21 Authors . . . 195
6.22 History . . . 195
6.23 QSPI0 Feature Set . . . 196
6.24 HW Interfaces of Module QSPI0 . . . 197
6.25 Register List of Module QSPI0 . . . 198
6.26 History . . . 199
6.27 Reference NOR-Flash Devices . . . 201
6.28 as_qspi FSM States . . . 203

6.29 `as_qspi` Internal Counters 204
6.30 `as_qspi` Ports – System and Configuration 209
6.31 `as_qspi` Ports – Status, FIFO and SPI 210
6.32 `as_slave_bpi` Ports 211
6.33 QSPI BPI Internals 211
6.34 Wishbone Classic Zero-Wait-State Timing 212
6.35 `as_qspi_top` AXI4 Slave Port Signals (AR + R channels only) 213
6.36 `as_qspi_top` Ports – System, Wishbone, and SPI 215
6.37 QSPI Top Signals 216
6.38 `as_qspi_top` Register Map 218
6.39 QSPI Interrupt Sources 229
6.40 UART Requirements – Category Codes 234
6.41 UART Requirements – Frame Format 235
6.42 UART Requirements – Baud Rate and Oversampling 236
6.43 UART Requirements – Transmitter 237
6.44 UART Requirements – Receiver 238
6.45 UART Requirements – FIFOs 239
6.46 UART Requirements – Error Detection 240
6.47 UART Requirements – Interrupts 241
6.48 UART Requirements – Software Interface 242
6.49 UART Requirements – Hardware Interface 243
6.50 UART0 Authors 243
6.51 UART0 Section History 244
6.52 UART0 Feature Set 245
6.53 HW Interfaces of Module UART0 246
6.54 Register List of Module UART0 247
6.55 History of UART Section 248
6.56 UART0 Service Requests 249
6.57 UART Register List 250
6.58 UART0_LCR Bit Fields 251
6.59 UART0_CTRL Bit Fields 251
6.60 UART0_STATUS Bit Fields 252
6.61 UART0_FIFOSTAT Bit Fields 252
6.62 UART0_RIS Bit Fields 253
6.63 UART0_ICR Bit Fields 253
6.64 ICU Requirements – Category Codes 254
6.65 ICU Requirements – Source Management 255
6.66 ICU Requirements – Priority Resolution 255
6.67 ICU Requirements – Masking 256
6.68 ICU Requirements – Pending/Clear Semantics 256
6.69 ICU Requirements – Cause Identification 257

6.70 ICU Requirements – Software Interface 258
6.71 ICU Requirements – Hardware Interface 259
6.72 ICU0 Authors 260
6.73 ICU0 Section History 260
6.74 ICU0 Feature Set 261
6.75 ICU0 Source Index Assignment (fixed, identical register map on both chips) 261
6.76 HW Interfaces of Module ICU0 262
6.77 Register List of Module ICU0 263
6.78 History of ICU Section 264
6.79 ICU Kernel (cc_icu) I/O 266
6.80 ICU Top (cc_icu_top) I/O 266
6.81 ICU0 Service Requests 267
6.82 ICU Register List 269
6.83 ICU0_ID Bit Fields 270
6.84 ICU0_RIS Bit Fields 270
6.85 ICU0_IMSC Bit Fields 270
6.86 ICU0_MIS Bit Fields 271
6.87 ICU0_ICR Bit Fields 271
6.88 ICU0_ISR Bit Fields 271
6.89 ICU0_PRIO Bit Fields 271
6.90 ICU0_IRQID Bit Fields 272
6.91 ICU0_CTRL Bit Fields 272

9.1 System Memory Map 279
9.2 GPIO0 Register Offsets (from base 0x0000_0001_0000_0000) – *Both variants* 280
9.3 QSPI0 Register Offsets (from base 0x0000_0001_0000_0200) – *RV_PipelineCacheFwd only* 281
9.4 CGU Register Offsets (from base 0x0000_0001_0000_0400) – *Both variants* 281
9.5 UART0 Register Offsets (from base 0x0000_0001_0000_0600) 282
9.6 External Interrupt Map (irq_ext_i) 282

2.4 Glossary

Table 2.2: Glossary Type Descriptions

Type	Description
UART	Universal Asynchronous Receiver Transmitter
Bus	Binary Unit System
RGB	Red-Green-Blue. A color mixing system.
VGA	Video Graphics Array. A computer graphics standard.

2.5 Naming Conventions

Table 2.3: Alias Names for Cores and other Objects in the Spec

Object	Names also used in the spec
ARM926EJ-S ™	ARM, Controller, Micro controller unit, MCU, Processor, CPU
Bus	Binary Unit System

2.6 Document List

Table 2.4: History

History		
Target Spec.	Current version : 1.0, 2022-11-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2022-11-22		
	2022-11-22	First draft (AS)

The following documents are referred to in this document. They are either available in a separate attachment directory or are general standards.

Table 2.5: Document List

Nr	Doc Type	Version	Date	Filename	Attached
1	ARM engineering specification	A09	15.10.2024	ARM926EJ-S_EngSpec_A09	No
2	ARM926 technical reference manual	-	16.10.2024	DDI0198B_926	Yes

Chapter 3

Product Overview

3.1 Top Level View

Table 3.1: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.1.1 Introduction

The **RWU-RV64I** is a RISC-V processor with the integer 64 bit instruction set.

Figure 3.1 shows the top level of the chip.

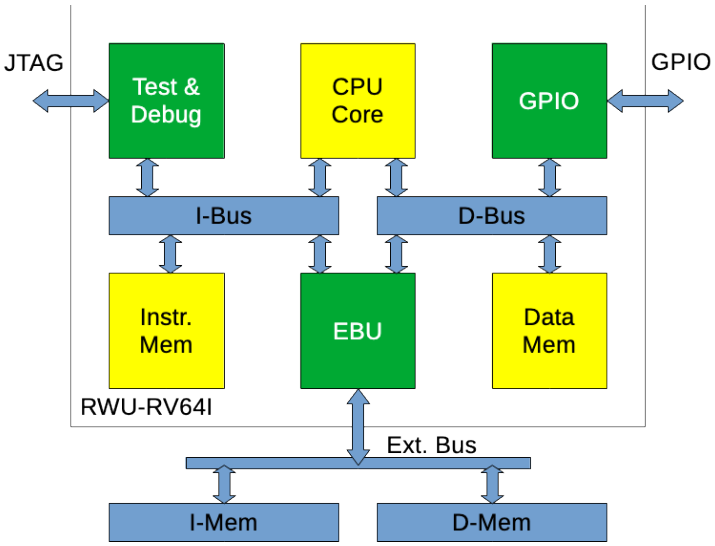


Figure 3.1: **RWU-RV64I** Overview

3.1.2 Key Features

Two chip variants are defined (see also Section 4.8.1):

Variant A – No-Cache (RTL: `RV_NoPipeline`):

- RISC-V RV64I core, non-pipelined single-cycle execution
- On-chip Instruction Memory: 32 kB (8 192 instructions, JTAG-loadable)
- On-chip Data Memory: 8 kB (Wishbone slave, byte-enable, R/W)
- GPIO and UART0 peripherals (Wishbone)
- Clock Generation Unit (CGU, Wishbone)
- JTAG TAP for I-Mem loading and boundary-scan
- No cache, no external Flash interface, no QSPI controller

Variant B – Pipeline-Cache (RTL: `RV_PipelineCacheFwd`):

- RISC-V RV64I core, 5-stage pipeline (IF/ID/EX/MEM/WB) with EX/MEM→EX and MEM/WB→EX forwarding; only load-use (1 cycle) and CSR hazards still stall
- Instruction Cache: 4 kB, 4-way set-associative, 32 B line

- Data Cache: 4 kB, 4-way set-associative, 32 B line, read-allocate, no-write (Flash is read-only; stores bypass to Scratchpad)
- On-chip Scratchpad SRAM: 8 kB (direct access, R/W, replaces Variant A D-Mem)
- Off-chip NOR Flash via QSPI: code and read-only data
- GPIO, CGU and UART0 peripherals (Wishbone); QSPI controller (Wishbone config registers, AXI4 cache-fill path)
- JTAG TAP (boundary-scan; I-Mem scan chain removed)

3.1.3 Functional Block Diagram

This block diagram shows only the most important architectural features. To maintain legibility some aspects have been simplified. More details of architecture, concepts and block integration are contained in other chapters of this specification.

Figure 3.2 shows a simplified block diagram of the chip.

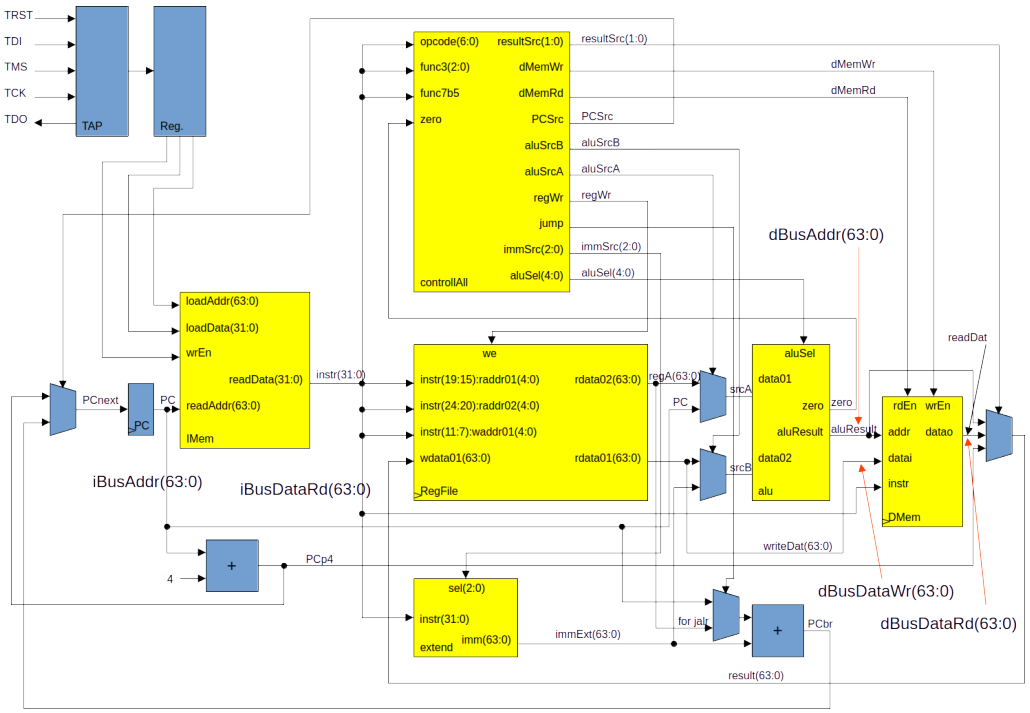


Figure 3.2: Simplified Block diagram

Figure 3.3 shows the top-level functional partitioning of the CPU core. Each dashed block is detailed in a dedicated diagram in the following sections.

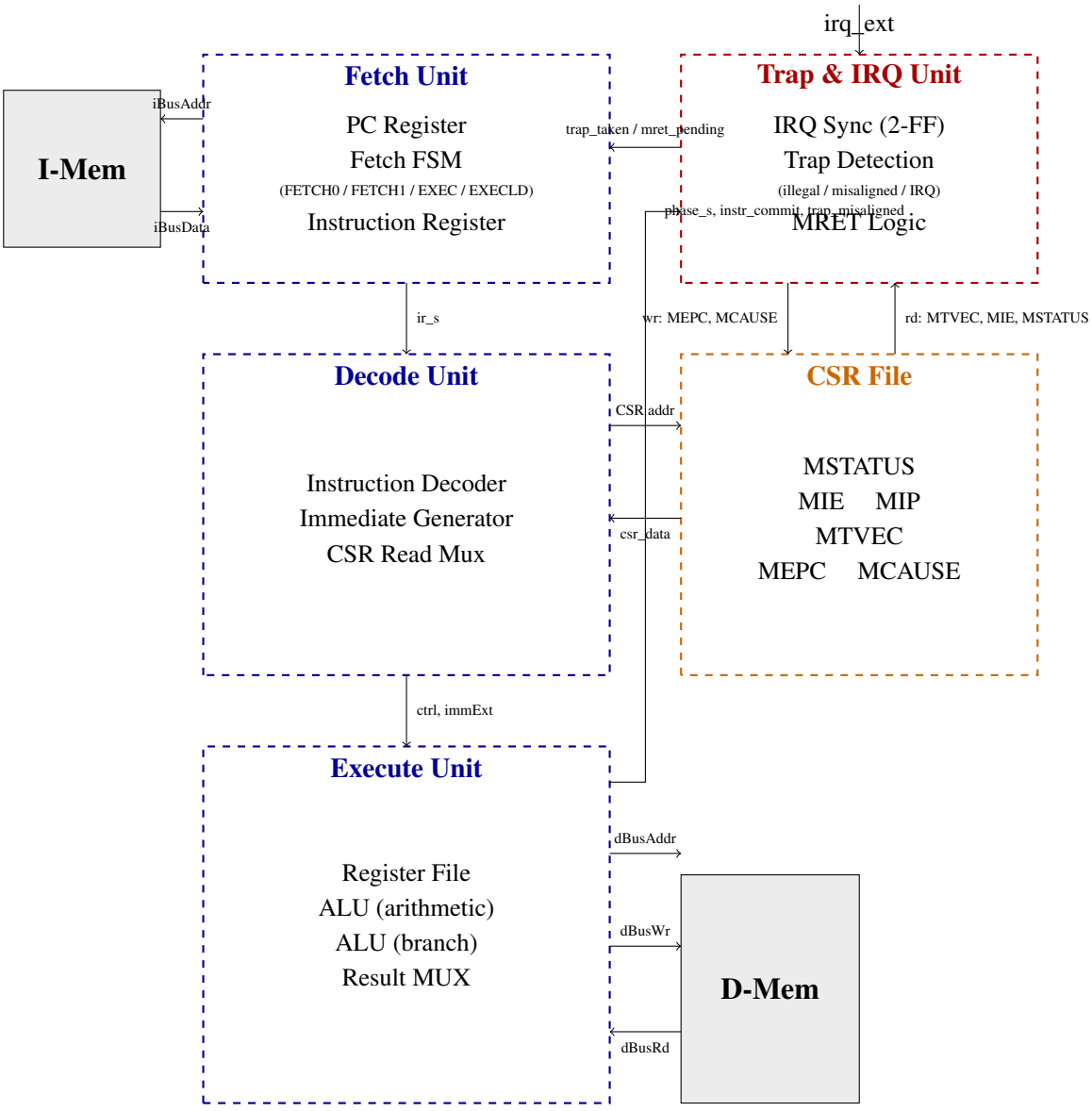


Figure 3.3: CPU Core – Top-Level Functional Block Diagram

3.1.4 Package

The package is not defined yet.

3.2 System View

Table 3.2: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

The system is not defined yet.

3.3 Architecture Concepts Overview

Table 3.3: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)
	2026-07-12	First draft (AS)
	2026-07-14	Documented `ecall`/`ebreak` exceptions (mcause 11/3), both variants (AS)

3.3.1 Technology

X-FAB **XH035** 350 nm (0.35 µm) technology.

X-FAB's XH035 is a 0.35 µm Modular Mixed-Signal / High-Voltage CMOS technology, designed for combined analog, digital, and mixed-signal circuit design. The digital standard-cell library used for this design is the `D_CELLS` library (`xh035-D_CELLS-liberty_MOS`), which supplies the Liberty (`.lib`) timing and power views for synthesis. On-chip SRAM (I-Cache, D-Cache, and Scratchpad; see Section 5.3) is generated with the X-FAB XH035 SPRAM (Single-Port RAM) compiler, available directly from the X-FAB design-kit portal.

The nominal core voltage is 3.3 V (a 2.2 V low-voltage core option exists but is not used in this design).

3.3.2 System Memory Concept

Two on-chip memory variants are defined (see Table 4.29 and Section 4.8.1):

- **Variant A (No-Cache):** The system memory consists of two on-chip SRAMs. The Instruction Memory (32 kB) is loaded at start-up via the JTAG scan chain and is read-only during normal execution. The Data Memory (8 kB) is fully read/write and serves as the sole writable data store (stack, heap, `.data`, `.bss`).
- **Variant B (Pipeline-Cache):** The system memory consists of an off-chip NOR Flash (code and read-only data) plus two on-chip caches and one on-chip Scratchpad SRAM. The I-Cache (4 kB) caches instruction fetches from Flash. The D-Cache (4 kB) caches load accesses to Flash; stores to the Flash address region are not permitted and cause a bus error. The Scratchpad SRAM (8 kB) is the sole writable data store and is accessed directly (cache bypass) for all load/store operations that target its address range. The NOR Flash is accessed through the on-chip QSPI controller via the AXI4 cache-fill path.

3.3.3 Software Architecture

The RISC-V compiler ??? is supported.

3.3.4 Interprocessor Communication Concept

Planned: UART and (Q)SPI.

3.3.5 Bus Concept

The main bus system is compliant to the Wishbone specification.

There is on WB bus for the instructions and one for the data.

3.3.6 Clock Concept

Not defined yet.

Until now, there is only one clock input. For a next generation, with external NOR flash memory for instruction and NAND flash for data, a QSPI interface will be needed. This requires a faster clock than the CPU clock.

3.3.7 System Control Concept

System control will be needed for:

- Reset and boot
- Interrupt

3.3.8 Interrupt Concept

The **RWU-RV64I** implements a minimal **single external interrupt source, machine-mode-only** interrupt concept, identical in both chip variants (`as_cpux` in `RV_NoPipeline` and `cc_cpupipefwd` in `RV_PipelineCacheFwd` implement the same CSR/-trap logic). There is no timer interrupt (no `mtime`/`mtimecmp`) and no software interrupt (`MSIP`); only the RISC-V **Machine External Interrupt** (MEI, `mcause` = `0x800...B`, cause code 11) is implemented.

Peripheral-to-CPU path. The top level (`as_top_mem`) presents an 8-bit vector, `irq_ext_i` (`irq_total_num_ext_c` = 8), to the CPU. Of these 8 bits, **only bit 7 is actually wired into the CPU's interrupt logic**: the core synchronises `irq_ext_i[7]` through a 2-flip-flop synchroniser and feeds it directly into `mip.MEIP`. Bits `[6:0]` are declared but not consumed by either CPU implementation (marked `UNUSEDSIGNAL` in `asCPUx.sv`). Bit 7 is driven by **ICU0** (Section 6.5), a Wishbone-addressable interrupt controller that aggregates GPIO0, UART0, and (on `RV_PipelineCacheFwd`) QSPI0 with software-configurable priority and per-source masking, plus 5 further source slots reserved for future peripherals (Table 6.75). GPIO0's own interrupt chain (per-pin change detection → `IRQSS`/`IRQSM` → single-bit `RIS[0]` → `IMSC[0]` → `MIS[0]` = `gpio_irq_o`) collapses all GPIO pins onto one line before it ever reaches ICU0; see Section 6.1 for the full GPIO0 register description. On `RV_NoPipeline`, UART0's interrupt now reaches the CPU for the first time via ICU0 – previously it was wired directly to `irq_ext_i[6]`, a bit the CPU never reads, so it had no effect on program flow.

CPU-side CSR mechanism.

- `mip[11]` (MEIP) is **read-only**: it directly mirrors the synchronised `icu_irq_o` line and cannot be set or cleared by software: it stays 1 for as long as ICU0 has at least one pending, unmasked, globally-enabled source, and clears only when firmware clears the responsible source(s) via ICU0's own **ICU0_ICR** (and, typically, the underlying peripheral's own clear register too).
- `mie[11]` (MEIE) is a normal read/write CSR bit, enabled (`=1`) by default out of reset.
- An interrupt is taken when `mip[11] & mie[11] & mstatus.MIE` all hold.
- `mtvec` supports **Direct mode only** (`mtvec[1:0] = 00`); every trap – exception or interrupt alike – jumps to the same handler address, reset default `0x0000_0000_0000_7F00`. There is no vectored mode and no per-cause offset.
- On trap entry: `mepc` ← PC (faulting instruction, for illegal-instruction/misaligned-address/`ecall`/`ebreak` exceptions) or PC+4 (return address, for the external interrupt); `mstatus.MPIE` ← `mstatus.MIE`; `mstatus.MIE` ← 0; PC ← `mtvec`.
- `mret` restores `mstatus.MIE` ← `mstatus.MPIE` and sets `mstatus.MPIE` ← 1.
- **Trap priority** (highest first): illegal instruction > misaligned load/store address > `ecall`/`ebreak` > external interrupt. Only one trap is taken per cycle; a pending external interrupt is deferred behind any pending exception. In practice this ordering is never contended: `ecall`/`ebreak` are a single, unambiguous instruction encoding that cannot simultaneously be illegal or trigger a misaligned load/store.

Synchronous exceptions: `ecall` and `ebreak`. Besides the illegal-instruction and misaligned-address traps, both CPU variants decode the two mandatory RV64I `SYSTEM` exceptions directly:

- `ecall` (`funct3 = 000`, `imm[11:0] = 0x000`) sets `mcause` = 11 (*Environment call from M-mode*).
- `ebreak` (`funct3 = 000`, `imm[11:0] = 0x001`) sets `mcause` = 3 (*Breakpoint*).

Both traps behave like the illegal-instruction/misaligned-address case: `mepc` points at the `ecall`/`ebreak` instruction itself (not PC+4), `mstatus.MPIE`/`MIE` update the same way, and `mret` returns control to the trap handler's chosen resume address. Neither instruction has any side effect beyond the trap – there is no environment-call ABI and no hardware breakpoint comparator; both exist purely to enter the trap handler, which is what `riscv-arch-test` compliance and a future GDB-stub breakpoint mechanism require. In `RV_NoPipeline` (`as_cpux`), the redirect to `mtvec` is triggered one cycle early via `ecbrk_pending_s` against the live I-Mem prefetch, mirroring how `mret_pending_s` already handles `mret` – otherwise the design's always-ack instruction prefetch would transiently execute one stale, wrong-path instruction before the redirect lands. `RV_PipelineCacheFwd` (`cc_cpupipefwd`) needs no such trick: its I-Cache uses an explicit req/rvalid handshake, so the ordinary commit-stage flush already used for illegal-instruction/misaligned traps is sufficient.

Open items. ICU0 provisions 8 source slots but only 3 are wired to real hardware on either chip (GPIO0, UART0, and – `RV_PipelineCacheFwd` only – QSPI0); connecting a future interrupt-capable peripheral only requires wiring it into one of the 5 remaining slots (Table 6.75), not a redesign of the register map. Both CPUs still support `mtvec` Direct mode only – ICU0's **ICU0_IRQID** register lets firmware branch on cause in software from the one shared handler, but hardware-vectored dispatch (a per-cause jump table) would require CPU changes outside ICU0's scope and remains unimplemented. Nested/preemptive interrupt handling (a higher-priority interrupt preempting an in-progress lower-priority ISR) is likewise not implemented; RISC-V M-mode already precludes it at the CPU level (`mstatus.MIE` is cleared on trap entry) independent of ICU0.

3.3.9 Debug Concept

Must be implemented.

3.3.10 Power Management

Must be implemented.

3.3.11 Protection and Security Concept

Must be implemented.

3.4 Functional Block Overview

Table 3.4: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.4.1 RISC-V Core

The core is a standard compliant RISC-V with 64 bit and integer implementation only. Two core implementations exist (see Section 4.2 and Table 4.29): `as_cpux` (Variant A, single cycle, no cache or memory hierarchy) and `cc_cpupipefwd` (Variant B, 5-stage pipeline with forwarding, backed by the I-Cache/D-Cache memory hierarchy).

3.4.2 Instruction Memory

Instruction memory implementation depends on the chip variant:

- **Variant A (No-Cache):** A 32 kB on-chip SRAM (`asIMem`) holds 8 192 RISC-V 32-bit instructions. It is loaded via the JTAG TAP before execution and is read-only during CPU operation. The CPU accesses it through a legacy flat bus interface (`iBusAddr_o`, `iBusDataRd_i`).
- **Variant B (Pipeline-Cache):** There is no on-chip instruction memory. Instructions reside on off-chip NOR Flash and are fetched into the 4 kB I-Cache on demand. Cache misses are served by a 4-beat AXI4 INCR burst through the Memory Arbiter and QSPI controller. The CPU instruction fetch interface is `as_icache_if`.

3.4.3 Data Memory

Data memory implementation depends on the chip variant:

- **Variant A (No-Cache):** An 8 kB on-chip SRAM (`asDMemTop`) serves as the data memory. It is connected to the CPU via a Wishbone B3 slave interface and supports byte, halfword, word, and doubleword accesses. Address range: `0x0000_0000`–`0x0000_1FFF`.
- **Variant B (Pipeline-Cache):** There is no traditional on-chip data memory. The data address space is split into two regions:
  - **Flash region** (addresses below the Scratchpad base): read-only data cached in the 4 kB D-Cache. Only load misses trigger cache-line fills from NOR Flash. Store operations to this region are not permitted and generate a bus error (`dc_err_o`).
  - **Scratchpad region** (`SP_BASE` = `0x0000_0000`, size = 8 kB by default): an on-chip SRAM accessed directly with byte granularity, bypassing the D-Cache. This region holds `.data`, `.bss`, heap, and stack.

3.4.4 Wishbone Peripherals

GPIO

There are 8 GPIO pins implemented. Together with the data, an address will be given to the pins. With this, 16 times 8 GPIOs can be driven with just one external decoder.

3.4.5 Debug

The test and debug will be done via a JTAG interface. Until now, the loading of the instruction memory is possible.

Chapter 4

Architecture Concepts

The logic style used here is as often as possible a "one pulse" logic.

Figure 4.1 shows the "pulsed" logic.

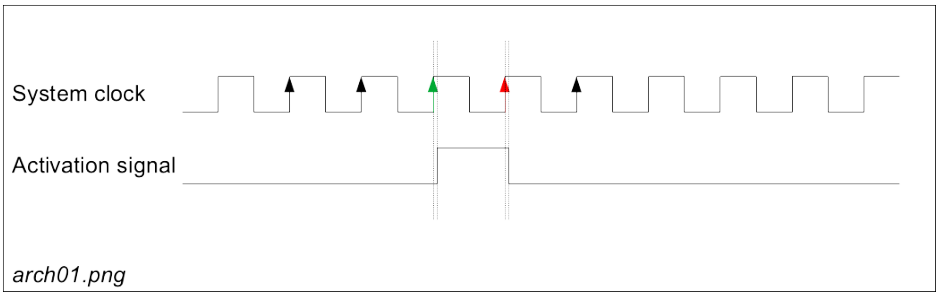


Figure 4.1: Enable with a Pulse

An "activation signal" will be generated synchronously with the "system clock" by its rising edge (green). This results in a delayed generation of the rising edge of the "activation signal". With the next rising edge of the "system clock" (red), the "activation signal" will be removed. Also here, the falling edge of the "activation signal" will have a small delay to the clock. So, the "activation signal" is exactly on "system clock" period high. This pair of signals can now be taken to start a following device (e.g. a counter), the counter will count only when the activation signal is high and the system clock will show a rising edge (red).

4.1 Technology

Table 4.1: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.2 CPU Concept

Table 4.2: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

The **RWU-RV64I** project is a students test chip and comprises two distinct CPU/memory variants (RTL tops `RV_NoPipeline` and `RV_PipelineCacheFwd`; see Table 4.29), neither of which implements an MMU:

- **RTL: `RV_NoPipeline`** – a single-cycle implementation without pipeline or cache; every instruction completes in one clock cycle; on-chip SRAM only (no memory hierarchy).
- **RTL: `RV_PipelineCacheFwd`** – a 5-stage pipelined implementation (IF/ID/EX/MEM/WB) with EX/MEM→EX and MEM/WB→EX forwarding, backed by the I-Cache/D-Cache memory hierarchy described in Section 4.8.3. Forwarding eliminates all non-load RAW stalls; only the load-use hazard (1 cycle) and CSR hazards still stall the pipeline.

Table 4.3 shows the notations which are used in the explanations for the instructions in tables 4.4 and 4.5.

Table 4.3: Notations for Instructions

Notation	Description
pc	program counter
rd	integer register destination
rsN	integer register source N
imm	immediate operand value
offs	immediate program counter relative offset
ux(reg)	unsigned XLEN-bit integer (32-bit on RV32, 64-bit on RV64)
sx(reg)	signed XLEN-bit integer (32-bit on RV32, 64-bit on RV64)
uN(reg)	zero extended N-bit integer register value
sN(reg)	sign extended N-bit integer register value
uN[reg + imm]	unsigned N-bit memory reference
sN[reg + imm]	signed N-bit memory reference

Table 4.4 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Table 4.4: RV32I Base Integer Instruction Set [2], [3], [1]

Instruction	Example	Meaning
Load Upper Immediate	*lui rd, imm*	$rd \leftarrow imm$
Add Upper Immediate to PC	*auipc rd, offs*	$rd \leftarrow pc + offs$
Jump and Link	*jal rd, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow pc + offs$
Jump and Link Register	*jalr rd, rs1, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow (rs1 + offs) \wedge -2$
Branch Equal	*beq rs1, rs2, offs*	$if\ rs1 = rs2\ then\ pc \leftarrow pc + offs$
Branch Not Equal	*bne rs1, rs2, offs*	$if\ rs1 \neq rs2\ then\ pc \leftarrow pc + offs$

... next page

Instruction	Example	Meaning
Branch Less Than	*blt rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal	*bge rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than Unsigned	*bltu rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal Unsigned	*bgeu rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Load Byte	*lb rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half	*lh rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Load Word	*lw rd, offs(rs1)*	$rd \leftarrow s32[rs1 + offs]$
Load Byte Unsigned	*lbu rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half Unsigned	*lhu rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Store Byte	*sb rs2, offs(rs1)*	$u8[rs1 + offs] \leftarrow rs2$
Store Half	*sh rs2, offs(rs1)*	$u16[rs1 + offs] \leftarrow rs2$
Store Word	*sw rs2, offs(rs1)*	$u32[rs1 + offs] \leftarrow rs2$
Add Immediate	*addi rd, rs1, imm*	$rd \leftarrow rs1 + sx(imm)$
Set Less Than Immediate	*slti rd, rs1, imm*	$rd \leftarrow sx(rs1) < sx(imm)$
Set Less Than Immediate Unsigned	*sltiu rd, rs1, imm*	$rd \leftarrow ux(rs1) < ux(imm)$
Xor Immediate	*xori rd, rs1, imm*	$rd \leftarrow ux(rs1) \oplus ux(imm)$
Or Immediate	*ori rd, rs1, imm*	$rd \leftarrow ux(rs1) \vee ux(imm)$
And Immediate	*andi rd, rs1, imm*	$rd \leftarrow ux(rs1) \wedge ux(imm)$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << ux(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> ux(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> ux(imm)$
Add	*add rd, rs1, rs2*	$rd \leftarrow sx(rs1) + sx(rs2)$
Subtract	*sub rd, rs1, rs2*	$rd \leftarrow sx(rs1) - sx(rs2)$
Shift Left Logical	*sll rd, rs1, rs2*	$rd \leftarrow ux(rs1) << rs2$
Set Less Than	*slt rd, rs1, rs2*	$rd \leftarrow sx(rs1) < sx(rs2)$
Set Less Than Unsigned	*sltu rd, rs1, rs2*	$rd \leftarrow ux(rs1) < ux(rs2)$
Xor	*xor rd, rs1, rs2*	$rd \leftarrow ux(rs1) \oplus ux(rs2)$

... next page

Instruction	Example	Meaning
Shift Right Logical	*srl rd, rs1, rs2*	$rd \leftarrow ux(rs1) >> rs2$
Shift Right Arithmetic	*sra rd, rs1, rs2*	$rd \leftarrow sx(rs1) >> rs2$
Or	*or rd, rs1, rs2*	$rd \leftarrow ux(rs1) \vee ux(rs2)$
And	*and rd, rs1, rs2*	$rd \leftarrow ux(rs1) \wedge ux(rs2)$
Fence	*fence pred, succ*	
Fence Instruction	*fence.i*	

End of table.

Table 4.5 lists all base integer instructions of a RISC-V 64 bit architecture, additional to the 32 bit integer instruction set, according the specification [1].

Table 4.5: RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]

Instruction	Example	Meaning
Load Word Unsigned	*lwu rd, offs(rs1)*	$rd \leftarrow u32[rs1 + offs]$
Load Double	*ld rd, offs(rs1)*	$rd \leftarrow u64[rs1 + offs]$
Store Double	*sd rs2, offs(rs1)*	$u64[rs1 + offs] \leftarrow rs2$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << sx(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> sx(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> sx(imm)$
Add Immediate Word	*addiw rd, rs1, imm*	$rd \leftarrow s32(rs1) + imm$
Shift Left Logical Immediate Word	*slliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) << imm)$
Shift Right Logical Immediate Word	*srliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) >> imm)$
Shift Right Arithmetic Immediate Word	*sraiw rd, rs1, imm*	$rd \leftarrow s32(rs1) >> imm$
Add Word	*addw rd, rs1, rs2*	$rd \leftarrow s32(rs1) + s32(rs2)$
Subtract Word	*subw rd, rs1, rs2*	$rd \leftarrow s32(rs1) - s32(rs2)$

... next page

Instruction	Example	Meaning
Shift Left Logical Word	*sllw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) << rs2)$
Shift Right Logical Word	*srlw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) >> rs2)$
Shift Right Arithmetic Word	*sraw rd, rs1, rs2*	$rd \leftarrow s32(rs1) >> rs2$

End of table.

4.2.1 Pipeline Hazard Handling (`RV_PipelineCacheFwd` only)

`RV_NoPipeline` has no hazards to resolve (every instruction completes in a single cycle before the next one starts); this section applies only to `RV_PipelineCacheFwd`'s 5-stage pipeline (`cc_cpupipefwd.sv`, IF/ID/EX/MEM/WB).

Forwarding Paths

Two forwarding paths resolve read-after-write (RAW) hazards without stalling, feeding the EX-stage ALU operand muxes directly from later pipeline stages instead of the (stale) value latched in the ID/EX register:

- **EX/MEM → EX** (1 instruction back): the ALU result (or PC+4 / CSR read value) held in `ex_mem_aluRes_r` / `ex_mem_csr_rdata_r` / `ex_mem_pc_r` is forwarded when the MEM-stage instruction writes a register the EX-stage instruction reads. Excluded for loads: load data is not yet available at this point (see load-use stall below).

- **MEM/WB → EX** (2 instructions back): the final write-back value `wb_result_s` (already muxed between ALU result, load data, PC+4, and CSR read value) is forwarded when the WB-stage instruction writes a register the EX-stage instruction reads. This path also carries load data, so a load-to-use hazard 2 instructions apart is resolved without a stall.

A third RAW distance – **k=3** (producer in WB, consumer in ID, same clock edge) – needs no explicit forwarding mux: `as_regfile` is instantiated with `WBR=1`, which bypasses the write data combinatorially to the read ports within the same cycle, so `id_regA_s`/ `id_regB_s` already see the new value when both events coincide.

Remaining Stall Sources

Two hazard classes cannot be resolved by forwarding and still stall the pipeline (freeze all stages via `global_stall_s`, or insert a bubble via `data_stall_s`):

- **Load-use (1 cycle):** a `LOAD` is in EX (`id_ex_dMemRd_r`=1) and the very next instruction (in ID) reads the register it will load into. The load's result is only ready at the end of the following MEM stage, too late to forward to a consumer already in EX at the same time. The pipeline inserts one bubble (NOP into EX, consumer held in ID); the next cycle the load reaches WB and the consumer reaches EX, where MEM/WB $\rightarrow$ EX forwarding resolves it.

- **CSR hazard (conservative):** CSR writes commit at WB and there is no forwarding path for CSR register values. Any CSR instruction in ID is held (`data_stall_s`) until no earlier CSR instruction remains anywhere in the pipeline.

Control Hazards

A taken branch or an unconditional jump (JAL/JALR) is only resolved in EX, after the two following instructions have already been speculatively fetched into IF and ID. `ex_branch_taken_s` (branch) and `wb_trap_taken_s`/`wb_mret_s` (trap/`mret`) each assert `flush_s`, which squashes the IF/ID and ID/EX pipeline registers (2-cycle bubble) regardless of `global_stall_s` – see Section 7.1.2 for why that last part specifically had to be fixed.

Hazard Test Suite

Five directed ASM tests (`rv64Sim/hazard_*.asm`, run under `tb_rv64i_instr.sv` like the base ISA tests) exercise every hazard class above:

Table 4.6: Pipeline Hazard Test Suite

Test	Coverage
`hazard_raw_alu`	ALU→ALU RAW at distance k=1 (EX/MEM→EX), k=2 (MEM/WB→EX), k=3 (regfile WBR bypass); no stall expected for any of the three.
`hazard_raw_load`	Three load-use RAW hazards; each forces exactly one 1-cycle stall.
`hazard_waw_war`	Write-after-write (second write wins) and write-after-read (read observes the old value); both are naturally transparent in this in-order, single-issue pipeline.
`hazard_ctrl_branch`	Two taken branches (2-cycle flush each) plus one not-taken branch (no flush).
`hazard_ctrl_jump`	Forward `JAL` flush, backward `JAL` loop, and a `JAL`+`JALR` subroutine call/return pair.

A separate statistics testbench, `tb_rv64i_hazard_stat.sv`, re-runs each of the five programs and counts `clk_div_s` cycles from reset de-assertion to the program's final (successful) GPIO write, reporting `cycles`, `instr`, and `CPI` (see the regression log excerpt in Section 7.1); this is a performance/regression indicator, not a pass/fail correctness check – correctness is still the GPIO = 0x55 comparison every test in this suite uses.

4.3 Bus Concept

Table 4.7: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22 2026-02-19	First draft (AS) First fill in cooperation with Claude-AI (AS)

4.3.1 Overview

The RWU-RV64I chip uses a **dual-bus architecture** with two independent on-chip buses serving different purposes:

- **Wishbone B4 Classic** – the peripheral bus. All configuration registers, GPIO, UART, and QSPI control registers are connected to the CPU via this bus. It was chosen for its simplicity, open-source IP availability, and compatibility with the existing peripheral IP blocks.

- **AMBA AXI4** (read-only subset) – the cache memory bus. The I-Cache and D-Cache controllers use this bus to fetch 32-byte cache lines from NOR Flash via the Memory Arbiter and the QSPI controller data port. AXI4 was chosen for its burst capability: a 4-beat INCR burst fills one cache line in a single transaction instead of eight independent Wishbone single-cycle transfers.

The two buses are completely independent. There is no bridge between them. The Wishbone bus carries only single-word register accesses; the AXI4 bus carries only 32-byte cache-line bursts. Table 4.8 summarises the key parameters of each bus.

Table 4.8: On-Chip Bus Summary

Property	Wishbone B4 Classic	AXI4 (cache subset)	Comment
Purpose	Peripheral access	Cache line fill	
Standard	Wishbone B4 Classic	AMBA AXI4	
Data width	64 bit	64 bit	
Address width	64 bit	32 bit	Physical address only
Transfer type	Single-word	4-beat INCR burst	ARLEN=3, ARSIZE=3
Burst support	No	Yes (read only)	Write channels eliminated
Masters	CPU core	I-Cache, D-Cache	
Slaves	GPIO, UART, QSPI cfg regs	QSPI controller (data)	
Arbitration	Single master (no arbiter)	Fixed priority (D > I)	Memory Arbiter
Error signalling	None (ERR_O not driven)	RRESP≠OKAY	Maps to RISC-V exceptions

4.3.2 Bus Topology

Figure 4.2 shows the complete on-chip bus topology. The two buses are rendered in different colours: Wishbone in blue, AXI4 in red. The QSPI controller is the only block that participates in both buses (configuration registers on Wishbone, data port on AXI4).

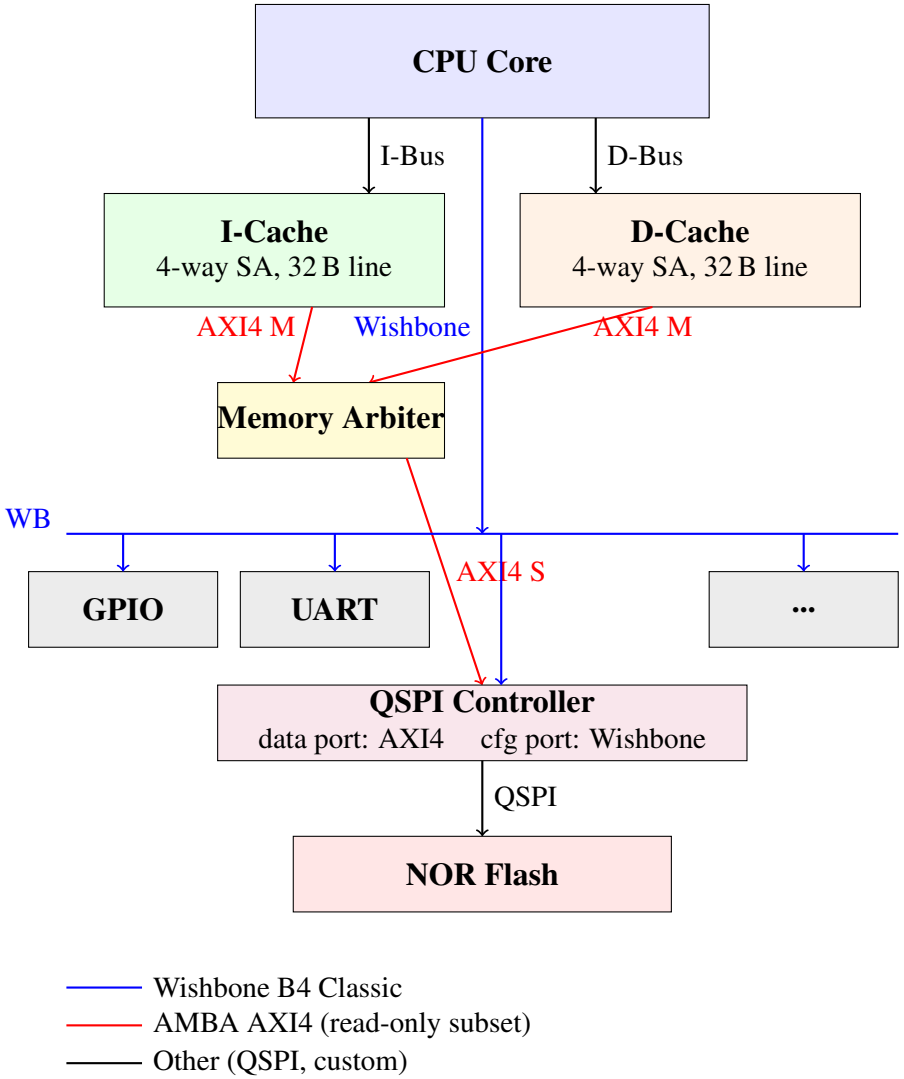


Figure 4.2: On-Chip Bus Topology – Dual-Bus Architecture

4.3.3 Bus Selection Rationale

Table 4.9 documents the decision to use a dual-bus architecture rather than a single unified bus for all on-chip traffic.

Table 4.9: Bus Selection Rationale

Criterion	Wishbone (peripherals)	AXI4 (cache)
Transfer pattern	Single register read/write; one word per transaction	32-byte cache line fill; 4 beats per transaction
Burst support	Not needed; register accesses are always single-word	Essential; without bursts, 8 independent transactions per cache miss instead of 1
Latency	Zero wait states (combinatorial ACK); adequate for register accesses	Many cycles per miss; decoupled AR and R channels allow the QSPI controller to accept the address and then stream data without stalling the bus
Complexity	Very simple; no state machines required in the BPI	Higher; separate address/-data channels, handshaking, ID tracking – justified by the burst benefit
IP compatibility	All existing peripheral IP blocks use Wishbone; no changes required	Cache controllers are new IP; designed for AXI4 from the start
Write support	Full read/write (register access)	Read-only (write channels eliminated by MEM-01 resolution; Flash is read-only)
Decision	**Keep Wishbone for all peripherals.** Adding AXI4 to GPIO/UART etc. would add complexity with no benefit.	**Use AXI4 for cache–memory path.** The 8× reduction in bus transactions per cache miss justifies the added complexity.

4.3.4 Wishbone B4 Classic – Peripheral Bus

Signal Naming Convention

Wishbone signal names in this design follow the convention shown in Table 4.10. The suffix `_i` indicates an input, `_o` an output, and `_s` an internal wire. The prefix `wb_s_` is used for slave-side signals and `wb_m_` for master-side signals.

Table 4.10: Wishbone Signal Naming

Wishbone Name	Slave Port	Master Port	Width	Description
ADR	wb_s_addr_i	wb_m_addr_o	64	Address bus
DAT_I	wb_s_dat_i	wb_m_dat_i	64	Data from master to slave (write data)
DAT_O	wb_s_dat_o	wb_m_dat_o	64	Data from slave to master (read data)
WE	wb_s_we_i	wb_m_we_o	1	Write enable: 1 = write, 0 = read
SEL	wb_s_sel_i	wb_m_sel_o	8	Byte select (all bytes must be set)
STB	wb_s_stb_i	wb_m_stb_o	1	Strobe: valid cycle indicator from master
ACK	wb_s_ack_o	wb_m_ack_i	1	Acknowledge: normal end of transfer from slave
CYC	wb_s_cyc_i	wb_m_cyc_o	1	Cycle: bus cycle active (held for complete transfer)

Transfer Protocol

A Wishbone Classic single transfer proceeds as follows:

1. The master asserts CYC and STB, presents a valid address on ADR, drives WE (1 for write, 0 for read), sets SEL to all-ones, and – for a write – provides data on DAT_O.

2. In this implementation the slave responds combinatorially in the same clock cycle: it drives read data on `DAT_O` and asserts `ACK`.

3. The transaction is complete when both `STB` and `ACK` are seen high on the rising clock edge. The master may then begin a new transfer in the very next cycle.

A transfer is considered valid only when both `CYC` and `STB` are asserted simultaneously and all byte-select bits are set. If any byte-select bit is clear the BPI treats the access as invalid and suppresses both the ACK and the write enable to the peripheral.

The timing diagram below illustrates a write followed by a read.

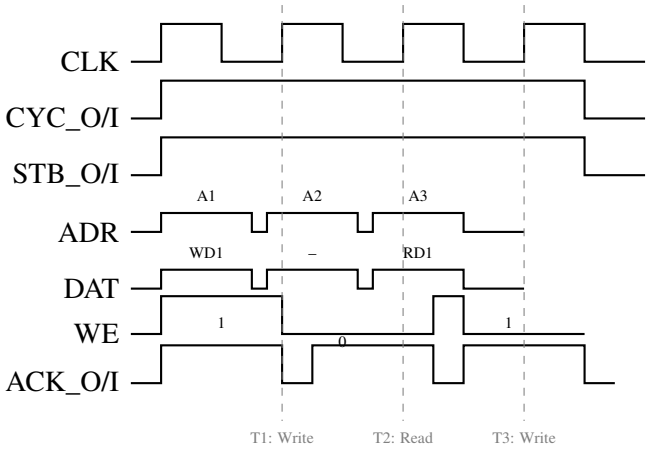


Figure 4.3: Wishbone Classic Single Transfers: Write – Read – Write

Byte Select

The 8-bit `SEL` signal indicates which byte lanes carry valid data. Bit n of `SEL` corresponds to byte lane n (bits $8n + 7 \ldots 8n$) of the 64-bit data bus. In the current implementation, the master BPI always drives all eight `SEL` bits to 1 (full-word transfers only). The slave BPI checks that all `SEL` bits are set before asserting `ACK` or issuing a write enable to the peripheral kernel. Sub-word accesses are therefore not supported.

Reset Behaviour

All Wishbone outputs are driven to their inactive state while the **asynchronous**, active-high reset `rst_i` is asserted (every clocked block in this design uses

`always_ff @(posedge clk_i, posedge rst_i)`; there is no synchronous-reset flip-flop anywhere in the RTL). For the master BPI this means `STB_O` and `CYC_O` are held low; for the slave BPI `ACK_O` is held low. Note that in the current purely combinatorial implementation the reset is not registered inside the BPI modules; the reset is applied by the surrounding peripheral logic.

4.3.5 AMBA AXI4 – Cache Memory Bus

Overview

The AXI4 bus connects the two cache controllers (I-Cache and D-Cache) through the Memory Arbiter to the QSPI controller data port. It carries exclusively 32-byte cache-line read bursts. Write channels (AW, W, B) are not instantiated: the AXI4 write path was eliminated when the scratchpad SRAM replaced write-back-to-Flash (MEM-01 resolution).

Participants

Table 4.11: AXI4 Bus Participants

Block	Role	Notes
I-Cache controller	AXI4 Master (ID=0x1)	Issues read burst on cache miss; read-only
D-Cache controller	AXI4 Master (ID=0x2)	Issues read burst on cache miss; read-only
Memory Arbiter	AXI4 Slave + Master	Accepts two masters; forwards one at a time to QSPI; fixed priority D-Cache > I-Cache
QSPI controller (data port)	AXI4 Slave	Translates AXI4 read burst to Quad Fast Read SPI command; slow slave (many cycles per burst)

Bus Parameters

The AXI4 bus uses a reduced parameter set: all transfers are fixed-length 4-beat INCR read bursts.

Table 4.12: AXI4 Cache Bus Parameters

Parameter	Value	Comment
Protocol	AXI4 (not AXI4-Lite)	Burst support required
Data width	64 bit	One RV64 doubleword per beat
Address width	32 bit	Physical address (`PA_WIDTH=32`)
ID width	4 bit	I-Cache: `4'h1`; D-Cache: `4'h2`
Burst type	INCR only	Sequential address; no WRAP, no FIXED
Burst length	ARLEN = 3	4 beats × 8 B = 32 B per cache line
Beat size	ARSIZE = 3	8 bytes per beat
Channels used	AR, R only	AW, W, B channels not instantiated
Outstanding tx	1 per master	Each cache has at most 1 transaction in flight

The full AXI4 channel signal definitions, the SystemVerilog interface `as_axi4_if`, and the AXI4 timing diagrams are documented in the Memory Concept section (Section “Memory Concept”, subsection “AXI4 Memory Bus”).

QSPI Controller – Dual Bus Participation

The QSPI controller is the only block that participates in both buses:

- **AXI4 slave port (data):** Receives 4-beat INCR read bursts from the Memory Arbiter. Translates each burst into a Quad Fast Read (`0x6B`) command sequence on the SPI bus and returns 32 bytes in 4 AXI4 R-channel beats. This port is part of the AXI4 cache memory bus.

- **Wishbone slave port (configuration):** Receives single-word register accesses from the CPU for QSPI mode configuration, status readback, and manual SPI command injection. This port is part of the Wishbone peripheral bus and uses the standard `as_slave_bpi` wrapper.

The two ports are independent and may be active simultaneously. The QSPI controller arbitrates internally between the AXI4 data path and the Wishbone configuration path, with the AXI4 burst taking priority over a concurrent Wishbone access.

4.3.6 Wishbone Slave BPI (`as_slave_bpi`)

History

Table 4.13: History of as_slave_bpi

Version	Date	Author	Change
0.1	2026-02-19	Claude-AI	Initial version, Wishbone B4 Classic, zero wait states

Functional Configuration of as_slave_bpi

The `as_slave_bpi` module implements the Wishbone B4 Classic slave interface. It converts Wishbone bus signals into simple register-file read and write enables for the peripheral kernel. Table 4.14 lists the configurable parameters.

Table 4.14: Functional Configuration of as_slave_bpi

Parameter	Type	Default	Description
`addr_width`	integer	64	Width of the address bus in bits
`data_width`	integer	64	Width of the data bus in bits; also determines SEL width as `data_width/8`

HW Interfaces

Table 4.15 lists all input and output ports of the `as_slave_bpi` module.

Table 4.15: I/O of as_slave_bpi

Port	Direction	Width	Description
rst_i	in	1	Unused internally (purely combinatorial BPI); kept for Wishbone port symmetry – see Section 4.3.4
clk_i	in	1	Bus clock
Peripheral kernel side:			
addr_o	out	64	Decoded register address to peripheral
dat_from_core_i	in	64	Read data returned by peripheral kernel
dat_to_core_o	out	64	Write data forwarded to peripheral kernel
wr_o	out	1	Write enable to peripheral: asserted for one cycle on a valid write
rd_o	out	1	Read enable to peripheral: asserted for one cycle on a valid read
Wishbone bus side:			
wb_s_addr_i	in	64	Address from Wishbone master
wb_s_dat_i	in	64	Write data from Wishbone master
wb_s_dat_o	out	64	Read data to Wishbone master
wb_s_we_i	in	1	Write enable from Wishbone master
wb_s_sel_i	in	8	Byte select from Wishbone master
wb_s_stb_i	in	1	Strobe from Wishbone master
wb_s_ack_o	out	1	Acknowledge to Wishbone master
wb_s_cyc_i	in	1	Cycle from Wishbone master

Bus Interface

Wishbone Classic Slave.

Registers

The as_slave_bpi module contains no registers of its own. All logic is purely combinatorial. Registers are located in the peripheral kernel that is connected via

the kernel-side interface of this module.

Related Sections of this Specification

- Wishbone Master BPI Section 4.3.7
- QSPI Peripheral Section 6.3, which uses this BPI
- GPIO Peripheral Section 6.1, which uses this BPI

Functional Overview

The slave BPI converts two Wishbone control signals plus the byte-select into two conditions used throughout the peripheral:

$$\textit{valid_transfer} = \texttt{CYC} \wedge \texttt{STB}$$

$$\textit{all_sel} = \& \ \texttt{SEL}_{7:0}$$

From these two conditions the write and read enables to the peripheral kernel are derived:

$$\texttt{wr_o} = \textit{valid_transfer} \wedge \texttt{WE} \wedge \textit{all_sel}$$

$$\texttt{rd_o} = \textit{valid_transfer} \wedge \neg\texttt{WE} \wedge \textit{all_sel}$$

The Wishbone acknowledge is generated combinatorially:

$$\texttt{ACK_O} = \textit{valid_transfer} \wedge \textit{all_sel}$$

Because ACK is driven combinatorially from STB and CYC, the slave introduces zero wait states. The peripheral kernel must therefore provide read data (`dat_from_core_i`) in the same clock cycle that `rd_o` is asserted.

Structural Overview

Figure 4.4 shows the internal signal flow of the slave BPI.

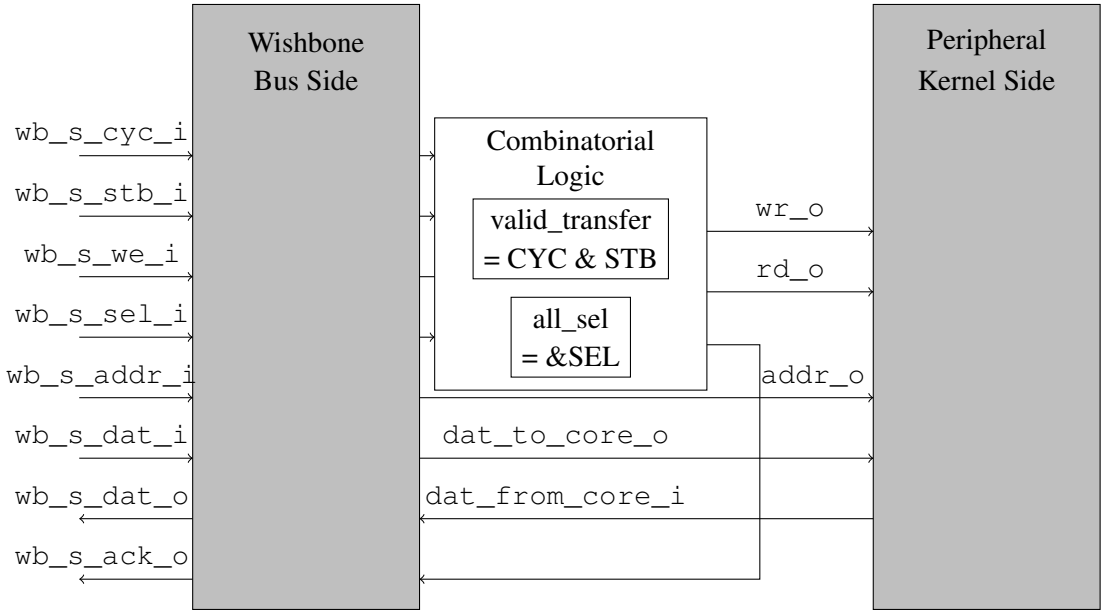


Figure 4.4: as_slave_bpi – Internal Signal Flow

Table 4.16 lists the internal signals.

Table 4.16: as_slave_bpi Internal Signals

Signal	Type	Width	Description
valid_transfer_s	logic	1	High when both CYC and STB are asserted. Indicates an active Wishbone bus cycle.
all_sel_s	logic	1	High when all byte-select bits are set. Only full 64-bit word accesses are accepted.

Instantiation

The module is parameterised and instantiated as follows:

```
as_slave_bpi #(
    .addr_width ( 64 ),
    .data_width ( 64 )
) u_qspi_bpi (
    .rst_i              ( rst_i              ),
    .clk_i              ( clk_i              ),
    // kernel side
```

```
    .addr_o             ( addr_s             ),
    .dat_from_core_i    ( dat_from_core_s    ),
    .dat_to_core_o      ( dat_to_core_s      ),
    .wr_o               ( wr_s               ),
    .rd_o               ( rd_s               ),
    // wishbone side
    .wb_s_addr_i        ( wb_s_addr_i        ),
    .wb_s_dat_i         ( wb_s_dat_i         ),
    .wb_s_dat_o         ( wb_s_dat_o         ),
    .wb_s_we_i          ( wb_s_we_i          ),
    .wb_s_sel_i         ( wb_s_sel_i         ),
    .wb_s_stb_i         ( wb_s_stb_i         ),
    .wb_s_ack_o         ( wb_s_ack_o         ),
    .wb_s_cyc_i         ( wb_s_cyc_i         )
);
```

Limitations and Design Rules

- **Zero wait states only.** The peripheral kernel must provide read data combinatorially. If the kernel uses registered outputs, a wait-state mechanism (delayed ACK) must be added externally.

- **Full-word accesses only.** Sub-word byte or half-word accesses are not supported. The `SEL` check ensures that partial accesses are rejected silently.

- **No ERR_O.** Bus errors are not signalled. Invalid accesses are simply not acknowledged (ACK stays low).

- **No burst support.** The module supports only Wishbone Classic single-cycle transfers. Pipeline or burst modes are not implemented.

- **Combinatorial ACK.** Connecting multiple slave BPIs in a shared-bus topology requires that ACK is only driven by the selected slave. Address decoding and ACK gating must be handled by the bus interconnect logic, not by this module.

4.3.7 Wishbone Master BPI (`as_master_bpi`)

History

Table 4.17: History of as_master_bpi

Version	Date	Author	Change
0.1	2026-02-19	Claude-AI	Initial version, Wishbone B4 Classic, continuous CYC/STB

Functional Configuration of as_master_bpi

The `as_master_bpi` module implements the Wishbone B4 Classic master interface. It is a simple combinatorial wrapper that translates address, data and direction signals from the CPU or DMA core into Wishbone master signals. Table 4.18 lists the configurable parameters.

Table 4.18: Functional Configuration of as_master_bpi

Parameter	Type	Default	Description
`addr_width`	integer	64	Width of the address bus in bits
`data_width`	integer	64	Width of the data bus in bits; also determines SEL width as `data_width/8`

HW Interfaces

Table 4.19 lists all input and output ports of the `as_master_bpi` module.

Table 4.19: I/O of as_master_bpi

Port	Direction	Width	Description
rst_i	in	1	Unused internally (purely combinatorial BPI); kept for Wishbone port symmetry – see Section 4.3.4
clk_i	in	1	Bus clock
CPU / master core side:			
addr_i	in	64	Address from CPU core
dat_from_core_i	in	64	Write data from CPU core
dat_to_core_o	out	64	Read data returned to CPU core
wr_i	in	1	Write enable from CPU core: 1 = write, 0 = read
Wishbone bus side:			
wb_m_addr_o	out	64	Address to Wishbone bus
wb_m_dat_i	in	64	Read data from Wishbone bus (from selected slave)
wb_m_dat_o	out	64	Write data to Wishbone bus
wb_m_we_o	out	1	Write enable to Wishbone bus
wb_m_sel_o	out	8	Byte select to Wishbone bus (always all-ones)
wb_m_stb_o	out	1	Strobe to Wishbone bus (tied high)
wb_m_ack_i	in	1	Acknowledge from Wishbone bus (from selected slave)
wb_m_cyc_o	out	1	Cycle to Wishbone bus (tied high)

Bus Interface

Wishbone Classic Master.

Registers

The as_master_bpi module contains no registers of its own. All logic is purely combinatorial.

Related Sections of this Specification

- Wishbone Slave BPI Section 4.3.6
- RWU-RV64I Core Chapter, Bus Interface subsection

Functional Overview

The master BPI is a thin combinatorial bridge. All Wishbone master outputs are driven directly from the CPU core signals without any registered state:

$$
\begin{aligned}
\texttt{wb_m_addr_o} &= \texttt{addr_i} \\
\texttt{wb_m_dat_o} &= \texttt{dat_from_core_i} \\
\texttt{wb_m_we_o} &= \texttt{wr_i} \\
\texttt{wb_m_sel_o} &= \texttt{8'hFF} \quad \text{(all bytes always valid)} \\
\texttt{wb_m_stb_o} &= 1 \\
\texttt{wb_m_cyc_o} &= 1 \\
\texttt{dat_to_core_o} &= \texttt{wb_m_dat_i}
\end{aligned}
$$

Because `STB` and `CYC` are continuously asserted, a new Wishbone transfer is initiated on every clock cycle as long as a slave is present on the bus. The transfer completes when the slave returns `ACK`. In the zero-wait-state case (slave BPI also combinatorial), ACK arrives in the same cycle as STB, so every clock cycle is a completed transfer.

The CPU core must handle the `ack_i` signal to determine when a read result is valid or when the next write can proceed.

Structural Overview

Figure 4.5 shows the internal signal flow of the master BPI.

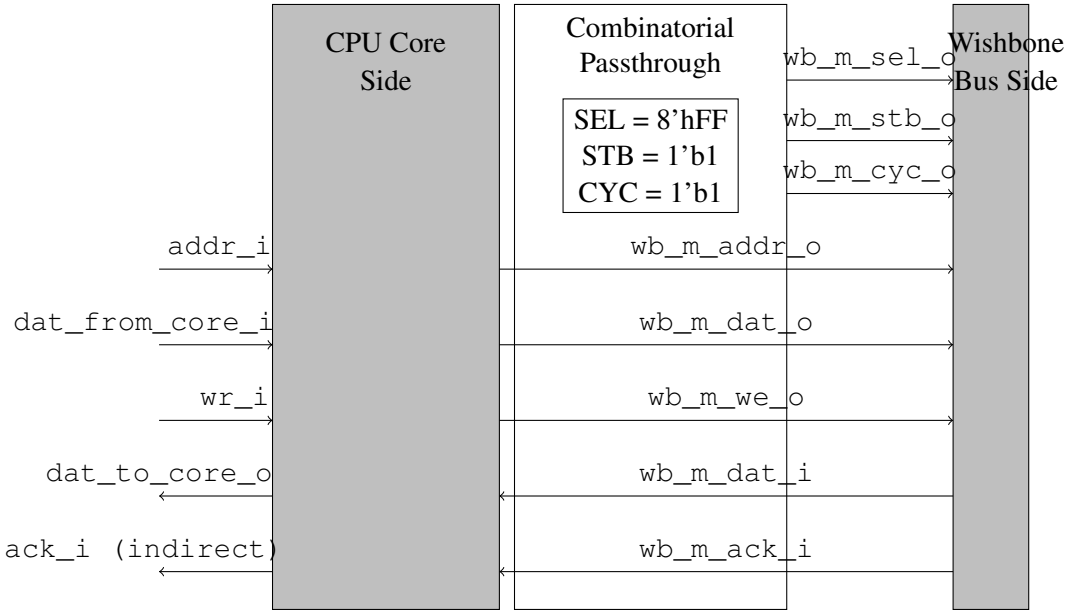


Figure 4.5: as_master_bpi – Internal Signal Flow

Table 4.20 lists the internal signals (none in this implementation, as the module is purely combinatorial):

Table 4.20: as_master_bpi Internal Signals

Signal	Type	Width	Description
(none)	–	–	All logic is combinatorial; no internal signals

Instantiation

```
as_master_bpi #(
    .addr_width ( 64 ),
    .data_width ( 64 )
) u_cpu_bpi (
    .rst_i              ( rst_i              ),
    .clk_i              ( clk_i              ),
    // CPU core side
    .addr_i             ( cpu_addr_s         ),
    .dat_from_core_i    ( cpu_wdat_s         ),
    .dat_to_core_o      ( cpu_rdat_s         ),
    .wr_i               ( cpu_wr_s           ),
    // wishbone side
```

```
    .wb_m_addr_o        ( wb_m_addr_s       ),
    .wb_m_dat_i         ( wb_m_dat_i_s      ),
    .wb_m_dat_o         ( wb_m_dat_o_s      ),
    .wb_m_we_o          ( wb_m_we_s         ),
    .wb_m_sel_o         ( wb_m_sel_s        ),
    .wb_m_stb_o         ( wb_m_stb_s        ),
    .wb_m_ack_i         ( wb_m_ack_s        ),
    .wb_m_cyc_o         ( wb_m_cyc_s        )
);
```

Limitations and Design Rules

- **Continuous bus occupation.** Because `STB` and `CYC` are permanently asserted, the master occupies the bus at all times. Multi-master arbitration requires that these signals are gated by the arbiter before being placed on the shared bus.

- **No idle state.** The master BPI does not support idle cycles between transfers. The surrounding CPU core logic must insert idle cycles if needed by controlling `addr_i` and `wr_i` appropriately and ignoring any ACK responses during idle periods.

- **ACK not evaluated internally.** The `wb_m_ack_i` signal is passed through directly to the CPU core via the bus-side port; the BPI itself does not act on it. The CPU must sample ACK and stall its pipeline if required.

- **No error handling.** There is no support for Wishbone `ERR_I` or `RTY_I`.

4.3.8 Bus Interconnect

Two Independent Buses

The Wishbone and AXI4 buses are completely separate. There is no bridge, no shared wires, and no arbitration between them. Traffic routing is determined at design time by which port of each block is connected to which bus:

- All CPU accesses to peripheral registers use the Wishbone bus.

- All cache-line fills use the AXI4 bus.

- The QSPI controller has one port on each bus; they operate independently.

Wishbone Address Map

Each Wishbone slave is mapped to a contiguous, non-overlapping region of the address space. Address decoding is performed by the bus interconnect logic, which forwards `STB` and `CYC` only to the addressed slave. Slaves whose address range is not selected must drive `ACK_O` low and `DAT_O` to zero. The actual address boundaries are defined in the chip-level memory map (Chapter "Memory Map and Registers").

Wishbone Topology

The current Wishbone implementation uses a **shared bus** topology with a single master (the CPU core). A crossbar switch is not implemented because there is currently only one Wishbone master and the zero-wait-state slave design means bus bandwidth is limited only by the clock frequency. If a DMA controller is added, its data buffers should be placed in the Scratchpad SRAM (which is not a Wishbone slave), not on the Wishbone bus, so the Wishbone topology does not need to change.

AXI4 Topology

The AXI4 bus uses a simple **Memory Arbiter** (two masters, one slave): I-Cache and D-Cache are both AXI4 masters; the QSPI controller data port is the single AXI4 slave. The arbiter implements fixed priority (D-Cache $>$ I-Cache) and never interrupts an in-progress burst. Details are documented in the Memory Concept section.

Peripheral Register Offset Alignment

All peripheral registers are 64 bit wide and aligned to 8-byte boundaries. The address offset within a peripheral starts at `0x00` for the ID register and increases in steps of `0x08`. This is consistent with the Wishbone bus data width and the 64-bit alignment of the RISC-V load and store instructions used to access the peripheral registers.

Read-Modify-Write

The Wishbone protocol as implemented does not support atomic read-modify-write operations. Software must therefore use disable-interrupt / critical-section patterns when accessing shared registers from multiple contexts.

4.3.9 Comparison: Wishbone vs. AXI4

Table 4.21 summarises the key differences between the two on-chip buses and the rationale for using each.

Table 4.21: Wishbone B4 Classic vs. AXI4 – Key Differences

Property	Wishbone B4 Classic	AXI4 (cache subset)
Transfer unit	1 word (64 bit)	4 beats × 64 bit = 32 B
Channels	Shared address/data	Decoupled AR and R
Handshake	STB + ACK	VALID + READY per channel
Outstanding tx	0 (blocking)	1 per master
Burst	Not supported	INCR, 4 beats fixed
Write support	Full	None (write channels removed)
Error signalling	None	RRESP; maps to RISC-V exceptions
Latency model	0 wait states typical	Variable; QSPI controller stalls until Flash data arrives
Complexity	Minimal	Moderate (5 channels, ID tracking, burst counter)
Used for	Peripheral registers	Cache-line fills from NOR Flash

4.4 Interrupt Concept

Table 4.22: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.5 Clock Concept

Table 4.23: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.5.1 Clock Generation Unit (CGU) Overview

Both chip variants (Table 4.29) share the same Clock Generation Unit IP (`as_cgu_top` / `as_cgucore`). The chip has a single external clock input, `clk_i`, from which the CGU derives all internal clock domains using free-running counter-based dividers. There is no PLL: every derived clock is an integer division of `clk_i`.

Table 4.24: CGU Clock Domains

Output	Divider (fixed)	Consumers	Notes
clk_core_o	clk_core_div = 80	CPU core, GPIO, UART0, D-Mem/Scratchpad, D-/I-Cache (Variant B)	Functional clock for the CPU and almost all peripherals
clk_bus1_o	clk_bus1_div = 80	Reserved	Not currently consumed in as_top_mem
clk_bus2_o	clk_bus2_div = 100	Reserved	Not currently consumed in as_top_mem
clk_qspi_o	clk_qspi_div = 4	QSPI controller (Variant B only)	Not present in Variant A (no QSPI controller)
tck_i	— (external JTAG clock)	JTAG TAP, I-Mem scan chain, boundary scan	Fully asynchronous to clk_i; no timing relationship is analysed between the two domains

The divider ratios are fixed at RTL compile time, not run-time configurable. Unlike the QSPI and UART peripherals (which expose a genuine CLKDIV Wishbone register), the CGU implements only a read/write peripheral **ID register**; the four divider ratios above are SystemVerilog localparams in as_pack.sv and take effect unconditionally. Changing a divider therefore requires an RTL/-constant change and a full re-synthesis – it cannot be adjusted by software or by writing a register.

Scan-Test Clock Bypass

During normal operation the CPU and Wishbone peripherals are clocked by clk_div_s, which is generated in as_top_mem as:

```
assign clk_div_s = dr_cap_s ? clk_i : clk_core_s;
```

dr_cap_s is driven by the JTAG TAP. In normal function mode (dr_cap_s = 0) the CPU runs on the divided clk_core_s. In scan-test mode (dr_cap_s = 1) the divider is bypassed and the CPU is clocked directly by the raw external clk_i, so that an ATE tester can single-step or free-run the scan chain independently of the CGU divide ratio. The CGU itself (as_cgu_top) always runs on the undivided clk_i; only its divided outputs are subject to the bypass mux.

4.5.2 Clock Requirements – Variant A (RV_NoPipeline)

Status

No X-FAB XH035 gate-level static timing analysis (STA) has been run yet for RV_NoPipeline. The Genus synthesis script (scripts/01b_rv64_syn.tcl) still carries a placeholder constraint (create_clock -period 20.000, i.e. 50 MHz on clk_i) and is explicitly marked in its own comments as provisional, pending SRAM-macro integration for I-Mem/D-Mem. The recommendation below is therefore a conservative pre-STA target, not a timing-signed-off number.

Recommended Frequencies

Table 4.25: RV_NoPipeline – Recommended Clock Frequencies

Signal	Recommended value	Rationale
clk_i (external)	100 MHz (10 ns period)	Matches the operating point already exercised by every existing xsim and cocotb testbench (clk_2_t = 5 ns half-period)
clk_core_s (CPU core, clk_i/80)	1.25 MHz (800 ns period)	Matches clk_80_t = 400 ns in the testbenches and the 1.25 MHz core clock already assumed in the UART timing calculation (Section 7.1.1)
tck_i (JTAG)	$\leq$ clk_i/4 (typ. 25 MHz or less)	Conservative default; JTAG is asynchronous to clk_i (Table 4.24) and not on the CPU's timing-critical path

Rationale

- **Why 1.25 MHz and not higher:** `RV_NoPipeline` is a multi-cycle, non-pipelined core (Section 4.2): each instruction's critical path runs through instruction decode, register file, the ALU, and (for loads/stores) the Wishbone master/slave BPI handshake, all within states of a single clock. X-FAB XH035 is a 350 nm process; without a completed STA on the actual gate-level netlist (SRAM macros not yet integrated, see Status above), committing to a high frequency is premature. 1.25 MHz leaves a very large safety margin against any plausible critical-path length on a 350 nm standard-cell library.

- **Why keep the already-validated operating point:** every xsim and cocotb regression test (58 xsim / 57 cocotb tests, see Section 7.1) already runs at `clk_i` = 100 MHz / `clk_core_s` = 1.25 MHz, and the UART bit-timing calculation in Section 7.1.1 is derived from exactly this core clock. Choosing the same frequency for the first tapeout avoids re-deriving or re-validating any functional timing (UART baud rate, GPIO polling loops, etc.).

- **Fixed divider consequence:** because `clk_core_div` = 80 is fixed at compile time (Section 4.5.1), the external `clk_i` frequency is always exactly $80\times$ the desired core frequency. There is no register to trade off external frequency against core frequency at run time.

Open Items

Action (CLK-01): Once I-Mem/D-Mem SRAM macros are integrated into the `RV_NoPipeline` netlist, run Genus `report_timing` with a proper `create_generated_cl` `-divide_by 80` constraint on `clk_core_o` (in addition to the existing constraint on `clk_i`, which today only reflects the scan-test bypass path) and confirm that 1.25 MHz core / 100 MHz external clears timing with adequate margin at the worst-case corner. If margin is large, a higher core frequency may be considered for a later revision – this requires changing the `clk_core_div` localparam in `as_pack.sv` and re-synthesising, not a register write.

4.6 System Control Concept

Table 4.26: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.7 Power Concept

Table 4.27: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.8 Memory Concept

Table 4.28: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.8.1 Chip Variants

The **RWU-RV64I** project comprises two distinct chip variants that implement the same RV64I ISA but differ in both CPU core and memory subsystem:

Table 4.29: Chip Variant Comparison

	Variant A – No-Cache	**Variant B – Pipeline-Cache**
RTL top	`RV_NoPipeline`	`RV_PipelineCacheFwd`
CPU core	`as_cpux`: single-cycle, no pipeline	`cc_cpupipefwd`: 5-stage pipeline (IF/ID/EX/MEM/WB) with EX/MEM→EX and MEM/WB→EX forwarding
Instruction memory	On-chip I-Mem (32 KB), loaded via JTAG scan chain; no external Flash	No on-chip I-Mem; instructions fetched from off-chip NOR Flash via QSPI, cached in I-Cache (4 KB)
Data memory	On-chip D-Mem (8 KB), connected via Wishbone	No on-chip D-Mem; read-only data cached in D-Cache (4 KB); read-write data in on-chip Scratchpad SRAM (8 KB)
External Flash	None	NOR Flash via QSPI (code + read-only data)
Cache subsystem	None	I-Cache + D-Cache + Memory Arbiter + Scratchpad
Peripheral bus	Wishbone (I-Mem, D-Mem, GPIO, CGU, UART0)	Wishbone (GPIO, CGU, UART0, QSPI ctrl); cache uses AXI4
Design goal	Minimal area; standalone execution from on-chip memory	Higher performance via pipelining and cache; runs from off-chip Flash, writable Scratchpad

The remainder of this section (Sections 4.8.3 through the end) describes Variant B (RTL: `RV_PipelineCacheFwd`). Variant A does not use any cache and is described in Section 4.8.2.

4.8.2 Variant A – No-Cache Memory Subsystem

Variant A implements a simple Harvard memory subsystem with two on-chip SRAM blocks and no cache.

- **Instruction Memory (I-Mem, 32 KB):** A single-port synchronous SRAM containing 8 192 RISC-V 32-bit instructions (word-addressed). The CPU accesses I-Mem via a legacy flat bus. At reset, I-Mem is empty; the content is loaded via the JTAG scan chain (a dedicated instruction-memory shift register in the TAP). I-Mem is read-only during normal CPU execution.
- **Data Memory (D-Mem, 8 KB):** A single-port synchronous SRAM with a Wishbone B3 slave interface. Supports byte, halfword, word, and doubleword accesses controlled by the Wishbone `SEL` byte-enable signal. Holds `.data`, `.bss`, stack, and heap. Address range: `0x0000_0000`–`0x0000_1FFF` (8 KB, mapped at base address 0).
- **Peripheral Bus:** All CPU data accesses are routed through a Wishbone master BPI. Address decoding selects D-Mem, GPIO, CGU, or QSPI controller by chip select (`csx_s`).

There is no cache miss penalty in Variant A: every fetch or load/store completes in a fixed number of Wishbone cycles. The trade-off is that all code and data must fit in the two 32 KB + 8 KB on-chip memories.

4.8.3 Variant B – Pipeline-Cache Memory Subsystem: Overview

Variant B (RTL top `RV_PipelineCacheFwd`) pairs the 5-stage pipelined CPU core (`cc_cpupipefwd`, see Section 4.2) with a Harvard-style split-cache architecture. A dedicated **Instruction Cache (I-Cache)** serves the CPU fetch path, and a dedicated **Data Cache (D-Cache)** serves the load/store path. Both caches are 4-way set-associative with a fixed cache-line size of 32 bytes.

The backing store for both caches is an off-chip **NOR Flash** memory accessed through the on-chip QSPI controller. NOR Flash is **read-only** from the cache's perspective: the QSPI controller provides a cache-miss-driven read path only; there is no byte- or word-level write path to Flash. Accordingly the D-Cache is **read-allocate, no-write**: only load misses trigger cache-line fills from Flash; stores to the Flash address region cause a bus error (`dc_err_o`). Because no dirty line can ever exist, no dirty bit and no write-back FSM state are required. Cache data is held in two on-chip **SRAM macros** (X-Fab process), one per cache. A **Memory Arbiter** serialises simultaneous miss requests from the two cache controllers onto the single AXI4 bus that leads to the QSPI controller.

Read-write data (stack, heap, `.data`, `.bss`) resides in a dedicated on-chip **Scratchpad SRAM**. The D-Cache controller decodes the physical address of every load/store: if the address falls in the Flash region the request is served through the cache; if it falls in the Scratchpad region the cache is bypassed and the Scratchpad SRAM is accessed directly via a simple synchronous interface.

The on-chip bus between cache controllers and the Memory Arbiter uses a subset of the **AMBA AXI4** protocol with burst support, replacing the Wishbone B4 Classic interface used elsewhere in the design. This allows a full 32-byte cache-line fill to complete in a single four-beat burst transaction instead of eight independent single transfers.

Table 4.30: Memory Hierarchy Summary

Level	Block	Capacity	Technology	Latency (typical)
L1-I	I-Cache	4 KB (param.)	On-chip SRAM macro	1 cycle (hit)
L1-D	D-Cache	4 KB (param.)	On-chip SRAM macro	1 cycle (hit)
–	Scratchpad	TBD (param.)	On-chip SRAM macro	1–2 cycles (direct)
Main	NOR Flash	device-specific	Off-chip via QSPI	$\gg$10 cycles (miss)

4.8.4 System Memory Architecture

Figure 4.6 shows the system-level block diagram of the memory subsystem.

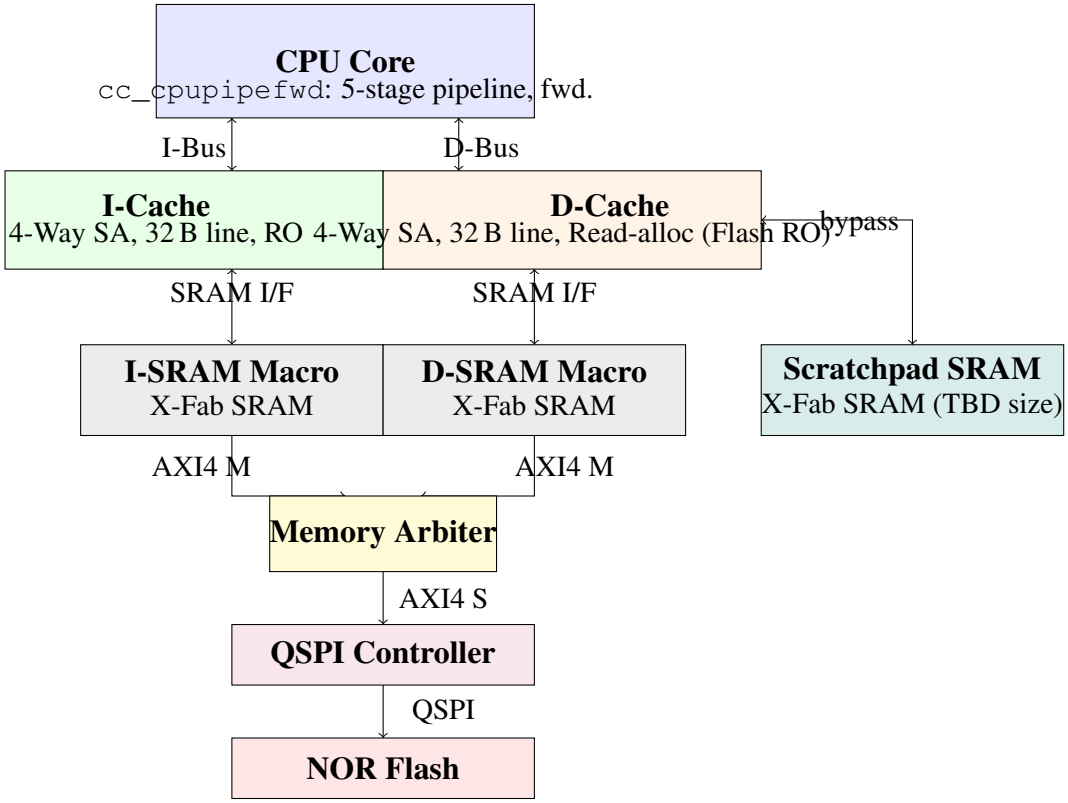


Figure 4.6: System Memory Architecture Block Diagram

4.8.5 Cache Configuration Parameters

Both caches share the same structural parameters, listed in Table 4.31. Parameters marked fixed are part of the cache-line protocol and cannot be changed without redesigning the AXI4 burst transactions and the SRAM macros.

Table 4.31: Cache Configuration Parameters

Parameter	Symbol	Default	Fixed?	Description
Cache size	`CACHE_SIZE_B`	4096	no	Total capacity in bytes per cache
Associativity	`WAYS`	4	yes	Number of ways per set
Cache-line size	`LINE_BYTES`	32	yes	Bytes per cache line
Number of sets	`SETS`	32	no	Derived: `CACHE_SIZE_B / (WAYS * LINE_BYTES)`
Physical addr width	`PA_WIDTH`	32	no	Physical address bits
AXI4 data width	`AXI_DW`	64	yes	AXI4 memory bus data width

4.8.6 Physical Address Decomposition

With the default parameters (4 KB cache, 4-way, 32 B line, 32-bit physical address), a physical address is divided as shown in Figure 4.7 and Table 4.32.

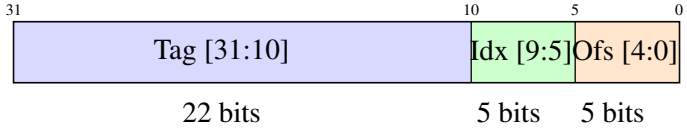


Figure 4.7: Physical Address Decomposition (32-bit PA, 4 KB cache, 32 B line)

Table 4.32: Physical Address Fields

Field	Bits	Width	Description
`tag`	[31:10]	22	Compared against stored tag to determine hit/miss
`index`	[9:5]	5	Selects one of 32 sets
`offset`	[4:0]	5	Byte offset within the 32-byte cache line
`word`	[4:3]	2	Selects one of four 64-bit words within the line
`instr`	[4:2]	3	Selects one of eight 32-bit instructions (I-Cache only)
`byte`	[2:0]	3	Byte offset within a 64-bit word (D-Cache sub-word)

The general formula for derived address parameters is:

$$\text{OFFSET_BITS} = \log_2(\texttt{LINE_BYTES}) = 5$$

$$\text{INDEX_BITS} = \log_2(\texttt{SETS}) = \log_2\left(\frac{\texttt{CACHE_SIZE_B}}{\texttt{WAYS} \times \texttt{LINE_BYTES}}\right) = 5$$

$$\text{TAG_BITS} = \texttt{PA_WIDTH} - \text{INDEX_BITS} - \text{OFFSET_BITS} = 32 - 5 - 5 = 22$$

4.8.7 Cache Tag and Metadata Arrays

Each set contains per-way metadata (valid, tag, and for D-Cache: dirty bit) plus shared set-level pseudo-LRU bits.

I-Cache Metadata Layout

The I-Cache is read-only; no dirty bit is required.

Table 4.33: I-Cache – Metadata Bits per Set

Field	Bits	Width	Description
Per way (repeated 4×):			
`valid[w]`	1 per way	1	Line present in cache; cleared on reset or flush
`tag[w]`	22 per way	22	Upper address bits stored at fill time
Per set (shared):			
`plru[2:0]`	3 per set	3	Pseudo-LRU replacement state (see Section 4.8.7)
Total per set:			
(sum)	–	95	$4 \times (1 + 22) + 3 = 95$ bits; rounded to 96 for SRAM alignment

D-Cache Metadata Layout

The D-Cache is read-allocate with no write-back; no dirty bit is required (identical structure to the I-Cache metadata).

Table 4.34: D-Cache – Metadata Bits per Set

Field	Bits	Width	Description
Per way (repeated 4×):			
`valid[w]`	1 per way	1	Line present in cache; cleared on reset or flush
`tag[w]`	22 per way	22	Tag stored at fill time
Per set (shared):			
`plru[2:0]`	3 per set	3	Pseudo-LRU replacement state (see Section 4.8.7)
Total per set:			
(sum)	–	95	$4 \times (1 + 22) + 3 = 95$ bits; rounded to 96 for SRAM alignment

Pseudo-LRU Replacement Policy

A 3-bit pseudo-LRU (PLRU) binary tree approximates LRU replacement for 4 ways without the overhead of true LRU. The tree is shown in Figure 4.8 and the bit semantics are defined in Table 4.35.

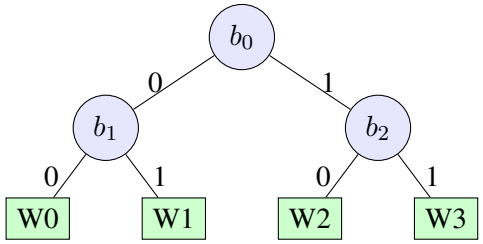


Figure 4.8: Pseudo-LRU Binary Tree for 4 Ways

Table 4.35: PLRU Bit Semantics

Bit	Meaning (bit=0)	Meaning (bit=1)
b_0	Left pair (W0, W1) is LRU; victim from left half	Right pair (W2, W3) is LRU; victim from right half
b_1	W1 is LRU within left pair; evict W1	W0 is LRU within left pair; evict W0
b_2	W3 is LRU within right pair; evict W3	W2 is LRU within right pair; evict W2

Victim selection and PLRU update rules are defined in Table 4.36.

Table 4.36: PLRU Victim Selection and Update on Cache Access

Accessed Way	Victim (for miss/eviction)	PLRU bits updated
W0	b_0=0: (b_1=0 $\Rightarrow$ W1), (b_1=1 $\Rightarrow$ W0)	$b_0 \leftarrow 1,\ b_1 \leftarrow 1$
W1	same left half	$b_0 \leftarrow 1,\ b_1 \leftarrow 0$
W2	b_0=1: (b_2=0 $\Rightarrow$ W3), (b_2=1 $\Rightarrow$ W2)	$b_0 \leftarrow 0,\ b_2 \leftarrow 1$
W3	same right half	$b_0 \leftarrow 0,\ b_2 \leftarrow 0$

The PLRU state is stored in a small register array (32 sets $\times$ 3 bits = 96 bits). It is reset to all-zero on power-on reset.

4.8.8 SRAM Macro Organisation

The memory subsystem uses five X-Fab XH035 SPRAM macros in total (see Section 4.8.9 for the selected instances). All macros operate as single-port synchronous SRAMs at 3.3 V.

Table 4.37: SRAM Macro Logical Requirements

Macro	Width	Depth	Size	Notes
Tag SRAM (I)	96 bit	32	384 B	valid + tag for 4 ways + 3 PLRU bits; rounded from 95
Tag SRAM (D)	96 bit	32	384 B	same structure as I-Cache (no dirty bit); rounded from 95
Data SRAM (I)	256 bit	128	4 KB	32 sets × 4 ways × 32 B; addressed as `{index,way}`
Data SRAM (D)	256 bit	128	4 KB	same organisation as I-Cache data SRAM
Scratchpad	64 bit	`SP_DEPTH`	`SP_SIZE_B`	Default: `SP_DEPTH=1024` ⇒ 8 KB; addressed by `dc_addr_i[12:3]`

The data SRAM is addressed by the concatenation `{index[4:0], way[1:0]}` giving a 7-bit address (128 entries). All 256 bits of one cache line are read or written in a single SRAM cycle.

During a cache-line fill, the cache controller writes the four 64-bit AXI4 beat words into the data SRAM on four consecutive cycles using a word-level write enable, while the full line address (`index` ∥ `fill_way`) is held constant.

The Scratchpad SRAM is accessed with a 64-bit data bus and an 8-bit byte enable (`dc_wstrb_i`), matching the D-Cache data bus width. Its address is derived directly from `dc_addr_i[12:3]` (for the default 8 KB instance).

4.8.9 SRAM Macro Selection – XH035 SPRAM

Library and Process

All SRAM macros are generated with the X-Fab **XH035** (350 nm Modular Sensor Technology Platform) **SPRAM** (Single-Port RAM) compiler at the nominal supply of 3.3 V (2.2 V low-voltage option not used). Key library properties relevant to this design:

- Zero static DC power consumption (only intrinsic leakage).

- Separate input and output data buses (no bus contention during read-modify-write).
- Byte-level write enable supported (required for sub-word stores to the Scratchpad SRAM).
- Wide range of instance sizes; the compiler generates macros for the exact width×depth parameters listed in Table 4.38.
- Optional BIST block: **not instantiated** in the first tapeout; test coverage via scan chain is sufficient.

Selected Instances

Table 4.38 lists the five SPRAM instances to be generated from the XH035 compiler. Macro names follow the compiler naming convention and are to be confirmed against the current catalogue release.

Table 4.38: XH035 SPRAM Instances

Instance	Width	Depth	Size	Macro name	Usage
I-Tag	96 bit	32	384 B	`SPRAM_96x32`	I-Cache tag + valid + PLRU
D-Tag	96 bit	32	384 B	`SPRAM_96x32`	D-Cache tag + valid + PLRU
I-Data	256 bit	128	4 KB	`SPRAM_256x128`	I-Cache data array
D-Data	256 bit	128	4 KB	`SPRAM_256x128`	D-Cache data array
SP	64 bit	1024	8 KB	`SPRAM_64x1024`	Scratchpad (stack-/heap/.data/.bss)

Notes on macro names: The names `SPRAM_{W}x{D}` are placeholders using the pattern ⟨width⟩x⟨depth⟩. The actual names generated by the XH035 SPRAM compiler must be verified against the compiler catalogue and substituted in the netlist and floorplan before tapeout. I-Tag and D-Tag use the same compiled macro; two instances are placed. I-Data and D-Data likewise share one compiled macro with two instances.

SRAM Port Mapping

Table 4.39 maps the logical cache-controller signals to the standard XH035 SPRAM port names.

Table 4.39: XH035 SPRAM Port Mapping

SPRAM port	Direction	Connected to
`CLK`	in	System clock `clk_i`
`CEN`	in	Chip enable, active-low; asserted when cache controller drives a valid address
`WEN`	in	Write enable, active-low; low for write, high for read
`BWEN`	in	Bit-write enable, active-low; all-zero = write all bits (tag/data SRAMs); `~dc_wstrb_i` expanded to bit-level (Scratchpad only)
`A`	in	Address bus; see Table 4.37 for width
`D`	in	Write data
`Q`	out	Read data; valid one cycle after `CEN` low on a read

Timing Parameters

Table 4.40 lists the timing parameters to be extracted from the XH035 SPRAM compiler output for the worst-case process corner (SS – slow NMOS, slow PMOS), maximum temperature (125 °C), and minimum supply voltage (3.0 V for 3.3 V nominal).

Table 4.40: XH035 SPRAM Timing Parameters (worst-case SS/125 °C/3.0 V)

Parameter	Symbol	Value	Description
Clock period (min.)	t_{cyc}	TBD ns	Minimum clock period for correct operation
Access time	t_{aa}	TBD ns	Address to `Q` valid; determines read latency
Setup time (data)	t_{ds}	TBD ns	`D` / `BWEN` setup before rising `CLK`
Hold time (data)	t_{dh}	TBD ns	`D` / `BWEN` hold after rising `CLK`
Setup time (addr.)	t_{as}	TBD ns	`A` / `CEN` / `WEN` setup before rising `CLK`
Hold time (addr.)	t_{ah}	TBD ns	`A` / `CEN` / `WEN` hold after rising `CLK`

The system clock frequency must satisfy:

$$f_{\text{clk}} \leq \frac{1}{t_{\text{cyc, worst}}}$$

All cache hit paths (tag compare + data SRAM read) must close timing within a single clock cycle at the selected frequency. Static Timing Analysis (STA) with the compiler-supplied Liberty (`.lib`) file is required before tapeout.

Action (MEM-02): Run the XH035 SPRAM compiler for all five instances listed in Table 4.38 and fill in the timing values in Table 4.40. Verify that the maximum target frequency is achievable at the SS/125 °C/3.0 V corner.

4.8.10 Instruction Cache (I-Cache)

Functional Description

The I-Cache is a read-only, 4-way set-associative cache that serves the CPU instruction fetch stage. It has no write path to the backing NOR Flash, and no dirty bit.

On every fetch request the cache controller:

1. Reads the tag SRAM at address `index`.
2. Compares the four stored tags with the request tag in parallel.
3. Checks the corresponding valid bits.
4. On a hit: reads the data SRAM at `{index, hit_way}`, selects the 32-bit instruction word using `offset[4:2]`, and returns it to the CPU in the same cycle.
5. On a miss: stalls the CPU, issues an AXI4 read burst to the Memory Arbiter, fills the selected SRAM line, updates the tag SRAM and PLRU state, then un-stalls the CPU.

Cache Hit Path

The hit path is combinatorial: tag comparison, data SRAM read, and instruction word selection complete within one clock cycle. The CPU pipeline sees a zero-cycle miss overhead when the cache hits.

Cache Miss Handling

On a miss the I-Cache controller:

- Uses the PLRU state to select the victim way.
- Asserts the `stall` signal to the CPU.
- Issues an AXI4 INCR burst of 4 beats (4 $\times$ 64 bit = 32 B) aligned to the cache-line boundary (`addr[63:5] & 5'b0`).
- Writes the four beats sequentially into the data SRAM.

- Updates the tag and valid bit for the filled way.
- Updates the PLRU bits.
- De-asserts `stall` on the cycle the last beat arrives.

I-Cache State Machine

Table 4.41: I-Cache Controller States

State	Description
`IDLE`	Waiting for a fetch request or flush signal from the CPU
`LOOKUP`	Tag comparison active; result available next cycle
`MISS`	AXI4 AR channel: sending read address burst to arbiter
`FILL`	Receiving 4 AXI4 R-channel beats; writing data SRAM
`RESPOND`	First word of filled line presented to CPU; stall de-asserted
`ERR`	AXI4 RRESP $\neq$ OKAY detected during FILL: valid bit not set; `ic_err_o` pulsed high for one cycle; return to IDLE
`FLUSH`	Invalidation: iterates set counter 0...31; writes `valid=0`, `plru=0` to tag SRAM each cycle; asserts `ic_flush_done_o` after 32 cycles

4.8.11 Data Cache (D-Cache)

Functional Description

The D-Cache is a **read-allocate** 4-way set-associative cache that serves the CPU load/store stage. Sub-word accesses (byte, halfword, word, doubleword) are supported.

The D-Cache controller decodes the physical address of every CPU load/store and dispatches to one of two paths:

- **Flash region (cacheable, read-only):** the request goes through the tag-lookup / fill pipeline described below. Stores to the Flash region are not supported; the cache controller asserts `dc_err_o` and raises a Load Access Fault (see Section 4.8.18).
- **Scratchpad region (uncacheable, read-write):** the cache is bypassed entirely. The controller drives the Scratchpad SRAM interface directly with the physical address, write data, byte enables, and read/write strobe, and returns the read data to the CPU after one SRAM cycle.

On a load from the Flash region the cache controller:

1. Reads the tag SRAM at address `index`.

2. Compares the four stored tags with the request tag in parallel.

3. Checks the corresponding valid bits.

4. On a read hit: reads the data SRAM, extracts the addressed word or sub-word, and returns it to the CPU in the same cycle.

5. On a miss: stalls the CPU, issues an AXI4 read burst to the Memory Arbiter, fills the selected SRAM line, updates tag and valid bit, updates the PLRU state, then un-stalls the CPU.

Address Region Policy and Absence of Write-Back

Because the Flash backing store is read-only and the Scratchpad is uncacheable, **the dirty bit is never set** and no cache line is ever marked for write-back. As a direct consequence:

- The D-Cache metadata (Table 4.34) does **not** contain a dirty bit; per-way metadata is $(1 + 22) = 23$ bits (valid + tag only, no dirty field).

- The `WRITEBACK` FSM state and the full AXI4 write path (AW, W, B channels) are **eliminated** from the D-Cache controller. The D-Cache requires only the AXI4 read channels (AR, R).

- Write-allocate is **not applicable**: stores go to the Scratchpad directly; there is no "write miss" into the cache.

This simplification resolves open item MEM-01 and reduces D-Cache controller complexity significantly.

Scratchpad SRAM – Software Segment Mapping

The Scratchpad SRAM serves as the sole read-write memory of the system. It holds all program data that must be writable at run time. Table 4.42 lists the standard linker segments assigned to the Scratchpad region.

Table 4.42: Software Segments in Scratchpad SRAM

Segment	Access	Content and lifecycle
`.data`	R/W	Initialised global and static variables. Values are copied from NOR Flash to Scratchpad by the start-up routine (`_start` / `crt0`) before `main()` is entered. Subsequent reads and writes go directly to Scratchpad.
`.bss`	R/W	Uninitialised (zero-initialised) global and static variables. The start-up routine zeroes this region in Scratchpad before `main()` is entered; no NOR Flash copy is required.
Heap	R/W	Dynamic memory managed by `malloc`/`free` (or equivalent). Grows upward from the end of `.bss` toward the stack. Access pattern is unpredictable (random reads and writes), making caching impractical.
Stack	R/W	Automatic variables, function call frames, and saved registers. Grows downward from the top of the Scratchpad region. Access is highly regular (push/pop, mostly 8-byte aligned) but write-intensive; caching would require frequent dirty evictions.

Why these segments cannot reside in NOR Flash. NOR Flash is a read-only medium from the system's perspective: the QSPI controller provides a cache-miss-driven read path only; there is no byte- or word-level write path to Flash. Even if write commands were issued via the QSPI configuration interface, Flash write latency is in the millisecond range and requires sector erase before re-programming – wholly unsuitable for stack or heap access.

Linker script responsibility. The linker script must place `.data`, `.bss`, heap, and stack inside the Scratchpad address window (`SP_BASE...SP_BASE + SP_SIZE - 1`). Code (`.text`) and read-only data (`.rodata`) remain in the NOR Flash address window and are served through the cache. The start-up routine copies `.data` from Flash to Scratchpad using the D-Cache bypass path (Scratchpad addresses); the Flash source copy is accessed through the D-Cache (Flash addresses).

D-Cache State Machine

Table 4.43: D-Cache Controller States

State	Description
IDLE	Waiting for a load/store request or flush signal from the CPU
DECODE	Physical address decoded; dispatch to CACHE path (Flash region) or BYPASS path (Scratchpad region)
LOOKUP	Flash region: tag comparison active; result available next cycle
FILL	Flash region: issuing AXI4 AR burst; receiving and writing 4 beats to data SRAM
RESPOND	Flash region: read data presented to CPU; stall de-asserted
BYPASS	Scratchpad region: driving Scratchpad SRAM interface directly; read data or write acknowledgement returned to CPU after 1 cycle
ERR	AXI4 RRESP $\neq$ OKAY detected during FILL: valid bit not set; dc_err_o pulsed high for one cycle; return to IDLE
FLUSH	Invalidation: iterates set counter 0...31; writes valid=0, plru=0 to tag SRAM each cycle; asserts dc_flush_done_o after 32 cycles

Note: The WRITEBACK and WRITE_HIT states present in earlier revisions of this document are removed. Neither state can be reached: no dirty lines exist (Flash is read-only) and stores bypass the cache via the BYPASS state.

4.8.12 CPU-to-Cache Interface

The interface between the CPU core and each cache uses a simple valid/ready + stall handshake. It is not based on Wishbone or AXI4: those protocols are reserved for the off-chip memory path.

Instruction Bus (CPU – I-Cache)

The Instruction Bus carries instruction fetch requests from the IF stage to the I-Cache.

Table 4.44: Instruction Bus Interface (`as_icache_if`)

Signal	Dir (CPU)	Width	Description
`ic_addr_i`	out	64	Instruction fetch address; must be 4-byte aligned
`ic_req_i`	out	1	Fetch request; held high while the CPU needs an instruction
`ic_flush_i`	out	1	Pulse high for one cycle to trigger full cache invalidation (FENCE.I)
`ic_stall_o`	in	1	High while a cache miss, flush, or error recovery is in progress
`ic_flush_done_o`	in	1	Pulsed high for one cycle when flush is complete
`ic_err_o`	in	1	Pulsed high for one cycle on AXI4 read error; CPU raises Instruction Access Fault (`mcause`=1, `mtval=ic_addr_i`)
`ic_rdata_o`	in	32	Instruction word returned by the I-Cache
`ic_rvalid_o`	in	1	`ic_rdata_o` is valid; asserted one cycle after a hit

```
interface as_icache_if (input logic clk_i,
                        input logic rst_i);
  logic [63:0] ic_addr;
  logic        ic_req;
  logic        ic_flush;
  logic        ic_stall;
  logic        ic_flush_done;
  logic        ic_err;
  logic [31:0] ic_rdata;
  logic        ic_rvalid;
  modport cpu  (output ic_addr, ic_req, ic_flush,
                input  ic_stall, ic_flush_done, ic_err,
                       ic_rdata, ic_rvalid);
  modport cache(input  ic_addr, ic_req, ic_flush,
                output ic_stall, ic_flush_done, ic_err,
                       ic_rdata, ic_rvalid);
endinterface
```

Data Bus (CPU – D-Cache)

The Data Bus carries load and store requests from the MEM stage to the D-Cache.

Table 4.45: Data Bus Interface (`as_dcache_if`)

Signal	Dir (CPU)	Width	Description
`dc_addr_i`	out	64	Data address
`dc_req_i`	out	1	Request valid (load or store)
`dc_wr_i`	out	1	1 = store, 0 = load
`dc_size_i`	out	3	Transfer size: 000=byte, 001=half, 010=word, 011=double
`dc_wdata_i`	out	64	Write data from CPU (store)
`dc_wstrb_i`	out	8	Byte enables for store; derived from `dc_size_i` and `dc_addr_i[2:0]`
`dc_flush_i`	out	1	Pulse high for one cycle to trigger full D-Cache invalidation (FENCE.I)
`dc_stall_o`	in	1	High while cache miss, scratchpad access, flush, or error recovery is in progress
`dc_flush_done_o`	in	1	Pulsed high for one cycle when flush is complete
`dc_err_o`	in	1	Pulsed high for one cycle on AXI4 read error; CPU raises Load Access Fault (`mcause`=5, `mtval`=`dc_addr_i`)
`dc_rdata_o`	in	64	Read data returned to CPU (load)
`dc_rvalid_o`	in	1	`dc_rdata_o` is valid

```
interface as_dcache_if (input logic clk_i,
                        input logic rst_i);
  logic [63:0] dc_addr;
  logic        dc_req;
  logic        dc_wr;
  logic [2:0]  dc_size;
  logic [63:0] dc_wdata;
  logic [7:0]  dc_wstrb;
```

```
  logic        dc_flush;
  logic        dc_stall;
  logic        dc_flush_done;
  logic        dc_err;
  logic [63:0] dc_rdata;
  logic        dc_rvalid;
  modport cpu  (output dc_addr, dc_req, dc_wr, dc_size,
                       dc_wdata, dc_wstrb, dc_flush,
                input  dc_stall, dc_flush_done, dc_err,
                       dc_rdata, dc_rvalid);
  modport cache(input  dc_addr, dc_req, dc_wr, dc_size,
                       dc_wdata, dc_wstrb, dc_flush,
                output dc_stall, dc_flush_done, dc_err,
                       dc_rdata, dc_rvalid);
endinterface
```

4.8.13 AXI4 Memory Bus (Cache Controllers to Memory Arbiter)

AXI4 Subset Used

Both cache controllers use the AMBA AXI4 protocol as masters on the memory bus. The Memory Arbiter presents a single AXI4 slave port toward the cache controllers and a single AXI4 master port toward the QSPI controller. Table 4.46 lists the AXI4 bus parameters.

Table 4.46: AXI4 Memory Bus Parameters

Parameter	Value	Comment
Protocol	AXI4 (not AXI4-Lite)	Burst support required for cache-line fills
Data width	64 bit	`AXI_DW=64`; 4 beats fill a 32 B cache line
Address width	32 bit	Matches physical address space
ID width	4 bit	Allows up to 16 outstanding transactions; I-Cache uses ID=0x1, D-Cache uses ID=0x2
Burst type	INCR (01)	Sequential address increment; no WRAP or FIXED
Burst length	ARLEN=3 / AWLEN=3	4 beats $\times$ 8 B = 32 B per cache line
Beat size	ARSIZE=3 (8 B)	One 64-bit word per beat
Outstanding	1 per master	Each cache has at most 1 transaction in flight
Protection	ARPROT=3'b000	Normal non-secure unprivileged
Cache hints	ARCACHE=4'b0010	Normal non-cacheable (device is the QSPI controller)
Write response	BRESP checked	Error on BRESP $\neq$ OKAY is flagged; TBD behaviour

AXI4 Channel Signals

Table 4.47 through Table 4.51 list the signals of each AXI4 channel. Signals marked † are tied to a fixed value; they do not require routing logic.

Table 4.47: AXI4 AR Channel (Read Address) – I/D Cache Master

Signal	Dir	Width	Description
`arid`	M→S	4	Transaction ID; I-Cache: 4'h1, D-Cache: 4'h2
`araddr`	M→S	32	Start address of the burst; must be aligned to 32 B (`addr[4:0]=0`)
`arlen`	M→S	8	Burst length minus one; always 8'h03 (4 beats)
`arsize`	M→S	3	Bytes per beat; always 3'b011 (8 bytes)
`arburst`	M→S	2	Burst type; always 2'b01 (INCR)
`arvalid`	M→S	1	Address and control information valid
`arready`	S→M	1	Slave ready to accept address
`arprot`†	M→S	3	Always 3'b000
`arcache`†	M→S	4	Always 4'b0010

Table 4.48: AXI4 R Channel (Read Data) – I/D Cache Master

Signal	Dir	Width	Description
`rid`	S→M	4	Echo of `arid`; cache controller checks this matches its pending ID
`rdata`	S→M	64	Read data beat; received in natural address order (beat 0 = lowest address)
`rresp`	S→M	2	2'b00 = OKAY; other values indicate an error
`rlast`	S→M	1	Asserted on the last beat of the burst (beat 3)
`rvalid`	S→M	1	Read data and strobes valid
`rready`	M→S	1	Cache controller ready to receive; held high once a miss is in progress

Note – AW, W, B channels not implemented: The D-Cache is read-allocate with no write-back (Flash is read-only; no dirty bit exists). The AW, W, and B channel tables below (Tables 4.49–4.51) and the write-burst timing figure (Fig. 4.10) are retained for reference only and must **not** be implemented in `asDCacheCtrl`. The D-Cache AXI4 master uses AR and R channels exclusively.

Table 4.49: AXI4 AW Channel (Write Address) – **NOT USED**; retained for reference only

Signal	Dir	Width	Description
`awid`	M→S	4	Always 4'h2 (D-Cache)
`awaddr`	M→S	32	Start address of cache-line burst; 32-byte aligned
`awlen`	M→S	8	Always 8'h03 (4 beats)
`awsize`	M→S	3	Always 3'b011 (8 bytes/beat)
`awburst`	M→S	2	Always 2'b01 (INCR)
`awvalid`	M→S	1	Write address and control valid
`awready`	S→M	1	Slave ready to accept write address
`awprot`†	M→S	3	Always 3'b000
`awcache`†	M→S	4	Always 4'b0010

Table 4.50: AXI4 W Channel (Write Data) – **NOT USED**; retained for reference only

Signal	Dir	Width	Description
wdata	M→S	64	One 64-bit beat of the cache line
wstrb	M→S	8	Always 8'hFF (full 64-bit write per beat)
wlast	M→S	1	Asserted on beat 3 (last beat)
wvalid	M→S	1	Write data valid
wready	S→M	1	Slave ready to accept write data

Table 4.51: AXI4 B Channel (Write Response) – **NOT USED**; retained for reference only

Signal	Dir	Width	Description
bid	S→M	4	Echo of awid
bresp	S→M	2	2'b00 = OKAY; error handling TBD
bvalid	S→M	1	Write response valid
bready	M→S	1	D-Cache ready to accept response

AXI4 SystemVerilog Interface Definition

```
interface as_axi4_if #(
  parameter int ADDR_W = 32,
  parameter int DATA_W = 64,
  parameter int ID_W   = 4,
  parameter int STRB_W = DATA_W / 8
)(
  input logic clk_i,
  input logic rst_i
);
  // AR channel
  logic [ID_W-1:0]   arid;
  logic [ADDR_W-1:0] araddr;
  logic [7:0]        arlen;
  logic [2:0]        arsize;
  logic [1:0]        arburst;
  logic              arvalid;
  logic aready;
  // R channel
  logic [ID_W-1:0]   rid;
```

```
  logic [DATA_W-1:0] rdata;
  logic [1:0]        rresp;
  logic              rlast;
  logic              rvalid;
  logic              rready;
  // AW channel
  logic [ID_W-1:0]   awid;
  logic [ADDR_W-1:0] awaddr;
  logic [7:0]        awlen;
  logic [2:0]        awsize;
  logic [1:0]        awburst;
  logic              awvalid;
  logic awready;
  // W channel
  logic [DATA_W-1:0] wdata;
  logic [STRB_W-1:0] wstrb;
  logic              wlast;
  logic              wvalid;
  logic wready;
  // B channel
  logic [ID_W-1:0]   bid;
  logic [1:0]        bresp;
  logic              bvalid;
  logic              bready;

  modport master (
    output arid, araddr, arlen, arsize, arburst, arvalid,
           rready, awid, awaddr, awlen, awsize, awburst,
           awvalid, wdata, wstrb, wlast, wvalid, bready,
    input  arready, rid, rdata, rresp, rlast, rvalid,
           awready, wready, bid, bresp, bvalid
  );
  modport slave (
    input  arid, araddr, arlen, arsize, arburst, arvalid,
           rready, awid, awaddr, awlen, awsize, awburst,
           awvalid, wdata, wstrb, wlast, wvalid, bready,
    output arready, rid, rdata, rresp, rlast, rvalid,
           awready, wready, bid, bresp, bvalid
  );
endinterface
```

Cache-Line Fill Transaction (Read Burst)

Figure 4.9 shows the timing of a 4-beat INCR read burst used for a cache-line fill. `ARLEN=3` encodes 4 beats (AXI4 convention: beats = ARLEN+1).

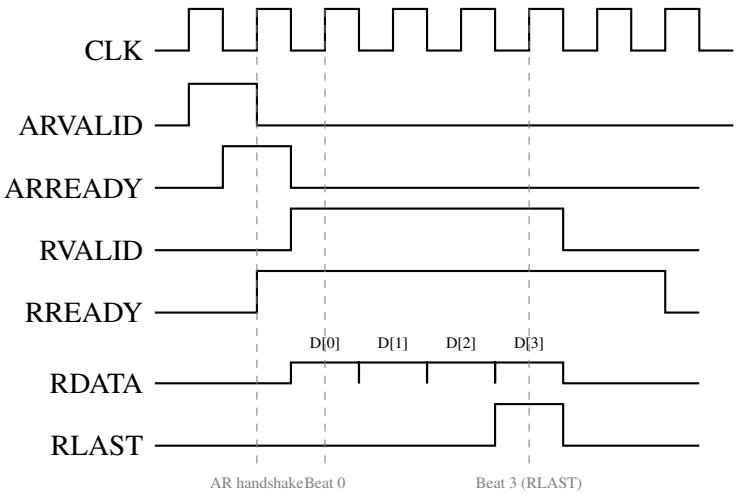


Figure 4.9: AXI4 Cache-Line Fill: 4-Beat INCR Read Burst (ARLEN=3)

Cache-Line Write-Back Transaction – NOT IMPLEMENTED

This section is retained for reference only. The current D-Cache design has no write-back path (AW/W/B channels eliminated; see Section 4.8.3).

Figure 4.10 shows how a 4-beat INCR write burst would look if write-back were added in a future revision. The AW and W channels are issued simultaneously.

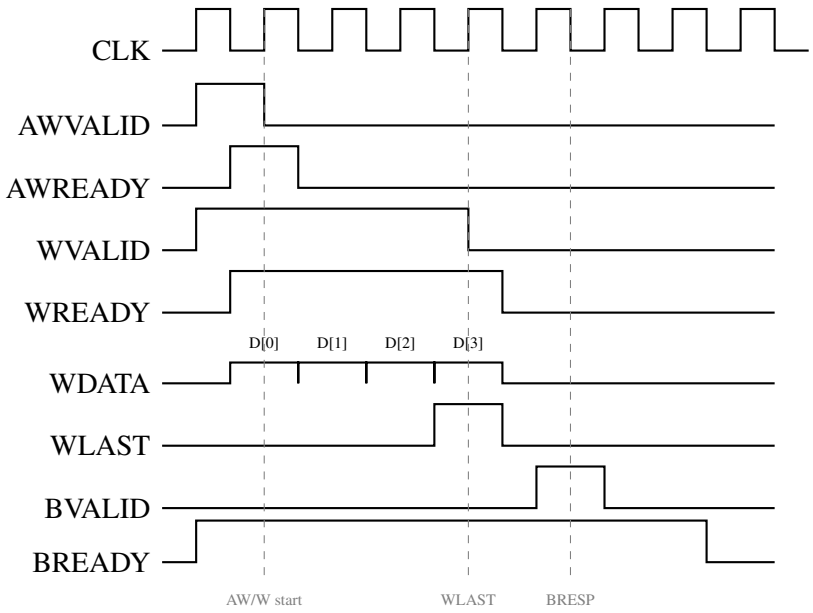


Figure 4.10: AXI4 Cache Write-Back: 4-Beat INCR Write Burst (AWLEN=3)

4.8.14 Memory Arbiter

Arbitration Policy

The Memory Arbiter serialises concurrent AXI4 requests from the I-Cache and D-Cache controllers onto the single AXI4 port that drives the QSPI controller. It implements a **fixed-priority** scheme with D-Cache having higher priority than I-Cache.

The rationale is that D-Cache misses in the MEM stage of the pipeline stall the entire pipeline, while I-Cache misses only stall the IF stage and can be partially hidden by a deeper instruction buffer in future pipeline implementations.

Table 4.52: Memory Arbiter Priority

Priority	Master	Notes
1 (highest)	D-Cache	Data miss (read-only; no write-back); stalls the entire pipeline
2	I-Cache	Instruction miss; stalls IF only

An active transaction is never interrupted: the arbiter grants the bus to one master for the full burst (ARLEN+1 beats or AWLEN+1 beats) before switching to the other.

Note: A round-robin policy with equal priority or dynamic priority based on stall urgency is a future option.

Memory Arbiter Interface

Table 4.53: Memory Arbiter Ports

Port	Type	Description
`icache_axi4`	AXI4 slave	AXI4 master port of I-Cache controller
`dcache_axi4`	AXI4 slave	AXI4 master port of D-Cache controller
`qspi_axi4`	AXI4 master	AXI4 slave port of QSPI controller

The arbiter is purely combinatorial (pass-through mux) for AR, R, AW, W, and B channels; it adds no pipeline registers. ID bits from the selected master are forwarded without modification; the QSPI controller sees only one active ID at a time.

4.8.15 QSPI Controller Interface

The QSPI controller presents an AXI4 slave port to the Memory Arbiter. From the arbiter's perspective the QSPI controller is a slow memory: it may de-assert `ARREADY` / `WREADY` for many cycles while the QSPI flash transaction completes.

The cache controllers handle this latency transparently because they hold `RREADY` asserted throughout the fill and wait for `RLAST` before returning data to the CPU.

Note: The QSPI controller itself is specified in a separate section of this document. Its AXI4 slave interface must be compatible with the parameter set in Table 4.46. In particular it must support INCR bursts of length 4 (ARLEN=3) at ARSIZE=3.

4.8.16 Cache Flush and Invalidation – FENCE.I

Overview

A cache flush invalidates all lines in a cache so that subsequent accesses reload data from the backing store. In the RWU-RV64I memory architecture, both I-Cache and D-Cache are **read-allocate with no write-back**; a flush therefore consists solely of clearing all `valid` bits and all PLRU bits in the tag SRAM. No dirty eviction is required (MEM-01 resolved).

Trigger Mechanism

The flush is triggered by the **FENCE.I** instruction (RISC-V Zifencei extension, encoding `0x0000100F`). When the instruction decoder recognises FENCE.I it:

1. Stalls the CPU pipeline.
2. Pulses `ic_flush_i` and `dc_flush_i` high for one clock cycle.
3. Waits until both `ic_flush_done_o` and `dc_flush_done_o` are asserted.
4. Releases the stall and advances the PC to PC+4.

Both caches execute their `FLUSH` state machines in parallel; the total stall duration is therefore **32 clock cycles** (one write per set, 32 sets). Step 1 above (pipeline stall) is what makes this safe on the pipelined `cc_cpupipefwd` core: FENCE.I stalls IF/ID/EX/MEM before the flush pulse is issued, so any older fetch or load already in flight has drained through the pipeline (or is held stalled) by the time `ic_flush_i` / `dc_flush_i` are asserted; no in-flight cache transaction races the flush.

No CSR is provided for cache control; FENCE.I is the sole software-visible mechanism. Scratchpad SRAM is not a cache and has no valid bits; it is unaffected by FENCE.I.

Flush State Machine

Both the I-Cache and D-Cache controllers implement an identical `FLUSH` state entered when `*_flush_i` is sampled high in `IDLE`:

Table 4.54: FLUSH State – Operation per Clock Cycle

Cycle	Action
0...30	Write `valid[3:0]=4'b0000, plru[2:0]=3'b000` to tag SRAM at address `flush_cnt[4:0]`; increment `flush_cnt`
31	Same write for set 31; assert `*_flush_done_o` for one cycle; return to `IDLE`

The 5-bit counter `flush_cnt` is implemented inside each cache controller and is reset to zero on entry to `FLUSH`. The `*_stall_o` signal remains asserted throughout the 32 cycles and is de-asserted together with `*_flush_done_o`.

Software Usage Guidelines

- **Self-modifying code:** not supported. All executable code must reside in NOR Flash; the scratchpad is data-only. FENCE.I is therefore not required for normal program flow.

- **DMA transfers:** if a future DMA engine writes to the Flash address region (e.g. via a shadow RAM or XIP staging area), the CPU must issue FENCE.I after the DMA completes and before executing any updated code. See MEM-04 for the coherency protocol.

- **Power-on reset:** both caches initialise with all valid bits cleared (reset value of tag SRAM flip-flops); no software flush is required at boot.

- **Flush latency:** 32 cycles. At a hypothetical 10 MHz clock (100 ns cycle) this is 3.2 µs – negligible for all expected use cases.

4.8.17 DMA Coherency Protocol

Overview

A DMA engine transfers data between memory regions without CPU intervention. Coherency problems arise when the CPU has cached a memory location and the DMA engine reads or writes that same location, causing one party to see stale data.

The RWU-RV64I cache architecture minimises DMA coherency complexity through two structural properties:

1. **Scratchpad is uncacheable.** All CPU stores bypass the D-Cache and go directly to the Scratchpad SRAM. A DMA engine accessing the Scratchpad always sees current data without any cache action.

2. **D-Cache is read-allocate with no dirty lines.** The D-Cache only caches Flash reads. Flash is read-only hardware; a DMA engine cannot write to it. Therefore the D-Cache can never hold data that is newer than the backing store.

Recommendation: place all DMA source and destination buffers in the Scratchpad address region. This eliminates every coherency concern at zero hardware cost.

DMA Access Classification

Table 4.55 enumerates all source/destination combinations and the required CPU action.

Table 4.55: DMA Coherency Requirements by Memory Region

DMA source	DMA destination	Coherency issue?	Required CPU action
Scratchpad	Peripheral	None	None. Scratchpad is uncacheable; DMA reads current data.
Peripheral	Scratchpad	None	None. CPU reads Scratchpad directly; no cache involved.
Scratchpad	Scratchpad	None	None. Both ends uncacheable.
Flash	Scratchpad	None	None. Flash data in D-Cache always matches Flash (read-only).
Flash	Peripheral	None	None. As above.
Scratchpad	Flash	N/A	Flash is read-only hardware; this transfer is not possible.
Flash	Flash	N/A	Flash is read-only hardware as destination; not possible.
Peripheral	Flash-mapped cached region[†]	**Yes**	CPU must issue `FENCE.I` after DMA completes to invalidate stale I-Cache and D-Cache lines.

[†]Applicable only if a future memory map revision introduces a writable RAM region within the cacheable address space (e.g. an on-chip SRAM shadow of the Flash area). In the current architecture this case does not arise.

Software Protocol

For the one coherency-relevant scenario (DMA writes to a cacheable region):

1. CPU configures the DMA transfer (source, destination, length, direction) by writing to DMA control registers (peripheral region, uncacheable).

2. CPU starts the DMA engine and waits for the DMA-done event (interrupt or polling of a status register).

3. CPU issues `FENCE.I` to invalidate both I-Cache and D-Cache (32 cycles).

4. CPU may now fetch instructions from, or load data out of, the DMA-written region; it will see the updated content.

No hardware handshake between the DMA engine and the cache controllers is required. The software ordering (wait for DMA done, then FENCE.I) is sufficient because:

- The DMA engine writes directly to memory (not through any cache).
- FENCE.I invalidates the cache after the DMA write is visible in memory.
- FENCE.I stalls the `cc_cpupipefwd` pipeline before triggering the flush (Section 4.8.16), so it is atomic with respect to subsequent loads even though the CPU is pipelined.

Future DMA Engine Interface Requirements

When the DMA engine is designed, it must expose at minimum:

Table 4.56: Minimum DMA Engine Signals for Cache Coherency

Signal	Type	Purpose
`dma_done_irq`	out (to CPU)	Interrupt asserted when DMA transfer completes; CPU uses this as the trigger to issue `FENCE.I` before accessing DMA-written data
`dma_active_o`	out (to system)	High while a transfer is in progress; optional, used to prevent conflicting CPU accesses to the same address range

No cache-flush or cache-snoop signal is required from the DMA engine; the software protocol (Section above) is sufficient.

4.8.18 AXI4 Error Handling

Overview

The AXI4 read response channel (`RRESP[1:0]`) carries a completion status from the QSPI controller back to the cache controllers. A non-OKAY response indicates that the underlying NOR Flash transaction failed (e.g. timeout, command error, or address out of Flash range).

In the RWU-RV64I design the AXI4 write channels (AW, W, B) are eliminated from the cache controllers (MEM-01); `BRESP` is therefore not observed and requires no handling.

AXI4 errors are surfaced to software as standard **RISC-V Machine-Mode Exceptions** following the Privileged Architecture specification, reusing the same trap-entry mechanism (`mepc`/`mcause`/`mstatus`, Section 3.3.8) already implemented for illegal-instruction and misaligned-address traps, so the trap handler can log, retry, or halt in a well-defined way. This is implemented end-to-end in `RV_PipelineCacheFwd` (`cc_cpupipefwd.sv`) for I-Cache fetch errors and D-Cache *load* errors; see Section 4.8.18 for the exact scope and the one deliberate limitation (D-Cache store errors). `RV_NoPipeline` has no cache/AXI4 subsystem and is unaffected.

AXI4 Response Codes

Table 4.57: AXI4 RRESP Codes Handled by Cache Controllers

`RRESP`	Name	Used?	Action
2'b00	OKAY	Yes	Normal completion; data written to cache SRAM
2'b01	EXOKAY	No	Exclusive access OK; not used (no LR/SC in this design)
2'b10	SLVERR	Yes	Slave error (QSPI / Flash fault); triggers exception
2'b11	DECERR	Yes	Decode error (address unmapped); triggers exception

Exception Mapping

Table 4.58 maps AXI4 error events to RISC-V exception causes.

Table 4.58: AXI4 Error to RISC-V Exception Mapping

Cache	AXI4 signal	mcause	Exception name	mtval
I-Cache	RRESP $\neq$ OKAY	1	Instruction Access Fault	Faulting fetch address (ic_addr_i at time of miss)
D-Cache	RRESP $\neq$ OKAY	5	Load Access Fault	Faulting load address (dc_addr_i at time of miss)

Error Detection in the Cache Controllers

Both cache controllers monitor RRESP on every beat of the R channel during the FILL state. If any beat carries RRESP $\neq$ 2'b00:

1. The cache controller immediately stops writing to the data SRAM.
2. The valid bit for the partially filled line is **not** set (the line remains invalid).
3. The controller transitions to the new ERR state.
4. In ERR, the controller pulses the error output (ic_err_o or dc_err_o) for one cycle and asserts the corresponding stall signal for that cycle.
5. The controller returns to IDLE.

The CPU samples the error output on the same cycle as the stall de-assertion, latches the faulting address, and raises the appropriate exception.

Updated Cache Controller FSMs

The ERR state is added to both I-Cache and D-Cache FSMs.

Table 4.59: ERR State – I-Cache and D-Cache (identical behaviour)

Condition	Action
Entry condition	Any AXI4 R-channel beat with RRESP $\neq$ 2'b00 during FILL
In ERR (1 cycle)	Do not update tag SRAM valid bit. Assert *_err_o for one cycle. Assert *_stall_o for the same cycle.
Exit condition	Unconditional; return to IDLE after one cycle

Updated Cache Interface Signals

One error output signal is added to each cache interface.

Table 4.60: Error Signals Added to Cache Interfaces

Signal	Interface	Dir (CPU)	Width	Description
ic_err_o	as_icache_if	in	1	Pulsed high for one cycle on an I-Cache AXI4 read error; CPU raises Instruction Access Fault (mcause=1)
dc_err_o	as_dcache_if	in	1	Pulsed high for one cycle on a D-Cache AXI4 read error; CPU raises Load Access Fault (mcause=5)

CPU Exception Handling Sequence

Implemented in cc_cpupipefwd.sv (RV_PipelineCacheFwd only – RV_NoPipeline has no cache/AXI4 subsystem). Because ic_err/dc_err arrive mid-pipeline (an I-Cache error replaces what would otherwise have been a fetched instruction; a D-Cache error replaces what would otherwise have been a completed load), a same-cycle-detected AXI4 error cannot simply set a flag and jump – icache_stall_s/dcache_stall_s must first release on ic_err/dc_err exactly as they already do on ic_rvalid/dc_rvalid (otherwise the pipeline would hang forever waiting for a completion that will never arrive), and a fault flag is threaded through the same per-instruction pipeline registers (if_id/id_ex/ex_mem/mem_wb) that already carry the illegal-instruction trap flag, so the fault is attributed to the correct instruction slot and reaches the existing commit-stage (wb_trap_taken_s) trap logic without any early-redirect special-casing:

1. mepc ← the faulting instruction's own PC: the address that could not be fetched (I-Cache case) or the PC of the load instruction whose data access errored (D-Cache case) – same PC-threading mechanism already used for illegal/misaligned traps, not a separately latched address.

2. mcause ← 1 (I-Cache, *Instruction access fault*) or 5 (D-Cache, *Load access fault*).

3. `mtval` ← the faulting address: the fetch address (I-Cache) or the load's data address (D-Cache, already available as `ex_mem_aluRes_r`/`mem_wb_aluRes_r` – no separate address latch needed either). `mtval` is a new CSR (address `0x343`), hardware-write-only: readable via `CSRRS`/etc. like `mcause`, but – like `mcause` – has no software write path. All other trap types (illegal, misaligned, `ecall`/`ebreak`, external IRQ) leave `mtval` at 0; the RISC-V privileged spec permits this simplification for causes where the value is not otherwise specified.

4. `mstatus.MPIE` ← `mstatus.MIE`; `mstatus.MIE` ← 0 (same trap-entry sequence as every other trap, Section 3.3.8).

5. PC ← `mtvec` (Direct mode, as for every other trap).

Scope limitation – D-Cache store errors are not covered. Store requests are posted: unlike loads, `dcache_stall_s` does not wait for a store's completion (the CPU can move on before the D-Cache's internal write/eviction sequence finishes), so a store's AXI4 error can arrive several cycles after the triggering store instruction has already left the pipeline – there is no precise instruction to attribute it to without a store-completion tracker this design does not have. This matches the original scope of this table (D-Cache → *Load* Access Fault only); it was never claimed for stores.

The trap handler at `mtvec` may:

- **Halt:** for a non-recoverable Flash fault this is the correct default action (e.g. output error code via UART/GPIO, then `wfi`).
- **Retry:** re-execute the faulting instruction (`mret` after optionally re-initialising the QSPI controller). Appropriate for transient QSPI timeouts.
- **Substitute:** fill a scratchpad buffer with alternative data, remap the faulting address (if an MPU is present), and `mret`. Advanced; not required for the first tapeout.

Updated SystemVerilog Interfaces

```
// as_icache_if additions:
logic ic_err;// new: AXI4 read error → Instruction Access Fault
modport cpu  (..., input ic_err);
modport cache(..., output ic_err);

// as_dcache_if additions:
logic dc_err;// new: AXI4 read error → Load Access Fault
```

```
modport cpu  (..., input dc_err);
modport cache(..., output dc_err);
```

Full updated interface definitions are in Sections 4.8.12 and 4.8.12.

4.8.19 QSPI Operating Mode – Cache-Miss-Driven Read

XIP Terminology Clarification

The term Execute-In-Place (XIP) is used with two distinct meanings in the literature:

- **True XIP (Continuous Read mode):** the QSPI controller keeps the Flash chip-select asserted and the device in a special Continuous Read state; subsequent reads require only the address (no repeated opcode), reducing per-transaction overhead. Requires the Flash to support a Performance Enhance or XIP mode bit.
- **Indirect XIP (cache-miss-driven):** the CPU executes code that resides in Flash, but each cache miss triggers a complete SPI command sequence (opcode + address + dummy + data). The chip-select is asserted only for the duration of each burst. No special Flash mode is required.

Decision: the RWU-RV64I uses **indirect XIP** (cache-miss-driven burst mode). True XIP mode is neither necessary nor advantageous for this architecture, as explained below.

Rationale

True XIP mode is not adopted for the following reasons:

1. **The I-Cache already provides the XIP benefit.** Once a cache line is loaded, all subsequent accesses to it cost 1 clock cycle. The amortised QSPI overhead per instruction is negligible at realistic hit rates (>90 %).
2. **Sequential access is not guaranteed.** A 4-way set-associative cache evicts and reloads lines at non-sequential addresses. True XIP's main advantage (no repeated opcode/address) applies only to sequential reads; for random-address cache refills its benefit is minimal.
3. **Simpler QSPI controller.** Indirect XIP requires no Flash-mode-bit management, no continuous-read entry/exit sequence, and no special handling for the transition back from XIP mode (required before write/erase operations in True XIP).

4. **Lower power between cache misses.** With CS# de-asserted between bursts the Flash enters standby mode automatically, reducing leakage current.

QSPI Command Sequence per Cache Miss

On each cache miss the QSPI controller executes the following sequence on the SPI bus:

1. Assert CS# (chip select, active-low).
2. Send read opcode on IO0 (single-bit SPI).
3. Send 24-bit (3-byte) Flash address on IO0; address is taken directly from `ARADDR[24:0]` (for Flash devices up to 16 MB) or `ARADDR[31:0]` for larger devices. The lower 5 bits are always zero (cache-line aligned burst).
4. Send dummy cycles (device-specific; typically 8 clock cycles for Fast Read, 6–8 for Quad Fast Read).
5. Receive 32 bytes of read data, output on IO0 (Fast Read) or IO0–IO3 simultaneously (Quad Fast Read). Data is returned to the AXI4 R channel as 4 beats of 64 bits, LSB-first within each beat.
6. De-assert CS#.

Table 4.61: Supported QSPI Read Commands

Command	Opcode	Mode	Throughput	Notes
Fast Read	`0x0B`	1-bit SPI	Low	Fallback; maximum QSPI clock limited by Flash spec
Quad Fast Read	`0x6B`	4-bit QSPI	High	**Recommended**; data on IO0–IO3 in parallel; requires QUAD enable bit set in Flash status register

The Quad Fast Read command (`0x6B`) is recommended because it reduces the data phase from 256 QSPI clock cycles (single-bit) to 64 QSPI clock cycles (four-bit), significantly lowering cache-miss latency.

Address Mapping

The AXI4 `ARADDR` is used directly as the Flash physical address:

$$\texttt{Flash_Addr}[24:5] = \texttt{ARADDR}[24:5], \quad \texttt{Flash_Addr}[4:0] = 5'b00000$$

The five least-significant address bits are always zero because the cache controller aligns all burst requests to the 32-byte cache-line boundary before issuing the AXI4 transaction.

Latency Estimate

Table 4.62 estimates the QSPI bus cycles and system clock cycles consumed by a single cache-line fill using Quad Fast Read. Actual values depend on the Flash device datasheet and the QSPI clock divider setting.

Table 4.62: Cache-Line Fill Latency Estimate (Quad Fast Read, `0x6B`)

Phase	QSPI clocks	Notes
CS# assert + opcode	8	8-bit opcode on IO0
Address	24	24-bit address on IO0
Dummy cycles	8	Device-specific; 8 typical for `0x6B`
Data (32 bytes)	64	256 bits ÷ 4 (quad) = 64 QSPI clocks
CS# de-assert	2	Minimum CS# high time (device-specific)
Total	**106**	Per cache miss

At an example QSPI clock of 25 MHz and system clock of 10 MHz:

$$t_{\text{miss}} = \frac{106}{25\,\text{MHz}} = 4.24\,\mu\text{s} \mathrel{\widehat{=}} 42 \text{ system clock cycles.}$$

This is consistent with the "$\gg$10 cycles" estimate in Table 4.30. The exact QSPI clock frequency and divider ratio are to be determined once the target NOR Flash device is selected.

NOR Flash Device Requirements

The selected NOR Flash must support:

- **Quad Fast Read (0x6B)** with up to 8 dummy cycles.

- **3-byte address mode** (24-bit, supports Flash up to 16 MB); or 4-byte mode if a larger device is required.
- **AXI4 burst compatibility:** the device must sustain continuous 4-bit output for 64 QSPI clocks without inserting wait states.
- **3.3 V supply** (matching XH035 I/O voltage).

4.8.20 Memory Hierarchy Performance Analysis

Average Memory Access Time (AMAT)

The standard metric for cache hierarchy performance is the **Average Memory Access Time (AMAT)**:

$$\text{AMAT} = t_{\text{hit}} + m_1 \cdot t_{\text{miss}}$$

where t_{hit} is the cache hit latency, m_1 is the L1 miss rate, and t_{miss} is the miss penalty. For the current two-level hierarchy (L1 cache + NOR Flash) the values from this document are:

- $t_{\text{hit}} = 1$ cycle (cache hit, combinatorial tag compare + SRAM read).
- $t_{\text{miss}} = 42$ system clock cycles (cache-line fill from NOR Flash via QSPI Quad Fast Read at 25 MHz QSPI / 10 MHz system; see Table 4.62).

Thus:

$$\text{AMAT}_{\text{2-level}} = 1 + m_1 \cdot 42 \quad \text{[system cycles]}$$

Hypothetical Three-Level Hierarchy

If an additional on-chip SRAM layer (L2) were inserted between the L1 caches and NOR Flash, the AMAT becomes:

$$\text{AMAT}_{\text{3-level}} = 1 + m_1 \cdot \left(t_{\text{L2}} + m_2 \cdot 42\right)$$

where t_{L2} is the L2 SRAM access time (1–2 cycles for an on-chip synchronous SRAM) and m_2 is the fraction of L1 misses that also miss in L2. The resulting speed-up $S = \text{AMAT}_{\text{2-level}}/\text{AMAT}_{\text{3-level}}$ is shown in Table 4.63.

Table 4.63: AMAT and Speed-Up for a Hypothetical L2 SRAM ($t_{\mathrm{L2}} = 2$ cycles, $t_{\mathrm{miss}} = 42$ cycles)

L1 miss rate m_1	L2 miss rate m_2	$\mathrm{AMAT}_{\text{2-level}}$	$\mathrm{AMAT}_{\text{3-level}}$	Speed-Up
5 %	0 % (program fits in L2)	3.1	1.10	$2.8\times$
10 %	0 %	5.2	1.20	$4.3\times$
20 %	0 %	9.4	1.40	$6.7\times$
10 %	20 % (L2 80 % hit rate)	5.2	2.04	$2.5\times$
10 %	50 %	5.2	3.10	$1.7\times$

The maximum benefit occurs when the active working set fits entirely in L2 ($m_2 \approx 0\,\%$), in which case NOR Flash is accessed only during the initial boot load and AMAT approaches $1 + m_1 \cdot t_{\mathrm{L2}}$.

System-Clock Scaling of the Miss Penalty

The QSPI transaction consumes a fixed number of QSPI clock cycles (106 at 25 MHz QSPI; see Table 4.62). Converting to system clock cycles:

$$t_{\mathrm{miss}}[\mathrm{sys}] = \frac{106}{f_{\mathrm{QSPI}}} \cdot f_{\mathrm{sys}} = 106 \cdot \frac{f_{\mathrm{sys}}}{f_{\mathrm{QSPI}}}$$

Table 4.64: Miss Penalty vs. System Clock (QSPI fixed at 25 MHz)

System clock	QSPI clock	Miss penalty	Remark
10 MHz	25 MHz	42 cycles	Reference design point
25 MHz	25 MHz	106 cycles	Penalty $2.5\times$ longer
50 MHz	25 MHz	212 cycles	Penalty $5\times$ longer

The penalty in system cycles grows linearly with the system clock when the QSPI clock is held constant. An additional on-chip SRAM layer therefore becomes increasingly beneficial as the system clock is raised.

Design Decision

The current architecture intentionally omits an L2 cache layer:

- **Chip area:** At X-Fab XH035 (350 nm) a true L2 cache (tag array, replacement logic, coherency protocol) consumes significant die area.

- **Complexity:** An L2 controller adds coherency and arbitration logic on top of the existing Memory Arbiter.

- **Scratchpad covers the critical path:** Stack, heap, `.data`, and `.bss` reside in the Scratchpad SRAM (1–2 cycle access); these are the write-intensive segments that would most benefit from low latency. Only code (`.text`) and read-only data (`.rodata`) remain in the Flash/cache path.

- **4 KB L1 miss rate:** For small embedded programs that fit in the 4 KB I-Cache, the instruction miss rate is low after the initial cache warm-up, limiting the absolute time spent on Flash accesses.

Should the system clock be raised significantly above 10 MHz or should larger programs with a high L1 miss rate become a requirement, adding a Tightly Coupled Memory (TCM) layer – an on-chip SRAM preloaded from Flash at boot – would be the recommended approach. A full L2 cache is not recommended for the first tape-out on this process node.

4.8.21 SystemVerilog Module Architecture

see CPU chapter

4.8.22 Open Items and Refinements

Table 4.65: Open Items – Memory Concept v0.1

ID	Description	Owner
MEM-01	**[RESOLVED]** Scratchpad SRAM chosen as read-write region. D-Cache operates as read-allocate only for the Flash address region; scratchpad is uncacheable (bypass path in D-Cache controller). Dirty bit and `WRITEBACK` state eliminated.	–
MEM-02	**[IN PROGRESS]** XH035 SPRAM library selected; five instances defined (I-Tag 96×32, D-Tag 96×32, I-Data 256×128, D-Data 256×128, Scratchpad 64×1024). Macro names and timing parameters (Table 4.40) to be filled after running the SPRAM compiler. See Section 4.8.9.	TBD
MEM-03	**[RESOLVED]** FENCE.I (Zifencei, opcode `0x0000100F`) triggers full invalidation of both I-Cache and D-Cache in 32 cycles. No CSR provided. Scratchpad unaffected. No dirty eviction (MEM-01). See Section 4.8.16.	–
MEM-04	**[RESOLVED]** Scratchpad is uncacheable; DMA buffers placed there require no cache action. For DMA writes to cacheable regions (future): CPU waits for `dma_done_irq`, then issues `FENCE.I`. No hardware handshake between DMA and cache controllers required. See Section 4.8.17.	–
MEM-05	Physical address width: 32 bits chosen (`PA_WIDTH=32`). With scratchpad and Flash regions both well within a 4 GB space this is confirmed sufficient; finalise when memory map is defined.	AS: 32 b / ok
MEM-06	**[RESOLVED]** RRESP ≠ OKAY mapped to RISC-V exceptions: I-Cache → Instruction Access Fault (`mcause`=1), D-Cache load → Load Access Fault (`mcause`=5). ERR state added to both FSMs; `ic_err_o`/`dc_err_o` signals added to cache interfaces **and now actually consumed by the CPU** (`cc_cpupipefwd.sv`: fault-flag threaded through the pipeline, new `mtval` CSR) – this end-to-end CPU wiring, and the top-level `dc_err` routing in `asTopMem.sv` that had been silently tied to 0, were only completed 2026-07-14; the cache-level half of this entry predates that fix. D-Cache *store* errors remain imprecise/unattributed (posted writes, no completion tracker) – out of scope, see Section 4.8.18. See Section 4.8.18.	–
MEM-07	**[RESOLVED]** Cache-miss-driven burst (indirect XIP) chosen. True XIP (Continuous Read mode) not	–

4.9 System Protection and Security Concept

Table 4.66: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.10 Debug Concept

Table 4.67: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.11 Register Concept

Table 4.68: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

Chapter 5

RWU-RV64I Core

5.1 ALU

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.1: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.2: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)

Functional Configuration of ALU0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a ALU peripheral are described in its functional description in Section 5.1.2. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for ALU0 in Table 5.3.

The **RWU-RV64I** has only one ALU.

Table 5.3: ALU0 Feature Set

	Generic GPIO Feature Set	Details
no	Seperate Kernel Clock	not needed
no	FIFO Data Buffering	not needed

HW Interfaces

The following table 5.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.4: HW Interfaces of Module ALU0

	Module internal name	Chip level name
Supply		
Clock	-	-
Bus	-	-
Interrupt	-	-
DMA	-	-
External Signals	-	-
Others	data01_i(63:0) data02_i(63:0) aluSel_i(4:0) aluZero_o aluNega_o aluCarr_o aluOver_o aluResult_o(63:0)	srcA_s(63:0) srcB_s(63:0) aluSel_s(4:0) zero_s nega_s carry_s overflow_s dBusAddr_s(63:0)

Bus Interface

None.

Registers

None.

Related Sections of this Spec

None.

5.1.1 History of ALU

Table 5.5: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
5.1	5.1 / 2025-02-13	First draft (AS)

5.1.2 Functional Overview

The ALU operates on 64 bit data and generates 64 bit data. The ALU indicates a zero result, a negative result, the carry of a result, and the overflow.

General Features:

- 64 bit integer addition
- 64 bit integer subtraction
- 64 bit AND
- 64 bit OR
- 64 bit XOR
- 64 bit set less than
- 64 bit set less than, unsigned
- 32 bit shift right arithmetic word
- 32 bit shift right logical word
- 32 bit shift left logical word
- 32 bit subtract word
- 32 bit add word

- 64 bit shift right arithmetic
- 64 bit shift right logical
- 64 bit shift left logical
- branch if equal
- branch if not equal
- branch if less than
- branch if greater or equal
- branch if less than, unsigned
- branch if greater or equal, unsigned

Synchronous Mode:

- -

Asynchronous Mode:

- -

5.1.3 Structural Overview

I/O of the ALU

Table 5.6 shows the input and outputs of the ALU0. This are all primary inputs and outputs. There is no bi-directional, tristate or open drain pin. All pins are high-active except the pins with a _n_ in its name.

Table 5.6: ALU0 I/O

Pin	Dir.	Wd.	Explanation
data01_i	in	64	Source data input. Comes either from the register file or the program conter (PC). Swiched by signal “aluSrcA”. Source one for all ALU operators.
data02_i	in	64	Source data input. Comes either from the register file or the immediate generator. Swiched by signal “aluSrcB”. Source two for all ALU operators.
aluSel_i	in	5	Selects the ALU operator.
aluZero_o	out	1	Zero flag of the ALU. Not buffered.
aluNega_o	out	1	Negative flag of the ALU. Not buffered.
aluCarr_o	out	1	Carry flag of the ALU. Not buffered.
aluOver_o	out	1	Overflow flag of the ALU. Not buffered.
aluResult_o	out	64	The result of each ALU operation.

Internals of the ALU

The following list (Table) shows the internal signals of the ALU. The table shows the signal name, the width, the driver, the target and a short explanation.

- Signal: aluSel_i[0]
  - Width: 1 bit
  - Source: aluSel_i
  - Destination: aluResult_o, overf_s
  - Explanation (special signal):
    - if aluSel_i[0] = 0, it is an ADD instruction
    - ... the second operand must be taken as it is
    - ... the carry-in of the addition is 0 (aluSel_i[0])
    - if aluSel_i[0] = 1, it is a SUB instruction
    - ... the second operand must be the 2-complement
    - ... the carry-in of the addition is 1 which acts as the +1 (aluSel_i[0])
- Signal: data01_s
  - Width: 64 bit, signed
  - Source: data01_i
  - Destination: aluResult_o, aluZero_o

  - Explanation: It is the signed interpretation of data01_i. It is needed for some branches (BLT, BGE) and SRA.

    logic signed [reg_width-1:0] data01_s;

- Signal: data02_s
  - Width: 64 bit, signed
  - Source: data02_i
  - Destination: aluResult_o, aluZero_o
  - Explanation: It is the signed interpretation of data02_i. It is needed for some branches (BLT, BGE) and SRA.

    logic signed [reg_width-1:0] data02_s;

- Signal: sum_s
  - Width: 64 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluResult_o
  - Explanation: It is the sum of the two data inputs data01_i and data02_i. The second data input could be negative and so 2-complemented. cinvb_s is the inversion of data02_e when $aluSel_i[0] = 1$, aluSel_i[0] also acts as the plus 1.

    assign {carry_s, sum_s} = data01_i + cinvb_s + aluSel_i[0];

- Signal: cinvb_s
  - Width: 64 bit
  - Source: data02_i, aluSel_i[0]
  - Destination: sum_s, carry_s
  - Explanation: cinvb_s is the inversion of data02_i when $aluSel_i[0] = 1$. Together with aluSel_i[0] ($cinvb_s+aluSel_i[0]$) it is the 2-complement of data02_i.

    assign cinvb_s = aluSel_i[0] ? data02_i : data02_i;

- Signal: carry_s

  - Width: 1 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluCarr_o (carry flag)
  - Explanation: It is the carry of sum_s.

    assign {carry_s, sum_s} = data01_i + cinvb_s + aluSel_i[0];

- Signal: sumw_s
  - Width: 32 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluResult_o
  - Explanation: For add word, etc. It is the sum of the two data inputs data01_i(31:0) and data02_i(31:0. The second data input could be negative and so 2-complemented. cinvbw_s is the inversion of data02_e(31:0) when $aluSel_i[0] = 1$, aluSel_i[0] also acts as the plus 1.

    assign {carryw_s, sumw_s} = data01_i[31:0] + cinvbw_s + aluSel_i[0];

- Signal: cinvbw_s
  - Width: 32 bit
  - Source: data02_e, aluSel_i[0]
  - Destination: sum_s, carry_s
  - Explanation: For add word, etc. cinvbw_s is the inversion of data02_i(31:0) when $aluSel_i[0] = 1$. Together with aluSel_i[0] ($cinvbw_s+aluSel_i[0]$) it is the 2-complement of data02_i(31:0).

    assign cinvbw_s = aluSel_i[0] ? data02_i[31:0] : data02_i[31:0];

- Signal: carryw_s
  - Width: 1 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluCarr_o (carry flag)
  - Explanation: For add word, etc. It is the carry of sumw_s.

    assign {carryw_s, sumw_s} = data01_i[31:0] + cinvbw_s + aluSel_i[0];

- Signal: sllw_s
  - Width: 32 bit
  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SLLW (shift left logical word)
    Implementation:
    $sllw_s = data01_i(31:0) << data02_i(4:0)$
    aluResult_o is sllw_s, sign extended.
- srlw_s
  - Width: 32 bit
  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SRLW (shift right logical word)
    Implementation:
    $srlw_s = data01_i(31:0) >> data02_i(4:0)$
    aluResult_o is srlw_s, sign extended.
- sraw_s
  - Width: 32 bit, signed
  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SRAW (shift right algorithmic word)
    Implementation:
    $sraw_s = data01w_s >>> data02_i(4:0)$
    aluResult_o is sraw_s, sign extended.
- data01w_s
  - Width: 32 bit, signed
  - Source: data01_i
  - Destination: sraw_s
  - Explanation: 32 bit signed representation of data01_i.
- sltiu_s
  - Width: 1 bit

  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SLT, SLTU (set less than; signed, unsigned)
    Implementation:
    if(data01_i < data02_i)
    sltiu_s = 1;
    else
    sltiu_s = 0;
    aluResult_o is sltiu_s, zero extended.

- overf_s
  - Width: 1 bit
  - Source: data01_i, data02_i, sum_s, aluSel_i[0]
  - Destination: aluOver_o
  - Explanation: Overflow occurs when the result-value affects the sign:
    - overflow when adding two positives yields a negative
    - or, adding two negatives gives a positive
    - or, subtract a negative from a positive and get a negative
    - or, subtract a positive from a negative and get a positive

- aluResult_o
  - Width: 64 bit
  - Source: all signals
  - Destination: output
  - Explanation: The result of each ALU operation.
    If aluSel_i =

0	aluResult_o = sum_s;	ADD, ADDI, LB, LH, LW, LBU, LHU, LD, LWU, AUIPC, LUI, SB, SH, SW, SD, JALR, JAL
1	aluResult_o = sum_s;	SUB
2	aluResult_o = data01_i & data02_i;	AND, ANDI
3		OR
4		XOR
5		SLT
6		SLTU

5.1.4 Service Requests of the ALU

None.

5.1.5 The ALU Peripheral Kernel

The ALU calculates a result (aluResult_o) and the corresponding flags zero (aluZero_o), negative (aluNega_o), carry (asCarr_o) and overflow (asOver_o). Table 5.7 and table 5.8 are describing the function of the ALU for each instruction.

The zero flag is not only a zero flag but also the control for all branch instruction.

Table 5.7 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Table 5.7: RV32I Base Integer Instruction Set [2], [3], [1]

Instruction	Operation	Remarks
Load Upper Immediate	-	-
Add Upper Immediate to PC	-	-
Jump and Link	*jal rd, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow pc + offs$
Jump and Link Register	*jalr rd, rs1, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow (rs1 + offs) \wedge -2$
Branch Equal	*beq rs1, rs2, offs*	$if\ rs1 = rs2\ then\ pc \leftarrow pc + offs$
Branch Not Equal	*bne rs1, rs2, offs*	$if\ rs1 \neq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than	*blt rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal	*bge rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than Unsigned	*bltu rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal Unsigned	*bgeu rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Load Byte	*lb rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half	*lh rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Load Word	*lw rd, offs(rs1)*	$rd \leftarrow s32[rs1 + offs]$

... next page

Instruction	Operation	Remarks
Load Byte Unsigned	*lbu rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half Unsigned	*lhu rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Store Byte	*sb rs2, offs(rs1)*	$u8[rs1 + offs] \leftarrow rs2$
Store Half	*sh rs2, offs(rs1)*	$u16[rs1 + offs] \leftarrow rs2$
Store Word	*sw rs2, offs(rs1)*	$u32[rs1 + offs] \leftarrow rs2$
Add Immediate	*addi rd, rs1, imm*	$rd \leftarrow rs1 + sx(imm)$
Set Less Than Immediate	*slti rd, rs1, imm*	$rd \leftarrow sx(rs1) < sx(imm)$
Set Less Than Immediate Unsigned	*sltiu rd, rs1, imm*	$rd \leftarrow ux(rs1) < ux(imm)$
Xor Immediate	*xori rd, rs1, imm*	$rd \leftarrow ux(rs1) \oplus ux(imm)$
Or Immediate	*ori rd, rs1, imm*	$rd \leftarrow ux(rs1) \vee ux(imm)$
And Immediate	*andi rd, rs1, imm*	$rd \leftarrow ux(rs1) \wedge ux(imm)$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << ux(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> ux(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> ux(imm)$
Add	aluResult_o = sum_s = data01_i + cinvb_s + aluSel_i[0]	aluSel_i = 0; cinvb_s = aluSel_i[0] ? not data02_i : data02_i
Subtract	*sub rd, rs1, rs2*	$rd \leftarrow sx(rs1) - sx(rs2)$
Shift Left Logical	*sll rd, rs1, rs2*	$rd \leftarrow ux(rs1) << rs2$
Set Less Than	*slt rd, rs1, rs2*	$rd \leftarrow sx(rs1) < sx(rs2)$
Set Less Than Unsigned	*sltu rd, rs1, rs2*	$rd \leftarrow ux(rs1) < ux(rs2)$
Xor	*xor rd, rs1, rs2*	$rd \leftarrow ux(rs1) \oplus ux(rs2)$
Shift Right Logical	*srl rd, rs1, rs2*	$rd \leftarrow ux(rs1) >> rs2$
Shift Right Arithmetic	*sra rd, rs1, rs2*	$rd \leftarrow sx(rs1) >> rs2$
Or	*or rd, rs1, rs2*	$rd \leftarrow ux(rs1) \vee ux(rs2)$
And	*and rd, rs1, rs2*	$rd \leftarrow ux(rs1) \wedge ux(rs2)$
Fence	*fence pred, succ*	
Fence Instruction	*fence.i*	

End of table.

Table 5.8 lists all base integer instructions of a RISC-V 64 bit architecture, additional to the 32 bit integer instruction set, according the specification [1].

Table 5.8: RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]

Instruction	Example	Meaning
Load Word Unsigned	*lwu rd, offs(rs1)*	$rd \leftarrow u32[rs1 + offs]$
Load Double	*ld rd, offs(rs1)*	$rd \leftarrow u64[rs1 + offs]$
Store Double	*sd rs2, offs(rs1)*	$u64[rs1 + offs] \leftarrow rs2$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << sx(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> sx(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> sx(imm)$
Add Immediate Word	*addiw rd, rs1, imm*	$rd \leftarrow s32(rs1) + imm$
Shift Left Logical Immediate Word	*slliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) << imm)$
Shift Right Logical Immediate Word	*srliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) >> imm)$
Shift Right Arithmetic Immediate Word	*sraiw rd, rs1, imm*	$rd \leftarrow s32(rs1) >> imm$
Add Word	*addw rd, rs1, rs2*	$rd \leftarrow s32(rs1) + s32(rs2)$
Subtract Word	*subw rd, rs1, rs2*	$rd \leftarrow s32(rs1) - s32(rs2)$
Shift Left Logical Word	*sllw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) << rs2)$
Shift Right Logical Word	*srlw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) >> rs2)$
Shift Right Arithmetic Word	*sraw rd, rs1, rs2*	$rd \leftarrow s32(rs1) >> rs2$

End of table.

5.1.6 Registers

None.

5.2 Instruction Decoder

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.9: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.10: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)

Functional Configuration of InstructionDecode0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a Instruction Decoder are described in its functional description in Section 5.2.2. Generic features, that can be decided for each specific

instantiation of the peripheral are listed and completed for InstructionDecode0 in Table 5.11.

The **RWU-RV64I** has only one Instruction Decoder.

Table 5.11: InstrDec0 Feature Set

	Generic InstrDec Feature Set	Details
no	Seperate Kernel Clock	Pure combinational block (`as_instr_decode`); no clock input, no internal state
no	FIFO Data Buffering	not needed
yes	Synchronous exception request	`trap_illegal_instr_o`, see Section 5.2 Service Requests

HW Interfaces

The following table 5.12 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.12: HW Interfaces of Module InstrDec0

	Module internal name	Chip level name
Supply		
Clock	-	- (pure combinational block, no clk_i/rst_i)
Bus	-	-
Interrupt	trap_illegal_instr_o	*RV_NoPipeline* (asCPUx.sv): trap_illegal_ *RV_PipelineCache* (cc_cpupipefw id_trap_illeg Both feed mcause=2 directly; see Section 3.3.8.
DMA	-	-
External Signals	-	-
Others	instr_opcode_i(6:0) instr_func3_i(2:0) instr_func7b5_i take_i	*NoPipeline*: ir_s(6:0). *PipelineCacheFwd* if_id_instr_ ir_s(14:12) / if_id_instr_ ir_s(30) / if_id_instr_ *NoPipeline*: take_s (branch-taken feedback from the

Both chip variants instantiate the identical, shared `as_instr_decode` module (`RWU_RV/RV_NoPipeline/src/asInstrDecode.sv`); only the chip-level net names at the instantiation point differ, as listed above.

Bus Interface

None.

Registers

None.

Related Sections of this Spec

None.

5.2.1 History of Instruction Decoder

Table 5.13: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
5.2	5.2 / 2025-02-13	First draft (AS)

5.2.2 Functional Overview

The Instruction Decoder operates on 32 bit instructions and generates all control signals needed for the operation of the CPU.

General Features:

- Decodes all instructions of the RV64I:
  - opcode=15 (`0x0F`): I-type, `fence`/`fence.i` – decoded identically (opcode only, `func3` not distinguished) as a legal no-op; every other control output stays at its default value.

  - opcode=3: I-type, loads
  - opcode=19: I-type, immediate arithmetics, logic and shift
  - opcode=23: U-type, auipc
  - opcode=27: I-type, immediate arithmetics, logic and shift (word)
  - opcode=35: S-type, stores
  - opcode=51: R-type, RV32I
  - opcode=55: U-type, lui
  - opcode=59: R-type, RV64I
  - opcode=99: B-type, all conditional branches
  - opcode=103: I-type, jalr
  - opcode=111: J-type, jal
  - opcode=115 (`0x73`): I-type, CSR instructions (`csrrw`/`csrrs`/`csrrc`/`csrrwi`/`csrrsi`/`csrrci`), `mret`, `ecall`, `ebreak` – the decoder itself only distinguishes CSR instructions (`func3` $\neq 000$, register write enabled) from `mret` (`func3` $= 000$, no register write); the finer distinction between `mret`, `ecall`, and `ebreak` (all `func3` $= 000$) is made outside this module, directly on the instruction word, by the CPU top level (`as_cpux`/`cc_cpupipefwd`) – see Section 3.3.8.

- generates the control signal for the ALU,
- ... for the D-Mem MUX,
- ... for the D-Mem read- and write enable,
- ... for the PC MUX,
- ... for the register A and B output MUXes,
- ... for the register file write enable,
- ... for the branch control,
- ... for the immediate generation selection,
- ... for the jump and branch control in the pipelined version,
- **raises `trap_illegal_instr_o`** for any opcode/func3 combination not listed above – the CPU's synchronous illegal-instruction exception (`mcause` = 2, Section 3.3.8) is sourced directly and exclusively from this signal.

Synchronous Mode:

- -

Asynchronous Mode:

- -

5.2.3 Structural Overview

I/O of the Instruction Decoder

Table 5.14 shows the input and outputs of the Instruction Decoder (`as_instr_decode`), matching `RWU_RV/RV_NoPipeline/src/asInstrDecode.sv`. This is a purely combinational module: no `clk_i`/`rst_i`, no internal state, all outputs are a function of the current inputs only. There is no bi-directional, tristate or open drain pin.

Table 5.14: InstrDec0 I/O

Pin	Dir.	Wd.	Explanation
instr_opcode_i	in	7	The opcode field (instruction bits [6:0]).
instr_func3_i	in	3	The func3 field (instruction bits [14:12]); further selects the operation within an opcode.
instr_func7b5_i	in	1	Bit 5 of the func7 field (instruction bit [30]); distinguishes e.g. ADD/SUB and the logical/arithmetic shift variants.
take_i	in	1	Branch-taken feedback. Not read by the decoder's combinational logic (kept only for interface compatibility with an earlier design iteration; marked `UNUSEDSIGNAL` for Verilator).
mux_resultSrc_o	out	2	`result_src_t` enum (`RES_ALU`=0, `RES_MEM`=1, `RES_PC4`=2, `RES_CSR`=3). Selects the mux feeding the register-file write data: ALU result, D-Mem load data, PC+4, or CSR read value.
en_dMemWr_o	out	1	D-Mem write enable.
en_dMemRd_o	out	1	D-Mem read enable.
mux_aluSrcB_o	out	1	ALU source-B mux select: 0 = register (rs2), 1 = immediate.
mux_aluSrcA_o	out	2	`mux_a_t` enum

The following ports from an earlier design iteration no longer exist on `as_instr_decode`: `opcode_i`, `func3_i`, `func7b5_i` (renamed to the `instr_*` forms above), `zero_i` (branch-taken detection now lives entirely in the branch ALU / `br_op_o` path, not the decoder), `PCSrc_o` (superseded by `br_op_o` = `BR_ALWAYS` plus the branch ALU's own `take_o`), `aluSel_o` (renamed and widened to `alu_op_o`), `branch_s_o`/`jump_s_o` (folded into `br_op_o`).

Internals of the Instruction Decoder

The following list shows the internal signals and per-opcode decode behaviour of the Instruction Decoder, matching `RWU_RV/RV_NoPipeline/src/asInstrDecode.sv` exactly (a single combinational `case (instr_opcode_i)` statement; all outputs default to `ALU_ADD`/`BR_NONE`/0/`SRC_REGA`/ `IMM_NONE`/`RES_ALU` before the case, so every branch below only lists the outputs it changes from that default).

- Signal: `instr_opcode_i`
  - Width: 7 bit unsigned
  - Source: Instruction from I-Mem/I-Cache.
  - Destination: `case` statement generating all other outputs.
  - Explanation:
    - opcode=15 (`0x0F`, `fence`/`fence.i`):
      - All outputs stay at their default value (no register write, no D-Mem access, `ALU_ADD`/`BR_NONE`, `trap_illegal_instr_o`=0). Decoded purely on opcode; `instr_func3_i` is not inspected.
    - opcode=3 (loads):
      - en_regWr_o=1; mux_aluSrcB_o=1 (immediate); sel_immSrc_o=IMM_I; mux_resultSrc_o=RES_MEM; alu_op_o=ALU_ADD (address = rs1+imm); en_dMemRd_o=1
    - opcode=19 (immediate arithmetic/logic/shift):
      - en_regWr_o=1; mux_aluSrcB_o=1; sel_immSrc_o=IMM_I; alu_op_o selected by `instr_func3_i`/`instr_func7b5_i` (see below); `trap_illegal_instr_o` cannot be set here – all 8 `func3` values are decoded.
    - opcode=23 (auipc):
      - en_regWr_o=1; sel_immSrc_o=IMM_U; mux_aluSrcA_o=SRC_PC; mux_aluSrcB_o=1; alu_op_o=ALU_ADD
    - opcode=27 (immediate arithmetic/shift, word):

  - en_regWr_o=1; mux_aluSrcB_o=1; sel_immSrc_o=IMM_I; alu_op_o ∈ {ALU_ADDW, ALU_SLLW, ALU_SRLW, ALU_SRAW} (see `instr_func3_i` below); only `func3` ∈ {000,001,101} are valid – any other value sets `trap_illegal_instr_o`=1.
* opcode=35 (stores):
  - en_dMemWr_o=1; mux_aluSrcB_o=1; sel_immSrc_o=IMM_S; alu_op_o=ALU_ADD
* opcode=51 (R-type, RV32I arithmetic/logic/shift):
  - en_regWr_o=1; mux_aluSrcA_o=SRC_REGA; mux_aluSrcB_o=0 (register); alu_op_o selected by `instr_func3_i`/ `instr_func7b5_i` (see below); all 8 `func3` values decoded, `trap_illegal_instr_o` cannot be set here.
* opcode=55 (lui):
  - en_regWr_o=1; sel_immSrc_o=IMM_U; mux_aluSrcA_o=SRC_ZERO; mux_aluSrcB_o=1; alu_op_o=ALU_ADD (result = 0 + imm)
* opcode=59 (R-type, RV64I word arithmetic/shift):
  - en_regWr_o=1; alu_op_o ∈ {ALU_ADDW/SUBW, ALU_SLLW, ALU_SRLW/SRAW}; only `func3` ∈ {000,001,101} are valid – any other value sets `trap_illegal_instr_o`=1.
* opcode=99 (conditional branches):
  - sel_immSrc_o=IMM_B; br_op_o selected by `instr_func3_i` (see below); only `func3` ∈ {000,001,100,101,110,111} are valid – 010 and 011 are reserved and set `trap_illegal_instr_o`=1.
* opcode=103 (jalr):
  - en_regWr_o=1; sel_immSrc_o=IMM_I; mux_jump_o=1; mux_resultSrc_o=RE
    br_op_o=BR_ALWAYS
* opcode=111 (jal):
  - en_regWr_o=1; sel_immSrc_o=IMM_J; mux_resultSrc_o=RES_PC4; br_op_o=BR_ALWAYS
* opcode=115 (`0x73`, SYSTEM – CSR/`mret`/ `ecall`/`ebreak`):
  - mux_resultSrc_o=RES_CSR always; en_regWr_o=1 unless `instr_func3_i`=000 (`mret`/`ecall`/ `ebreak` do not write `rd`). The decoder does not further distinguish `mret`, `ecall`, and `ebreak` from one another (all three are legal, `func3`=000, differing only in `instr[31:20]`) – that distinction, and the CSR address decode for the six `csrr*` variants, happens outside this module in the CPU top level (`as_cpux`/`cc_cpupipefwd`); see Section 3.3.8.

    - default (any other opcode):
        - trap_illegal_instr_o=1; all other outputs stay at their default (inert) value.

- Signal: `instr_func3_i`
  - Width: 3 bit, unsigned
  - Source: Instruction from I-Mem/I-Cache.
  - Destination: nested `case` statements selecting `alu_op_o` (opcodes 19/27/51/59) or `br_op_o` (opcode 99); also gates `en_regWr_o` for opcode 115.
  - Explanation (`alu_op_o` selection; `instr_func7b5_i` distinguishes the two rows sharing the same `func3`):
    - opcode=19 (immediate, 64-bit width): func3=000 → `ALU_ADD`; 001 → `ALU_SLL`; 010 → `ALU_SLT`; 011 → `ALU_SLTU`; 100 → `ALU_XOR`; 101 → `ALU_SRL` (`func7b5_i`=0) or `ALU_SRA` (=1); 110 → `ALU_OR`; 111 → `ALU_AND`
    - opcode=27 (immediate, word width): func3=000 → `ALU_ADDW`; 001 → `ALU_SLLW`; 101 → `ALU_SRLW` (`func7b5_i`=0) or `ALU_SRAW` (=1)
    - opcode=51 (register, 64-bit width): func3=000 → `ALU_ADD` (`func7b5_i`=0) or `ALU_SUB` (=1); 001 → `ALU_SLL`; 010 → `ALU_SLT`; 011 → `ALU_SLTU`; 100 → `ALU_XOR`; 101 → `ALU_SRL` (`func7b5_i`=0) or `ALU_SRA` (=1); 110 → `ALU_OR`; 111 → `ALU_AND`
    - opcode=59 (register, word width): func3=000 → `ALU_ADDW` (`func7b5_i`=0) or `ALU_SUBW` (=1); 001 → `ALU_SLLW`; 101 → `ALU_SRLW` (`func7b5_i`=0) or `ALU_SRAW` (=1)
  - Explanation (`br_op_o` selection, opcode=99): func3=000 → `BR_EQ` (`beq`); 001 → `BR_NE` (`bne`); 100 → `BR_LT` (`blt`); 101 → `BR_GE` (`bge`); 110 → `BR_LTU` (`bltu`); 111 → `BR_GEU` (`bgeu`); 010/011 → `trap_illegal_instr_o`=1 (reserved encodings).

- Signal: `instr_func7b5_i`
  - Width: 1 bit, unsigned
  - Source: Instruction from I-Mem/I-Cache (instruction bit [30]).
  - Destination: the `alu_op_o` case branches listed above.

  - Explanation: distinguishes ADD/SUB and the logical/arithmetic shift variants (SRL/SRA, SRLI/SRAI, SRLW/SRAW, SRLIW/SRAIW) that share the same `func3` encoding.

- Signal: `mux_resultSrc_o`
  - Width: 2 bit, `result_src_t` enum
  - Source: generated from the opcode, see `instr_opcode_i` above.
  - Destination: WB-stage result mux (ALU result / D-Mem load data / PC+4 / CSR read value → register file write data).
  - Explanation: `RES_ALU`=0 (default, most instructions), `RES_MEM`=1 (loads), `RES_PC4`=2 (`jal`/`jalr`), `RES_CSR`=3 (opcode 115, SYSTEM).

- Signal: `en_dMemWr_o`
  - Width: 1 bit, unsigned
  - Source: generated from the opcode (stores only).
  - Destination: D-Mem/D-Cache write enable.
  - Explanation: Write-enable for the data memory.

- Signal: `en_dMemRd_o`
  - Width: 1 bit, unsigned
  - Source: generated from the opcode (loads only).
  - Destination: D-Mem/D-Cache read enable; also gates the load-use hazard stall in `RV_PipelineCacheFwd` (Section 4.2.1).
  - Explanation: Read-enable for the data memory.

- Signal: `mux_aluSrcB_o`
  - Width: 1 bit, unsigned
  - Source: generated from the opcode.
  - Destination: ALU source-B input mux.
  - Explanation:

    $0 \Rightarrow$ register (rs2) to ALU source B
    $1 \Rightarrow$ immediate value to ALU source B

- Signal: `mux_aluSrcA_o`

  - Width: 2 bit, `mux_a_t` enum
  - Source: generated from the opcode.
  - Destination: ALU source-A input mux.
  - Explanation: `SRC_REGA`=0 (register rs1, default), `SRC_PC`=1 (`auipc`), `SRC_ZERO`=2 (`lui`).

- Signal: `en_regWr_o`
  - Width: 1 bit, unsigned
  - Source: generated from the opcode (and, for opcode 115, `instr_func3_i`).
  - Destination: Register-file write enable.
  - Explanation: Write enable for the register file.

- Signal: `mux_jump_o`
  - Width: 1 bit, unsigned
  - Source: opcode=103 (`jalr`) only.
  - Destination: Branch-target-adder source-A select.
  - Explanation:

    $0 \Rightarrow$ PC to branch-target-adder input A (default: branches, JAL)
    $1 \Rightarrow$ register rs1 to branch-target-adder input A (JALR)

- Signal: `sel_immSrc_o`
  - Width: 3 bit, `imm_src_t` enum
  - Source: generated from the opcode.
  - Destination: Immediate generator format select.
  - Explanation:

    `IMM_NONE` $= 0 \Rightarrow$ no immediate (R-type, FENCE, default)
    `IMM_I` $= 1 \Rightarrow$ I-type (loads, immediate arithmetic, jalr)
    `IMM_S` $= 2 \Rightarrow$ S-type (stores)
    `IMM_B` $= 3 \Rightarrow$ B-type (branches)
    `IMM_U` $= 4 \Rightarrow$ U-type (lui, auipc)
    `IMM_J` $= 5 \Rightarrow$ J-type (jal)

- Signal: `alu_op_o`
  - Width: 6 bit declared (`alu_op_t`), 15 values used

  - Source: generated from the opcode and, where applicable, `instr_func3_i`/`instr_f` (see above).
  - Destination: Algorithmic ALU operation select.
  - Explanation:

    0 ⇒ `ALU_ADD` (ADD, ADDI, loads, stores, jal/jalr target, auipc, lui, default)
    1 ⇒ `ALU_SUB`
    2 ⇒ `ALU_AND`
    3 ⇒ `ALU_OR`
    4 ⇒ `ALU_XOR`
    5 ⇒ `ALU_SLT`
    6 ⇒ `ALU_SLTU`
    7 ⇒ `ALU_SLL`
    8 ⇒ `ALU_SRL`
    9 ⇒ `ALU_SRA`
    10 ⇒ `ALU_ADDW`
    11 ⇒ `ALU_SUBW`
    12 ⇒ `ALU_SLLW`
    13 ⇒ `ALU_SRLW`
    14 ⇒ `ALU_SRAW`

- Signal: `br_op_o`
  - Width: 3 bit, `br_op_t` enum
  - Source: opcode 99 (branches, via `instr_func3_i`) and opcodes 103/111 (`jalr`/`jal`, unconditional).
  - Destination: Branch ALU operation select (`take_o` generation, both chip variants – see Section 5.1).
  - Explanation:

    0 ⇒ `BR_NONE` (default, no branch/jump)
    1 ⇒ `BR_EQ` (beq)
    2 ⇒ `BR_NE` (bne)
    3 ⇒ `BR_LT` (blt)
    4 ⇒ `BR_GE` (bge)
    5 ⇒ `BR_LTU` (bltu)
    6 ⇒ `BR_GEU` (bgeu)
    7 ⇒ `BR_ALWAYS` (jal, jalr – unconditional)

- Signal: `trap_illegal_instr_o`

- Width: 1 bit, unsigned
- Source: the case-statement default (unrecognized opcode) and the `default` arms of the `instr_func3_i` sub-cases for opcodes 27, 59, and 99 (the only opcodes whose 8 possible `func3` values are not all valid RV64I encodings).
- Destination: CPU top-level trap logic (`as_cpux`/ `cc_cpupipefwd`); sets `mcause`=2.
- Explanation: see Section 3.3.8 for the full trap-priority chain and CSR update sequence.

5.2.4 Service Requests of the Instruction Decoder

Table 5.15: Instruction Decoder Service Requests

Service Request	Explanation
`trap_illegal_instr_o`	Synchronous, combinational request: any `instr_opcode_i`/ `instr_func3_i` combination that does not decode to a valid RV64I instruction. This is the sole source of the illegal-instruction exception (`mcause` = 2) on both chip variants – the decoder itself does not distinguish `ecall`, `ebreak`, or `mret` (all legal opcode=115 encodings from its point of view); those are recognized directly on the instruction word by the CPU top level. See Section 3.3.8 (Interrupt Concept) for the full trap-priority chain and CSR update sequence this signal feeds into.

The Instruction Decoder has no other service request: it is a combinational block with no clock, no interrupt/DMA request of its own kind, and no error condition beyond the single illegal-instruction case above.

5.2.5 The InstrDec Peripheral Kernel

The Instruction Decoder decodes all RV64I instructions and delivers the control signals to all design elements in the CPU.

Table 5.16 lists every RV64I base integer instruction this core decodes (RV32I base set [2], [3], [1] plus the RV64I word-width and Zicsr/privileged additions),

with the exact control-signal values produced by `as_instr_decode.sv` for each. FENCE/FENCE.I and the nine SYSTEM-opcode instructions (missing from earlier revisions of this table) are now included.

Legend:

- Ins: Instruction short
- op: `instr_opcode_i`
- f3: `instr_func3_i`
- f7: `instr_func7b5_i`
- S: `mux_resultSrc_o` (0=`RES_ALU`, 1=`RES_MEM`, 2=`RES_PC4`, 3=`RES_CSR`)
- W: `en_dMemWr_o`
- R: `en_dMemRd_o`
- B: `mux_aluSrcB_o`
- A: `mux_aluSrcA_o` (0=`SRC_REGA`, 1=`SRC_PC`, 2=`SRC_ZERO`)
- r: `en_regWr_o`
- j: `mux_jump_o`
- im: `sel_immSrc_o` (0=`IMM_NONE`, 1=`IMM_I`, 2=`IMM_S`, 3=`IMM_B`, 4=`IMM_U`, 5=`IMM_J`)
- alu: `alu_op_o` (0..14, see Table 5.14/the Internals list for the name mapping; shown as "-" where the ALU result is not used, i.e. the field holds the don't-care default `ALU_ADD`=0)
- br: `br_op_o` (0=`BR_NONE` .. 6=`BR_GEU`, 7=`BR_ALWAYS`)

`trap_illegal_instr_o`=0 for every row in this table by construction (they are exactly the valid encodings); see Table 5.14 and the Internals list (Section 5.2) for which `func3` values are not listed here and therefore trap.

Table 5.16: RV32I/RV64I Base Integer Instruction Set, incl. FENCE/SYSTEM [2], [3], [1]

Ins	op	f3	f7	S	W	R	B	A	r	j	im	alu	br
fence / fence.i	0001111	-	-	0	0	0	0	0	0	0	0	-	0
lb	0000011	000	-	1	0	1	1	0	1	0	1	0	0
lh	0000011	001	-	1	0	1	1	0	1	0	1	0	0
lw	0000011	010	-	1	0	1	1	0	1	0	1	0	0
lbu	0000011	100	-	1	0	1	1	0	1	0	1	0	0
lhu	0000011	101	-	1	0	1	1	0	1	0	1	0	0
ld	0000011	011	-	1	0	1	1	0	1	0	1	0	0
lwu	0000011	110	-	1	0	1	1	0	1	0	1	0	0
addi	0010011	000	-	0	0	0	1	0	1	0	1	0	0
slli	0010011	001	0	0	0	0	1	0	1	0	1	7	0
slti	0010011	010	-	0	0	0	1	0	1	0	1	5	0
sltiu	0010011	011	-	0	0	0	1	0	1	0	1	6	0
xori	0010011	100	-	0	0	0	1	0	1	0	1	4	0
srli	0010011	101	0	0	0	0	1	0	1	0	1	8	0
srai	0010011	101	1	0	0	0	1	0	1	0	1	9	0
ori	0010011	110	-	0	0	0	1	0	1	0	1	3	0
andi	0010011	111	-	0	0	0	1	0	1	0	1	2	0
auipc	0010111	-	-	0	0	0	1	1	1	0	4	0	0
addiw	0011011	000	-	0	0	0	1	0	1	0	1	10	0
slliw	0011011	001	0	0	0	0	1	0	1	0	1	12	0
srliw	0011011	101	0	0	0	0	1	0	1	0	1	13	0
sraiw	0011011	101	1	0	0	0	1	0	1	0	1	14	0
sb	0100011	000	-	0	1	0	1	0	0	0	2	0	0
sh	0100011	001	-	0	1	0	1	0	0	0	2	0	0
sw	0100011	010	-	0	1	0	1	0	0	0	2	0	0
sd	0100011	011	-	0	1	0	1	0	0	0	2	0	0
add	0110011	000	0	0	0	0	0	0	1	0	0	0	0
sub	0110011	000	1	0	0	0	0	0	1	0	0	1	0
sll	0110011	001	0	0	0	0	0	0	1	0	0	7	0
slt	0110011	010	0	0	0	0	0	0	1	0	0	5	0
sltu	0110011	011	0	0	0	0	0	0	1	0	0	6	0
xor	0110011	100	0	0	0	0	0	0	1	0	0	4	0
srl	0110011	101	0	0	0	0	0	0	1	0	0	8	0
sra	0110011	101	1	0	0	0	0	0	1	0	0	9	0
or	0110011	110	0	0	0	0	0	0	1	0	0	3	0

... next page

Ins	op	f3	f7	S	W	R	B	A	r	j	im	alu	br
and	0110011	111	0	0	0	0	0	0	1	0	0	2	0
lui	0110111	-	-	0	0	0	1	2	1	0	4	0	0
addw	0111011	000	0	0	0	0	0	0	1	0	0	10	0
subw	0111011	000	1	0	0	0	0	0	1	0	0	11	0
sllw	0111011	001	0	0	0	0	0	0	1	0	0	12	0
srlw	0111011	101	0	0	0	0	0	0	1	0	0	13	0
sraw	0111011	101	1	0	0	0	0	0	1	0	0	14	0
beq	1100011	000	-	0	0	0	0	0	0	0	3	-	1
bne	1100011	001	-	0	0	0	0	0	0	0	3	-	2
blt	1100011	100	-	0	0	0	0	0	0	0	3	-	3
bge	1100011	101	-	0	0	0	0	0	0	0	3	-	4
bltu	1100011	110	-	0	0	0	0	0	0	0	3	-	5
bgeu	1100011	111	-	0	0	0	0	0	0	0	3	-	6
jalr	1100111	000	-	2	0	0	-	-	1	1	1	-	7
jal	1101111	-	-	2	0	0	-	-	1	0	5	-	7
csrrw	1110011	001	-	3	0	0	-	-	1	0	0	-	0
csrrs	1110011	010	-	3	0	0	-	-	1	0	0	-	0
csrrc	1110011	011	-	3	0	0	-	-	1	0	0	-	0
csrrwi	1110011	101	-	3	0	0	-	-	1	0	0	-	0
csrrsi	1110011	110	-	3	0	0	-	-	1	0	0	-	0
csrrci	1110011	111	-	3	0	0	-	-	1	0	0	-	0
ecall[(1)]	1110011	000	-	3	0	0	-	-	0	0	0	-	0
ebreak[(1)]	1110011	000	-	3	0	0	-	-	0	0	0	-	0
mret[(1)]	1110011	000	-	3	0	0	-	-	0	0	0	-	0

EoT

[(1)] `ecall`, `ebreak`, and `mret` are indistinguishable from one another at the `instr_opcode_i`/`instr_func3_i` granularity this table is keyed on (all three: op=1110011, f3=000); `as_instr_decode` does not need to tell them apart (all three get the same, shown, control signals). The CPU top level distinguishes them by inspecting `instr[31:20]` directly (`ecall=0x000`, `ebreak=0x001`, `mret=0x302`); see Section 3.3.8. Columns marked "-" hold their module-default value, which is never consumed for that instruction (e.g. `mux_aluSrcA_o`/`mux_aluSrcB_o`/`alu_op_o` for `jal`/`jalr`/SYSTEM: the register write-back value comes from `RES_PC4`/`RES_CSR`, not the main ALU, and the branch-target address is computed by a separate adder gated by `mux_jump_o`, not by `alu_op_o`).

5.2.6 Registers

None.

5.3 Memory System

This section specifies the SystemVerilog module decomposition of the memory subsystem. Each module is described with its complete port list (signal, direction, width, connected to/from, and purpose) and its internal algorithm. The module names follow the project-wide `as` prefix convention.

Module Hierarchy: Table 5.17 lists all modules. `asMemTop` is the sole instantiation point; the CPU core and the QSPI controller are outside this boundary.

Table 5.17: Memory Subsystem – SystemVerilog Module Overview

Module	Type	Purpose
`asMemTop`	top wrapper	Instantiates all sub-modules; routes CPU, SRAM and AXI4 interfaces
`asICacheCtrl`	RTL (FSM)	Instruction cache controller: tag compare, PLRU, AXI4 master (AR/R)
`asDCacheCtrl`	RTL (FSM)	Data cache controller: tag compare, PLRU, bypass path, AXI4 master (AR/R)
`asICacheBlock`	SRAM wrapper	Holds I-Tag SRAM (96 bit$\times$32) and I-Data SRAM (256 bit$\times$128)
`asDCacheBlock`	SRAM wrapper	Holds D-Tag SRAM (96 bit$\times$32) and D-Data SRAM (256 bit$\times$128)
`asSpBlock`	SRAM wrapper	Holds Scratchpad SRAM (64 bit$\times$1024)
`asMemArb`	RTL (mux)	Fixed-priority AXI4 arbiter: D-Cache $>$ I-Cache, single output port
`asSpramModel`	sim. model	Behavioural 1-cycle synchronous SRAM; replaces X-Fab SPRAM macro in simulation

The three SRAM wrapper modules (`asICacheBlock`, `asDCacheBlock`, `asSpBlock`) each instantiate one or two X-Fab XH035 SPRAM macros. In simulation the macros are replaced by `asSpramModel` via a synthesis guard (`` `ifdef SIMULATION ``).

5.3.1 System Integration of MemSys0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.18: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2026

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.19: History

Date	Name	Content Changes
Version of this document: V0.1		
2026-05-14	Andreas Siggelkow	First draft (AS, Claude Code)

Functional Configuration of MemSys0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a MemSys peripheral are described in its functional description in Section 5.3.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for MemSys0 in Table 5.20.

Table 5.20: Memory System Feature Set

	Generic MemSys Feature Set	Details
yes	Separate Kernel Clock	`clk_core_s` (CGU ÷ 80 from external `clk_i`); target frequency TBD for `RV_PipelineCacheFwd` – see the analogous `RV_NoPipeline` target in Clock Concept Section 4.5.2

HW Interfaces

The following table 5.21 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.21: HW Interfaces of Module MemSys0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
I-Cache-Bus	I-Cache-Bus slave	-
D-Cache-Bus	D-Cache-Bus slave	-
Interrupt	None	-
DMA	None	-
External Signals	None	-

Bus Interface

The memory system exposes three independent bus interfaces: AXI, I-Mem-IF, D-Mem-IF.

Registers

No registers.

Related Sections of this Spec

- MemSys functional description starting with Section 5.3.3.
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”
  - AXI4 Cache Bus Section “Architecture Concepts” (Bus Concept)
  - Peripheral ID Registers Section “Architecture Concepts”
  - Peripheral Clocking Concept Section “Architecture Concepts”
  - Clock Control Registers Section “Architecture Concepts”
  - Interrupt Source Combining Register Set Section “Architecture Concepts”
  - Synchronization Block Section “Architecture Concepts”
  - Standardized Flip-Flop based FIFO Section “Architecture Concepts”
  - TX-FIFO Registers Section “Architecture Concepts”
  - RX-FIFO Registers Section “Architecture Concepts”

5.3.2 History of MemSys

Table 5.22: History

History		
Target Spec.	Current version : 1.0, 2026-02-17 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2026-02-17		
5.3	5.3 / 2026-05-14	First draft (AS, Claude Code)

5.3.3 Functional Overview

Purpose: Top-level wrapper that wires the CPU interfaces, the SRAM blocks, the cache controllers, and the memory arbiter together. It presents three external interfaces:

- the CPU instruction bus,

- the CPU data bus, and

- the AXI4 master port toward the QSPI controller.

No combinatorial or sequential logic resides in this module beyond wire connections and interface bindings.

Internal algorithm: `asMemTop` contains no logic. It instantiates exactly the seven modules listed in Table **??** (excluding `asSpramModel`, which lives inside the SRAM wrappers) and connects them via the SV interfaces `as_icache_if`, `as_dcache_if`, and `as_axi4_if`. The SRAM wrappers are wired to their respective controllers through dedicated flat port groups (see Sections 5.3.4 through 5.3.4).

5.3.4 Structural Overview

The MemSys see Chapter "Architectural Concepts - Memory System"

Port list of the memory top-level:

Table 5.23: `asMemTop` Port List (1)

Signal	Dir	Width	Connected to	Description
Clock and Reset				
`clk_i`	in	1	All sub-modules	System clock (synchronous rising edge)
`rst_i`	in	1	All sub-modules	Asynchronous active-high reset
CPU Instruction Bus (via `as_icache_if`)				
`ic_addr_i`	in	64	`asICacheCtrl`	Fetch address from CPU IF stage
`ic_req_i`	in	1	`asICacheCtrl`	Fetch request valid
`ic_flush_i`	in	1	`asICacheCtrl`	FENCE.I pulse from CPU
`ic_stall_o`	out	1	CPU IF stage	Cache miss / flush in progress
`ic_flush_done_o`	out	1	CPU IF stage	Flush complete pulse
`ic_err_o`	out	1	CPU IF stage	AXI4 read error pulse
`ic_rdata_o`	out	32	CPU IF stage	Instruction word (32-bit)
`ic_rvalid_o`	out	1	CPU IF stage	Instruction word valid

Table 5.24: asMemTop Port List (2)

Signal	Dir	Width	Connected to	Description
CPU Data Bus (via as_dcache_if)				
dc_addr_i	in	64	asDCacheCtrl	Load/store address from CPU MEM stage
dc_req_i	in	1	asDCacheCtrl	Load/store request valid
dc_wr_i	in	1	asDCacheCtrl	1 = store, 0 = load
dc_size_i	in	3	asDCacheCtrl	Transfer size (byte/half/word/-double)
dc_wdata_i	in	64	asDCacheCtrl	Store data from CPU
dc_wstrb_i	in	8	asDCacheCtrl	Byte enables for store
dc_flush_i	in	1	asDCacheCtrl	FENCE.I pulse from CPU
dc_stall_o	out	1	CPU MEM stage	Cache miss / bypass / flush in progress
dc_flush_done_o	out	1	CPU MEM stage	Flush complete pulse
dc_err_o	out	1	CPU MEM stage	AXI4 read error pulse
dc_rdata_o	out	64	CPU MEM stage	Load data (64-bit)
dc_rvalid_o	out	1	CPU MEM stage	Load data valid
AXI4 Master Port toward QSPI Controller (via as_axi4_if)				
m_axi4	out	–	asQspiTop	AXI4 master bundle (AR, R channels only; see Table 4.46)

asICacheBlock – I-Cache SRAM Wrapper

Purpose. Wraps two X-Fab XH035 SPRAM macros: the I-Tag SRAM (96 bit × 32) and the I-Data SRAM (256 bit × 128). Translates the logical controller signals into the physical SPRAM port signals (CLK, CEN, WEN, BWEN, A, D, Q) described in Section **??**. In simulation the SPRAM macros are replaced by asSpramModel.

Port list of the I-cache:

Table 5.25: `asICacheBlock` Port List

Signal	Dir	Width	Connected to	Description
`clk_i`	in	1	SPRAM CLK	System clock
Tag SRAM port (I-Tag SRAM, 96 bit × 32)				
`it_addr_i`	in	5	`asICacheCtrl`	Set index (0–31)
`it_wdata_i`	in	96	`asICacheCtrl`	Tag write data: {PLRU[2:0], way3{valid,tag[21:0]}, …, way0{valid,tag[21:0]}}
`it_we_i`	in	1	`asICacheCtrl`	Tag write enable (active-high)
`it_ce_i`	in	1	`asICacheCtrl`	Tag chip enable (active-high)
`it_rdata_o`	out	96	`asICacheCtrl`	Tag read data; valid one cycle after `it_ce_i` on read
Data SRAM port (I-Data SRAM, 256 bit × 128)				
`id_addr_i`	in	7	`asICacheCtrl`	Address: {index[4:0], way[1:0]}
`id_wdata_i`	in	64	`asICacheCtrl`	One 64-bit beat to write; placed into the lane selected by `id_wlane_i`
`id_wlane_i`	in	2	`asICacheCtrl`	Beat index (0–3); selects 64-bit lane within 256-bit word
`id_we_i`	in	1	`asICacheCtrl`	Data write enable (active-high)
`id_ce_i`	in	1	`asICacheCtrl`	Data chip enable (active-high)
`id_rdata_o`	out	256	`asICacheCtrl`	Full 256-bit cache line; valid one cycle after `id_ce_i` on read

Internal algorithm. On a write (`it_we_i` or `id_we_i` high):

- Tag SRAM: `BWEN` is set to all-zero (write all 96 bits); `WEN` is driven low; `A` receives `it_addr_i`; `D` receives `it_wdata_i`.

- Data SRAM: `id_wlane_i` selects one 64-bit lane. `BWEN[255:0]` is constructed as: bits $[(lane+1)\cdot 64-1 \,:\, lane\cdot 64]$ are driven to `0` (write-enable); all other bits are `1` (mask). `D[63:0]` is replicated four times to fill `D[255:0]`; only the selected lane is actually written due to BWEN masking.

On a read (`it_ce_i` or `id_ce_i` high, write enables low): `WEN` is driven high; `BWEN` is irrelevant (ignored on reads by the SPRAM); the read data appears on `Q` one clock cycle later.

`asDCacheBlock` – D-Cache SRAM Wrapper

Purpose. Identical in structure to `asICacheBlock` but serves the D-Cache controller. Wraps D-Tag SRAM (96 bit $\times$ 32) and D-Data SRAM (256 bit $\times$ 128); same SPRAM macros, same timing, same simulation guard.

Port list of the D-cache:

Table 5.26: `asDCacheBlock` Port List

Signal	Dir	Width	Connected to	Description
`clk_i`	in	1	SPRAM CLK	System clock
Tag SRAM port (D-Tag SRAM, 96 bit × 32)				
`dt_addr_i`	in	5	`asDCacheCtrl`	Set index (0–31)
`dt_wdata_i`	in	96	`asDCacheCtrl`	Tag write data (same layout as I-Cache; no dirty bit)
`dt_we_i`	in	1	`asDCacheCtrl`	Tag write enable (active-high)
`dt_ce_i`	in	1	`asDCacheCtrl`	Tag chip enable (active-high)
`dt_rdata_o`	out	96	`asDCacheCtrl`	Tag read data; valid one cycle after chip enable
Data SRAM port (D-Data SRAM, 256 bit × 128)				
`dd_addr_i`	in	7	`asDCacheCtrl`	Address: {index[4:0], way[1:0]}
`dd_wdata_i`	in	64	`asDCacheCtrl`	One 64-bit AXI4 beat (fill path)
`dd_wlane_i`	in	2	`asDCacheCtrl`	Beat index (0–3) selecting 64-bit lane
`dd_we_i`	in	1	`asDCacheCtrl`	Data write enable (active-high)
`dd_ce_i`	in	1	`asDCacheCtrl`	Data chip enable (active-high)
`dd_rdata_o`	out	256	`asDCacheCtrl`	Full 256-bit cache line

Internal algorithm. Identical to `asICacheBlock`. The D-Tag SRAM contains no dirty bit (removed; see Section **??**): the 96-bit tag word layout is {PLRU[2:0], way3{valid[0],tag[21:0]}, way2{...}, way1{...}, way0{valid[0],tag[21:0]}}, identical to I-Cache.

`asSpBlock` – Scratchpad SRAM Wrapper

Purpose. Wraps a single X-Fab XH035 SPRAM macro configured as 64 bit × 1024 (8 KiB default; parameterisable via `SP_DEPTH`). Provides byte-level write access required for sub-word stores (byte, halfword, word) to stack, heap, `.data`, and `.bss`.

Port list of the scratch-pad memory:

Table 5.27: `asSpBlock` Port List

Signal	Dir	Width	Connected to	Description
`clk_i`	in	1	SPRAM CLK	System clock
`sp_addr_i`	in	10	`asDCacheCtrl`	Word address within Scratchpad (byte address bits [12:3])
`sp_wdata_i`	in	64	`asDCacheCtrl`	Write data (64-bit aligned; byte enables select active bytes)
`sp_wstrb_i`	in	8	`asDCacheCtrl`	Byte write enables (1 = write byte, 0 = mask); driven from `dc_wstrb_i`
`sp_we_i`	in	1	`asDCacheCtrl`	Write enable (active-high); 0 = read
`sp_ce_i`	in	1	`asDCacheCtrl`	Chip enable (active-high)
`sp_rdata_o`	out	64	`asDCacheCtrl`	Read data; valid one cycle after chip enable on a read

Internal algorithm.

- **Read** (`sp_we_i` low, `sp_ce_i` high): SPRAM `WEN` driven high; `CEN` driven low (active-low); `A` receives `sp_addr_i`; `sp_rdata_o` latches `Q` in the cycle after access.
- **Write** (`sp_we_i` high, `sp_ce_i` high): SPRAM `WEN` driven low; `CEN` driven low; `BWEN[63:0]` is formed as the bit-expansion of `~sp_wstrb_i`: each byte-enable bit expands to eight BWEN bits (0 = write, 1 = mask), enabling sub-byte granularity. `D` receives `sp_wdata_i`.
- **Idle**: `CEN` driven high (chip disabled, SPRAM in zero-power standby).

`asICacheCtrl` – Instruction Cache Controller

Purpose. Implements the I-Cache FSM (Table 4.41). Drives the `asICacheBlock` SRAM interface, accepts instruction fetch requests from the CPU via `as_icache_if`, and issues AXI4 AR/R burst transactions to `asMemArb` on a cache miss.

Port list of the I-cache controller:

Table 5.28: `asICacheCtrl` Port List (1)

Signal	Dir	Width	Connected to	Description
`clk_i`	in	1	`asMemTop`	System clock
`rst_i`	in	1	`asMemTop`	Asynchronous reset; clears FSM state and all tag valid bits
CPU Instruction Bus (`as_icache_if.cache` modport)				
`ic_addr_i`	in	64	CPU IF stage	Fetch address
`ic_req_i`	in	1	CPU IF stage	Fetch request valid
`ic_flush_i`	in	1	CPU IF stage	Invalidation request pulse
`ic_stall_o`	out	1	CPU IF stage	Stall: miss, flush, or error recovery
`ic_flush_done_o`	out	1	CPU IF stage	Flush complete pulse
`ic_err_o`	out	1	CPU IF stage	AXI4 RRESP error pulse
`ic_rdata_o`	out	32	CPU IF stage	Instruction word selected from hit line
`ic_rvalid_o`	out	1	CPU IF stage	`ic_rdata_o` valid
I-Tag SRAM port (to `asICacheBlock`)				
`it_addr_o`	out	5	`asICacheBlock`	Set index from `ic_addr_i[9:5]`
`it_wdata_o`	out	96	`asICacheBlock`	Tag word written on fill completion
`it_we_o`	out	1	`asICacheBlock`	Tag write enable
`it_ce_o`	out	1	`asICacheBlock`	Tag chip enable
`it_rdata_i`	in	96	`asICacheBlock`	Tag read data
I-Data SRAM port (to `asICacheBlock`)				
`id_addr_o`	out	7	`asICacheBlock`	{index[4:0], way[1:0]}
`id_wdata_o`	out	64	`asICacheBlock`	One AXI4 R-channel beat
`id_wlane_o`	out	2	`asICacheBlock`	Beat index (0–3)
`id_we_o`	out	1	`asICacheBlock`	Data write enable
`id_ce_o`	out	1	`asICacheBlock`	Data chip enable
`id_rdata_i`	in	256	`asICacheBlock`	Full 256-bit cache line

Table 5.29: `asICacheCtrl` Port List (2)

Signal	Dir	Width	Connected to	Description
AXI4 Master Port – AR and R channels only (to `asMemArb`)				
`arid_o`	out	4	`asMemArb`	Always 4'h1 (I-Cache ID)
`araddr_o`	out	32	`asMemArb`	Cache-line-aligned miss address (`ic_addr_i[31:5],5'b0`)
`arlen_o`	out	8	`asMemArb`	Always 8'h03 (4 beats)
`arsize_o`	out	3	`asMemArb`	Always 3'b011 (8 B/beat)
`arburst_o`	out	2	`asMemArb`	Always 2'b01 (INCR)
`arvalid_o`	out	1	`asMemArb`	AR channel valid; asserted in FILL state
`arready_i`	in	1	`asMemArb`	AR channel ready from arbiter
`rid_i`	in	4	`asMemArb`	Checked against 4'h1; mismatch raises `ic_err_o`
`rdata_i`	in	64	`asMemArb`	R-channel beat data
`rresp_i`	in	2	`asMemArb`	2'b00 = OKAY; error triggers ERR state
`rlast_i`	in	1	`asMemArb`	Last beat indicator
`rvalid_i`	in	1	`asMemArb`	R-channel data valid
`rready_o`	out	1	`asMemArb`	Held high throughout FILL

Internal algorithm. The FSM is described in Table 4.41. The sequential logic is as follows:

1. **IDLE → LOOKUP:** On `ic_req_i` = 1 assert `it_ce_o` with `it_addr_o` = `ic_addr_i[9:5]`. Advance to LOOKUP.
2. **LOOKUP (tag comparison):** Read `it_rdata_i[95:0]`. For each way $w \in \{0, 1, 2, 3\}$ extract `valid[w]` = `it_rdata_i`[$23w$+22] and `tag[w][21:0]` = `it_rdata_i`[$23w$+21 : $23w$]. Compare each `tag[w]` against `ic_addr_i[31:10]`. A hit requires `valid[w]` = 1 and tag match.
   - **Hit:** assert `id_ce_o` with `id_addr_o` = {index, hit_way}; update PLRU bits in `it_wdata_o` and assert `it_we_o`; advance to RESPOND.
   - **Miss:** select victim way via PLRU; assert `ic_stall_o`; advance to FILL.
3. **FILL (AXI4 AR handshake):** Drive AR channel with aligned miss address; hold `arvalid_o` = 1 until `arready_i` = 1. For each R-channel beat

(rvalid_i = 1): write rdata_i to asICacheBlock via id_wdata_o, id_wlane_o = beat_counter[1:0], id_we_o = 1. On rlast_i: write tag word (it_we_o = 1, it_wdata_o with valid = 1 for the filled way); advance to RESPOND.

4. **RESPOND:** Read id_rdata_i[255:0]; select 32-bit instruction word using ic_addr_i[4:2]; assert ic_rdata_o and ic_rvalid_o; de-assert ic_stall_o; return to IDLE.

5. **FLUSH:** Iterate set counter from 0 to 31 (one cycle per set); write it_wdata_o = 96'h0 (it_we_o = 1) to clear all valid bits and PLRU bits. On counter = 31: pulse ic_flush_done_o; return to IDLE.

6. **ERR:** Pulse ic_err_o for one cycle; de-assert ic_stall_o; return to IDLE.

The complete Moore-FSM of asICache is shown in Fig. 5.1. Outputs depend only on the current state (state nodes contain key active signals); input conditions label the transitions. All signal names correspond directly to the asICache port list (Tables 5.28 and 5.29).

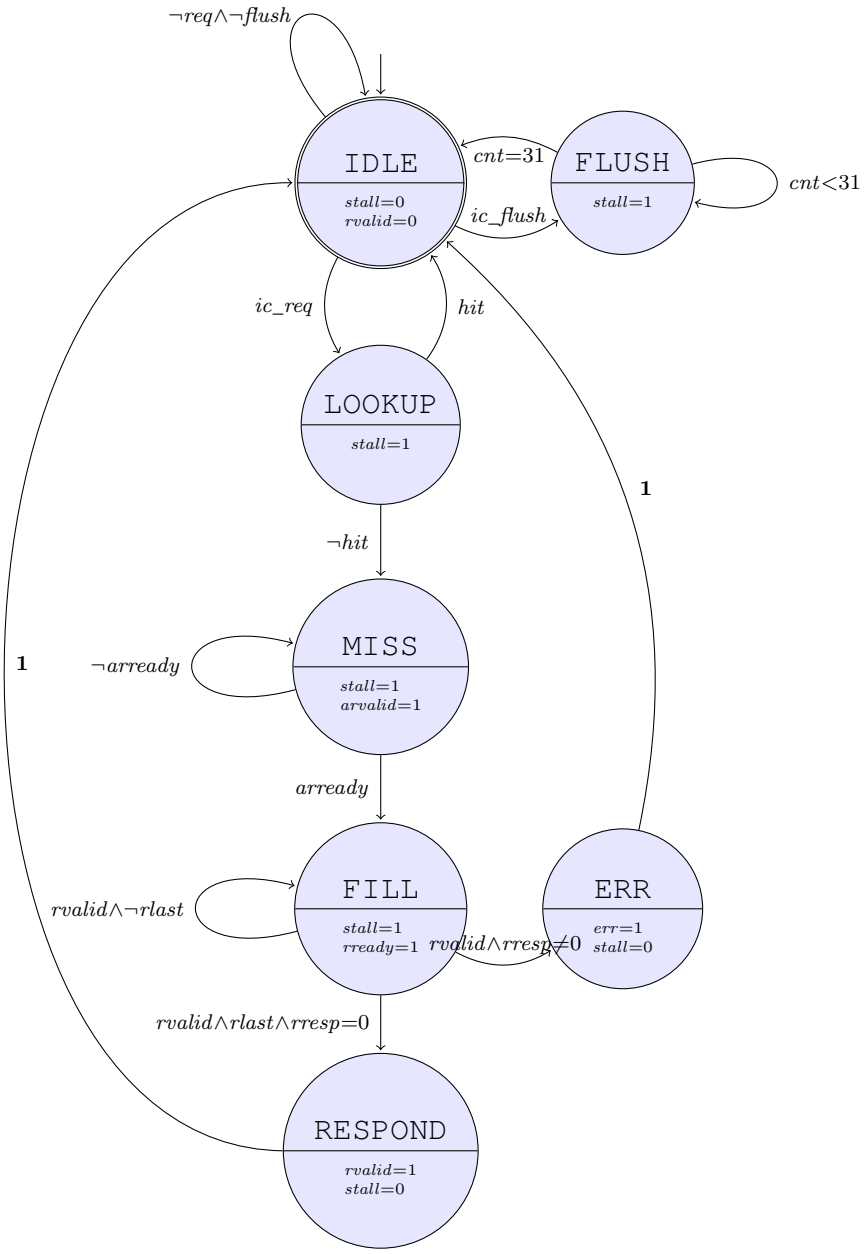


Figure 5.1: Moore-FSM: `asICache` Instruction Cache Controller (`ip_icache/src/asICache.sv`). **State nodes** show active outputs only; inactive signals are zero. *cnt* is the 5-bit set counter `flush_cnt_r` (counts 0…31 during cache-line invalidation). The self-loop label **1** on RESPOND and ERR denotes an unconditional one-cycle transition.

State-by-state description of `asICache`:

`IDLE` The cache is quiescent. `ic_stall_o` is de-asserted; the CPU may issue a new request. On each clock cycle one of three branches is taken:

- *ic_flush_i*=1: assert `ic_stall_o`, initialise `flush_cnt_r` to zero, transition to FLUSH.
- *ic_req_i*=1: latch `ic_addr_i` into `lk_tag_r`, `lk_idx_r`, `lk_instr_r`; present `req_idx_s` to the data and tag SRAM read ports (registered output will be valid in LOOKUP); assert `ic_stall_o`; transition to LOOKUP.
- Otherwise: remain in IDLE.

`LOOKUP` The SRAM registered output (`data_rdata_s`, `tag_rdata_s`) is now valid. The combinatorial valid array `valid_rdata_s` is read via `lk_idx_r`. Hit detection computes `hit_vec_s[w] = valid[w] & (tag[w] == lk_tag_r)` for all ways $w \in \{0, 1, 2, 3\}$:

- **Hit** (*hit_s*=1): extract the 32-bit instruction word from the hit way using `lk_instr_r[4:2]` as the selector; assert `ic_rdata_o` and `ic_rvalid_o`; update PLRU; de-assert `ic_stall_o`; transition to IDLE. Total latency: **2 cycles** (IDLE + LOOKUP).
- **Miss** (*hit_s*=0): select victim way via PLRU (prefer invalid way); latch `fill_way_r`; drive the AXI4 AR channel with the cache-line-aligned miss address (`lk_line_addr_r`); assert `ar_valid_r`; transition to MISS.

`MISS` The AXI4 AR channel handshake is in progress. `arvalid` is held high; the FSM waits until the memory arbiter (`asMemArb`) asserts `arready`. On acknowledgement: clear `ar_valid_r`; transition to FILL. Stall remains asserted throughout.

`FILL` AXI4 R-channel beats are received. `rready` is held high (combinatorial from state). For each beat with *rvalid*=1:

- If *rresp*≠0: assert `ic_err_o`; de-assert stall; transition to ERR.
- If *rlast*=0: write the beat into `fill_buf_r` at position `beat_r`; increment `beat_r`; remain in FILL.
- If *rlast*=1 and *rresp*=0: write the full line (data + tag + valid) to the SRAM; update PLRU; latch the requested instruction word into `rdata_r`; transition to RESPOND.

RESPOND One-cycle response state. `ic_rvalid_o` is pulsed high; `ic_stall_o` is de-asserted; the FSM returns to IDLE. This extra cycle exists because the SRAM write (in FILL) and the read-data extraction occur in the same cycle, and the CPU must sample valid data on the following edge.

ERR One-cycle error state. `ic_err_o` is pulsed high; `ic_stall_o` is de-asserted; the FSM returns to IDLE. The filled cache line is not written (valid bit stays clear).

FLUSH Cache invalidation loop. In each cycle the combinatorial write-port decode (`valid_flush_en_s`) clears all four valid bits for set index `flush_cnt_r`, and PLRU bits for that set are zeroed. After 32 cycles (all 32 sets), `ic_flush_done_o` is pulsed and the FSM returns to IDLE with stall de-asserted.

`asDCacheCtrl` – Data Cache Controller

Purpose. Implements the D-Cache FSM (Table 4.43). Serves CPU load/store requests via `as_dcache_if`; routes Flash-region loads through the `asDCacheBlock` SRAM and the AXI4 miss path; routes Scratchpad-region loads and stores directly to `asSpBlock` (bypass path). No write path to Flash exists.

Port list of the D-cache controller:

Table 5.30: `asDCacheCtrl` Port List (1)

Signal	Dir	Width	Connected to	Description
`clk_i`	in	1	`asMemTop`	System clock
`rst_i`	in	1	`asMemTop`	Asynchronous reset
CPU Data Bus (`as_dcache_if.cache` modport)				
`dc_addr_i`	in	64	CPU MEM stage	Load/store address
`dc_req_i`	in	1	CPU MEM stage	Request valid
`dc_wr_i`	in	1	CPU MEM stage	1 = store, 0 = load
`dc_size_i`	in	3	CPU MEM stage	Transfer size
`dc_wdata_i`	in	64	CPU MEM stage	Store data
`dc_wstrb_i`	in	8	CPU MEM stage	Byte enables
`dc_flush_i`	in	1	CPU MEM stage	Invalidation pulse
`dc_stall_o`	out	1	CPU MEM stage	Stall signal
`dc_flush_done_o`	out	1	CPU MEM stage	Flush complete pulse
`dc_err_o`	out	1	CPU MEM stage	AXI4 error pulse
`dc_rdata_o`	out	64	CPU MEM stage	Load data
`dc_rvalid_o`	out	1	CPU MEM stage	Load data valid
D-Tag SRAM port (to `asDCacheBlock`)				
`dt_addr_o`	out	5	`asDCacheBlock`	Set index
`dt_wdata_o`	out	96	`asDCacheBlock`	Tag write data
`dt_we_o`	out	1	`asDCacheBlock`	Tag write enable
`dt_ce_o`	out	1	`asDCacheBlock`	Tag chip enable
`dt_rdata_i`	in	96	`asDCacheBlock`	Tag read data
D-Data SRAM port (to `asDCacheBlock`)				
`dd_addr_o`	out	7	`asDCacheBlock`	{index[4:0], way[1:0]}
`dd_wdata_o`	out	64	`asDCacheBlock`	AXI4 R-channel beat
`dd_wlane_o`	out	2	`asDCacheBlock`	Beat index (0–3)
`dd_we_o`	out	1	`asDCacheBlock`	Data write enable
`dd_ce_o`	out	1	`asDCacheBlock`	Data chip enable
`dd_rdata_i`	in	256	`asDCacheBlock`	Full cache line

Table 5.31: `asDCacheCtrl` Port List (2)

Signal	Dir	Width	Connected to	Description
Scratchpad SRAM port (to `asSpBlock`)				
`sp_addr_o`	out	10	`asSpBlock`	Word address `dc_addr_i[12:3]`
`sp_wdata_o`	out	64	`asSpBlock`	Store data from `dc_wdata_i`
`sp_wstrb_o`	out	8	`asSpBlock`	Byte enables from `dc_wstrb_i`
`sp_we_o`	out	1	`asSpBlock`	Write enable: high on store to Scratchpad region
`sp_ce_o`	out	1	`asSpBlock`	Chip enable: high on any Scratchpad access
`sp_rdata_i`	in	64	`asSpBlock`	Load data from Scratchpad
AXI4 Master Port – AR and R channels only (to `asMemArb`)				
`arid_o`	out	4	`asMemArb`	Always 4'h2 (D-Cache ID)
`araddr_o`	out	32	`asMemArb`	Cache-line-aligned miss address
`arlen_o`	out	8	`asMemArb`	Always 8'h03
`arsize_o`	out	3	`asMemArb`	Always 3'b011
`arburst_o`	out	2	`asMemArb`	Always 2'b01 (INCR)
`arvalid_o`	out	1	`asMemArb`	AR valid; asserted in FILL
`arready_i`	in	1	`asMemArb`	AR ready
`rid_i`	in	4	`asMemArb`	Checked against 4'h2
`rdata_i`	in	64	`asMemArb`	R-channel beat
`rresp_i`	in	2	`asMemArb`	Response; OKAY = 2'b00
`rlast_i`	in	1	`asMemArb`	Last beat
`rvalid_i`	in	1	`asMemArb`	R-channel valid
`rready_o`	out	1	`asMemArb`	Held high throughout FILL

Internal algorithm. The FSM is described in Table 4.43. Address decoding in the DECODE state is the critical dispatch:

1. **DECODE – address region check:** `dc_addr_i[31:13]` is compared against `SP_BASE[31:13]`.

   - Match ⇒ Scratchpad region: advance to BYPASS.
   - No match ⇒ Flash region: assert `dt_ce_o` and advance to LOOKUP.

2. **BYPASS (Scratchpad access):** For a **store**: drive `sp_addr_o`, `sp_wdata_o`, `sp_wstrb_o`, `sp_we_o` = 1, `sp_ce_o` = 1. De-assert `dc_stall_o` after one cycle; return to IDLE. For a **load**: drive `sp_addr_o`, `sp_we_o` = 0, `sp_ce_o` = 1. Latch `sp_rdata_i` into `dc_rdata_o` in the following cycle; assert `dc_rvalid_o`; return to IDLE.

3. **LOOKUP – Flash region, tag comparison:** Same tag-compare logic as `asICacheCtrl`.

   - **Hit:** read `dd_rdata_i[255:0]`; extract 64-bit double word at `dc_addr_i[4:3]` × 64 bit; sign/zero-extend according to `dc_size_i`; assert `dc_rdata_o`, `dc_rvalid_o`; update PLRU; return to IDLE.
   - **Miss:** assert `dc_stall_o`; advance to FILL.

   Note: Stores to the Flash region are not permitted. If `dc_wr_i` = 1 and address is in Flash region the controller asserts `dc_err_o` and returns to IDLE without any SRAM or AXI4 access.

4. **FILL:** Identical to `asICacheCtrl` FILL, using D-Cache AXI4 ID 4'h2 and `asDCacheBlock` SRAM ports.

5. **RESPOND, FLUSH, ERR:** Identical in structure to `asICacheCtrl`; see Section 5.3.4.

D-Cache FSM overview. The `asDCache` module implements a write-back, write-allocate 4-way set-associative cache with a full 11-state Moore-FSM (Fig. 5.2). The three columns in the figure reflect the three independent activity paths:

- **Centre column (normal cache path):** IDLE → LOOKUP → MISS → FILL → FILL_RESP → IDLE.
- **Left column (dirty-eviction path):** EVICT_AW → EVICT_W → EVICT_B; entered from LOOKUP (miss on dirty victim) or from the flush path.
- **Right column (flush path):** FLUSH → FLUSH_EVICT_PREP → EVICT_AW; entered from IDLE on `dc_flush_i`.

The ERR state (reached from FILL and EVICT_B on AXI4 error response) returns unconditionally to IDLE. Self-loops that keep a state while an AXI4 handshake is pending (MISS, EVICT_AW, EVICT_B) are shown for completeness. $flush_mode_r$ is a one-bit register that distinguishes a normal dirty-victim eviction from a flush-triggered write-back.

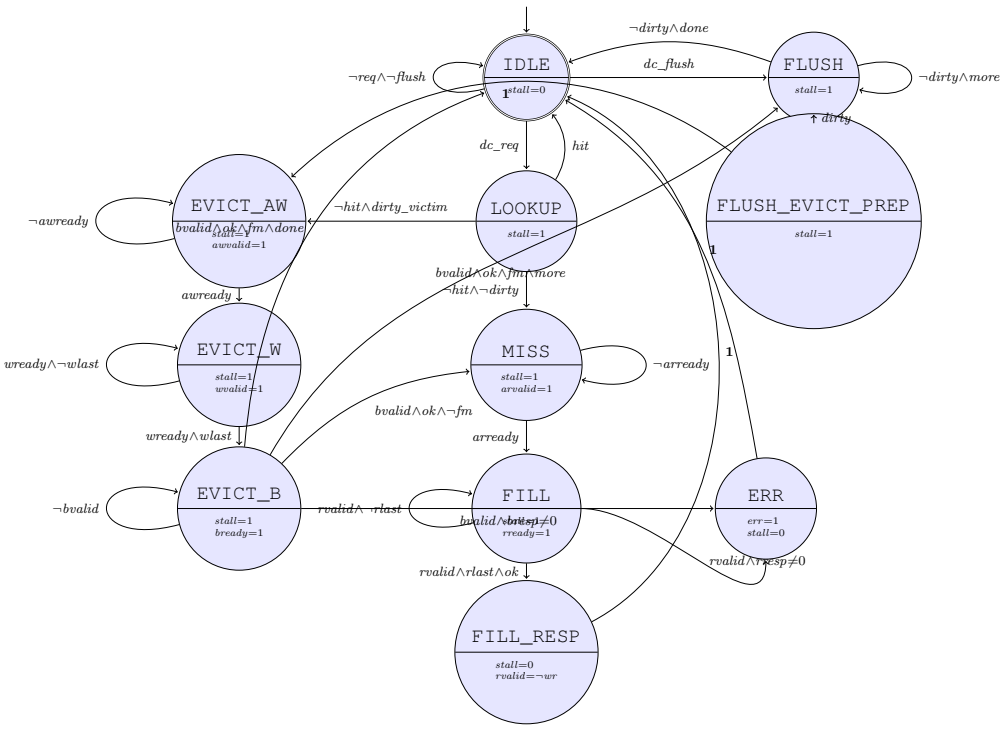


Figure 5.2: Moore-FSM: `asDCache` Data Cache Controller (`ip_dcache/src/asDCache.sv`). **Abbreviations on edges:** *dirty_victim*: victim way is valid and dirty; *fm* (`flush_mode_r`): eviction was triggered by a flush, not a normal miss; *more*: flush iterator has not yet scanned all sets/ways; *done*: flush complete (all sets/ways iterated); *ok*: AXI4 response is OKAY (*bresp*=0 or *rresp*=0); *wr*: the CPU request was a store (`lk_wr_r`). **Outputs** *rvalid*=¬*wr* in FILL_RESP: only a load returns data to the CPU. Self-loop label **1** denotes an unconditional single-cycle transition.

State-by-state description of `asDCache`:

`IDLE` The cache is ready for a new CPU request. `dc_stall_o` is de-asserted. On `dc_flush_i`: initialise `flush_idx_r` and `flush_way_r` to zero, assert stall, go to FLUSH. On `dc_req_i`: latch the full request (address, size, wr, wdata, wstrb) into the `lk_*_r` register bank; present `req_idx_s` to the data and tag SRAM read ports; assert stall; go to LOOKUP.

`LOOKUP` Registered SRAM output (`data_rdata_s`, `tag_rdata_s`) is valid from the previous-cycle read initiated in IDLE. Combinatorial valid/dirty arrays (`vd_valid_s`, `vd_dirty_s`) are read via `lk_idx_r`.

- **Hit:** update PLRU. Read hit – extract and sign/zero-extend the double word selected by `lk_dw_sel_r`; assert `dc_rdata_o`, `dc_rvalid_o`; de-assert stall; return to IDLE. Write hit – merge write data into the cache line via `merge_write()`; set dirty bit through the combinatorial SRAM write-port; de-assert stall; return to IDLE. In both cases total latency is **2 cycles**.
- **Miss, clean victim:** no write-back needed; drive AXI4 AR with the line-aligned miss address; assert `ar_valid_r`; go to MISS.
- **Miss, dirty victim:** write-back required; latch the eviction line (`evict_line_r`) from the registered SRAM output; drive AXI4 AW channel; assert `aw_valid_r`; clear `flush_mode_r`; go to EVICT_AW.

EVICT_AW AXI4 AW-channel handshake: `awvalid` held high until `awready` is asserted by the interconnect. On acknowledgement: clear `aw_valid_r`; go to EVICT_W.

EVICT_W AXI4 W-channel data phase: `wvalid` is driven high (combinatorial from state). The eviction line (`evict_line_r`, 256 bit) is streamed in four 64-bit beats; `beat_r` counts 0...3. On each `wready`: increment `beat_r` until $beat_r{=}3$ (`wlast` asserted); go to EVICT_B.

EVICT_B AXI4 B-channel response: `bready` held high; FSM waits for `bvalid`. On `bvalid`:

- $bresp{\neq}0$: assert `dc_err_o`; de-assert stall; go to ERR.
- $bresp{=}0$ and $\neg flush_mode_r$ (normal miss eviction): proceed to fill the line; drive AR channel; go to MISS.
- $bresp{=}0$ and $flush_mode_r$ and more flush work remains (`flush_way_r` or `flush_idx_r` not exhausted): invalidate the evicted line via the combinatorial write-port; advance the flush pointer; go back to FLUSH.
- $bresp{=}0$ and $flush_mode_r$ and all sets/ways exhausted: assert `dc_flush_done_o`; de-assert stall; go to IDLE.

MISS AXI4 AR-channel handshake for a cache-line fill. `arvalid` held high; waits for `arready`. On acknowledgement: clear `ar_valid_r`; go to FILL.

FILL AXI4 R-channel data phase: `rready` held high. Four 64-bit beats are assembled into `fill_buf_r` / `fill_line_s`.

- $rresp{\neq}0$: assert `dc_err_o`; de-assert stall; go to ERR.

- $rvalid \wedge \neg rlast$: store beat into `fill_buf_r`; increment `beat_r`; remain in FILL.
- $rvalid \wedge rlast \wedge ok$: on the last beat the combinatorial write-port decode writes data, tag, and valid bit to the SRAMs; if the original request was a write-miss (`lk_wr_r`), the write data is merged into the fill line before the SRAM write; update PLRU; for a read, latch the extracted data word into `rdata_r`; go to FILL_RESP.

FILL_RESP One-cycle response state after a successful fill. `dc_stall_o` is de-asserted. For a load: assert `dc_rvalid_o` with the latched data word. For a write-miss: no rdata output; the merged line has already been written to the SRAM in FILL. Return to IDLE.

ERR One-cycle error state. `dc_err_o` pulsed; `dc_stall_o` de-asserted. The affected cache line is not written; valid bit is not set. Return to IDLE.

FLUSH The flush iterator examines one (set, way) pair per cycle. The combinatorial valid/dirty read address is driven by `flush_idx_r` (via `vd_rd_addr_s` mux).

- **Dirty line:** present `flush_idx_r` to the data and tag SRAM read port (one cycle of registered latency is needed); go to FLUSH_EVICT_PREP.
- **Clean or invalid line:** invalidate the line via the combinatorial write-port; advance `flush_way_r` (then `flush_idx_r`); remain in FLUSH until all $32 \times 4 = 128$ entries are scanned, then assert `dc_flush_done_o` and return to IDLE.

FLUSH_EVICT_PREP One-cycle pipeline bubble that absorbs the SRAM read latency. In the previous cycle (FLUSH) the SRAM read address was already `flush_idx_r`; the registered output is now valid. Latch `evict_line_r` from `data_rdata_s[flush_way_r]`; compute the eviction address from `tag_rdata_s[flush_way_r]`; assert `aw_valid_r`; set `flush_mode_r` to distinguish this write-back from a normal dirty-victim eviction; go to EVICT_AW.

Scratchpad Controller (`asScratch`) – 2-state FSM. The `asScratch` module connects directly to the D-Cache CPU interface (`as_dcache_if`) and provides a single-port 64-bit wide SRAM (`as_sram_sp64`) as a byte-addressable scratchpad. Its controller is a minimal two-state machine whose state is captured by the single registered bit `busy_r` (Fig. 5.3).

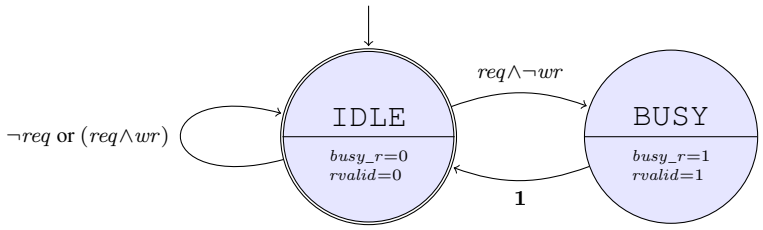


Figure 5.3: Moore-FSM: `asScratch` Scratchpad Controller (`ip_scratch/src/asScratch.sv`). **Note:** The `dc_stall_o` output is a Mealy signal ($stall = req \wedge \neg wr \wedge \neg busy_r$) and therefore depends on both state and inputs; it is not shown in the state node. Stores ($req \wedge wr$) are single-cycle and never cause a stall; the SRAM write completes in the same cycle. Only loads ($req \wedge \neg wr$) require one extra cycle for the registered SRAM read output.

`IDLE` ($busy_r{=}0$) The scratchpad is ready.

- **No request** ($\neg req$): remain idle; SRAM chip-enable (`cen_i`) is de-asserted.
- **Store** ($req{\wedge}wr$): assert `cen_i`, `we_i`, and the byte-write enables `wbe_i` from `dc_wstrb_i`; the SRAM completes the write in the current cycle; remain in IDLE with no stall. The `dc_stall_o` output is low for stores.
- **Load** ($req{\wedge}\neg wr$): latch `dc_size_i` and `dc_addr[2:0]` into `size_r` and `boff_r`; assert `cen_i` with `we_i` = `0`; assert `dc_stall_o` (Mealy output); set `busy_r` = `1`; go to BUSY.

`BUSY` ($busy_r{=}1$) The SRAM registered read output `sram_rdata_s` is now valid. The combinatorial output logic sign/zero-extends the 64-bit SRAM word according to `size_r` (byte, halfword, word, double) and byte offset `boff_r`, following the RISC-V load semantics (`LB`/`LH`/`LW`/`LD`/ `LBU`/`LHU`/`LWU`). `dc_rvalid_o` is asserted; `dc_stall_o` is de-asserted (Mealy: *req* would be 0 because the CPU sees stall and has not issued a new request). Unconditional transition back to IDLE.

`asMemArb` – Memory Arbiter

Purpose. Fixed-priority combinatorial AXI4 multiplexer. Serialises the AR/R (and potentially AW/W/B) channels of the I-Cache and D-Cache controllers onto the single AXI4 port of the QSPI controller. D-Cache has priority over I-Cache (Table 4.52).

Port list of the memory arbiter:

Table 5.32: `asMemArb` Port List (1)

Signal	Dir	Width	Connected to	Description
`clk_i`	in	1	`asMemTop`	System clock (used only for grant register)
`rst_i`	in	1	`asMemTop`	Asynchronous reset; clears grant register
I-Cache AXI4 Slave Port (AR + R channels)				
`ic_arvalid_i`	in	1	`asICacheCtrl`	I-Cache AR valid
`ic_araddr_i`	in	32	`asICacheCtrl`	I-Cache AR address
`ic_arid_i`	in	4	`asICacheCtrl`	Always 4'h1
`ic_arlen_i`	in	8	`asICacheCtrl`	Always 8'h03
`ic_arsize_i`	in	3	`asICacheCtrl`	Always 3'b011
`ic_arburst_i`	in	2	`asICacheCtrl`	Always 2'b01
`ic_arready_o`	out	1	`asICacheCtrl`	AR ready back to I-Cache
`ic_rvalid_o`	out	1	`asICacheCtrl`	R-channel valid routed to I-Cache
`ic_rdata_o`	out	64	`asICacheCtrl`	R-channel data routed to I-Cache
`ic_rresp_o`	out	2	`asICacheCtrl`	R-channel response to I-Cache
`ic_rlast_o`	out	1	`asICacheCtrl`	R-channel last beat to I-Cache
`ic_rid_o`	out	4	`asICacheCtrl`	R-channel ID to I-Cache
`ic_rready_i`	in	1	`asICacheCtrl`	R-channel ready from I-Cache
D-Cache AXI4 Slave Port (AR + R channels; identical structure)				
`dc_ar*_i` / `dc_r*_o` / `dc_rready_i`	in/out	–	`asDCacheCtrl`	Same signals as I-Cache port; prefix `dc_`

Table 5.33: `asMemArb` Port List (2)

Signal	Dir	Width	Connected to	Description
QSPI Controller AXI4 Master Port				
`qspi_arvalid_o`	out	1	`asQspiTop`	AR valid to QSPI
`qspi_araddr_o`	out	32	`asQspiTop`	AR address to QSPI
`qspi_arid_o`	out	4	`asQspiTop`	AR ID forwarded from winning master
`qspi_arlen_o`	out	8	`asQspiTop`	Always 8'h03
`qspi_arsize_o`	out	3	`asQspiTop`	Always 3'b011
`qspi_arburst_o`	out	2	`asQspiTop`	Always 2'b01
`qspi_arready_i`	in	1	`asQspiTop`	AR ready from QSPI
`qspi_rvalid_i`	in	1	`asQspiTop`	R-channel valid from QSPI
`qspi_rdata_i`	in	64	`asQspiTop`	R-channel data from QSPI
`qspi_rresp_i`	in	2	`asQspiTop`	R-channel response from QSPI
`qspi_rlast_i`	in	1	`asQspiTop`	R-channel last beat from QSPI
`qspi_rid_i`	in	4	`asQspiTop`	R-channel ID from QSPI
`qspi_rready_o`	out	1	`asQspiTop`	R-channel ready to QSPI (OR of both master readys)

Internal algorithm. The arbiter maintains a one-bit **grant register** (`grant_r`: 0 = I-Cache, 1 = D-Cache) and a **busy flag** (`busy_r`) that locks the grant for the duration of an active burst.

1. **IDLE** (`busy_r` = 0):
   - If `dc_arvalid_i` = 1: set `grant_r` = 1, `busy_r` = 1; forward D-Cache AR signals to QSPI master port.
   - Else if `ic_arvalid_i` = 1: set `grant_r` = 0, `busy_r` = 1; forward I-Cache AR signals to QSPI master port.
2. **BUSY** (`busy_r` = 1): All QSPI AR/R signals are forwarded to the granted master only. `arready_o` of the non-granted master is held low (bus not

available). `busy_r` is cleared when `qspi_rlast_i` = 1 and `qspi_rvalid_i` = 1 (last beat of the burst received).

3. **R-channel routing:** `qspi_rid_i` identifies the receiving master (4'h1 = I-Cache, 4'h2 = D-Cache); R-channel signals are forwarded to the matching master regardless of `grant_r`, ensuring correct response routing even during back-to-back grants.

The arbiter is purely combinatorial on the data path; only `grant_r` and `busy_r` are registered.

`asSpramModel` – Simulation SRAM Behavioural Model

Purpose. Provides a synthesisable-free behavioural model of the X-Fab XH035 SPRAM macro for use in RTL simulation (Vivado `xsim`, ModelSim, VCS). It is **not** used in synthesis; the three SRAM wrapper modules select between the real SPRAM macro and `asSpramModel` via a conditional compile guard:

```
`ifdef SIMULATION
  asSpramModel #(.WIDTH(W), .DEPTH(D)) u_sram (...);
`else
  SPRAM_XH035   #(.WIDTH(W), .DEPTH(D)) u_sram (...);
`endif
```

Table 5.34: `asSpramModel` Port List (matches XH035 SPRAM interface)

Signal	Dir	Width	Description
`CLK`	in	1	Rising-edge clock
`CEN`	in	1	Chip enable, active-low; high = standby (no access)
`WEN`	in	1	Write enable, active-low; high = read, low = write
`BWEN`	in	WIDTH	Bit write enable, active-low; 0 = write bit, 1 = mask
`A`	in	$\lceil \log_2(\text{DEPTH}) \rceil$	Address
`D`	in	WIDTH	Write data
`Q`	out	WIDTH	Read data; updated one clock after access

Port list.

Internal algorithm.

```
parameter int WIDTH = 96;
parameter int DEPTH = 32;
logic [WIDTH-1:0] mem [0:DEPTH-1];

always_ff @(posedge CLK) begin
  if (!CEN) begin
    if (!WEN)                          // write
      for (int b = 0; b < WIDTH; b++)
        if (!BWEN[b]) mem[A][b] <= D[b];
    Q <= mem[A];                       // read (write-first)
  end
end
```

The model uses write-first semantics: on a write cycle `Q` returns the newly written data, matching the XH035 SPRAM read-during-write behaviour. The `initial` block zero-initialises `mem` to model power-on state.

5.3.5 Service Requests of the MemSys

No

5.3.6 The MemSys Peripheral Kernel

Block Architecture:

5.3.7 Registers

No registers.

Chapter 6

Peripherals

6.1 GPIO

6.1.1 System Integration of GPIO0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.1: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.2: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)
2025-11-19	Andreas Siggelkow	Changed the BPI; removed the registers from the kernel and added them to GPIOTop. Added an interrupt. Removed the address output. changed the GPIO output to inout.

Functional Configuration of GPIO0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a GPOI peripheral are described in its functional description in Section 6.1.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for GPIO0 in Table 6.3.

Table 6.3: GPIO0 Feature Set

	Generic GPIO Feature Set	Details
yes	Seperate Kernel Clock	`clk_core_s` (CGU ÷ 80); 1.25 MHz recommended for `RV_NoPipeline`, see Clock Concept Section 4.5.2
no	FIFO Data Buffering	not yet

HW Interfaces

The following table 6.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailled list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.4: HW Interfaces of Module GPIO0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	gpio_irq_o	gpio_irq_s
DMA	None	-
External Signals	gpio_io cs_o	gpio_io asGpioCs_s

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.5: Register List of Module GPIO0

Register Name(s) in Chip	Name used in module register description	Comment Comment
GPIO0_ID	id_reg_s	ID of GPIO0 peripheral
GPIO0_DIR	dir_reg_s	Direction of the GPIO: • dir_reg_s[i] = 0 -> output, • dir_reg_s[i] = 1 -> input
GPIO0_DATA	data_reg_s	Data driven to the GPIO or received from the GPIO.
GPIO0_IRQSS	irqss_reg_s	GPIO Interrupt Request Source Status Register
GPIO0_IRQSC	irqsc_reg_s	GPIO Interrupt Request Source Clear Register
GPIO0_IRQSM	irqsm_reg_s	GPIO Interrupt Request Source Mask Register
GPIO0_ISR	isr_reg_s	GPIO Interrupt Set Register
GPIO0_RIS	ris_reg_s	GPIO Raw Interrupt Status Register
GPIO0_IMSC	imsc_reg_s	GPIO Interrupt Mask Control Register
GPIO0_MIS	mis_reg_s	GPIO Masked Interrupt Status Register

Related Sections of this Spec

- GPIO functional description starting with Section 6.1.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”
  - Peripheral ID Registers Section “Architecture Concepts”
  - Peripheral Clocking Concept Section “Architecture Concepts”
  - Clock Control Registers Section “Architecture Concepts”

- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.1.2 History of GPIO

Table 6.6: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
6.1	6.1 / 2025-02-13	First draft (AS)
6.1	6.1 / 2025-11-19	Changed the BPI; removed the registers from the kernel and added them to GPIOTop. Added an interrupt. Removed the address output. changed the GPIO output to inout. (AS)

6.1.3 Functional Overview

The GPIO peripheral allows to drive 8 bit data from the Wishbone data bus to GPIO pins gpio_io (7:0) (input and output). The direction must be programmed

with the **GPIO0_DIR** register. A chip select (cs) is used to activate the GPIO output (dummy citation [4]).

General Features:

- Chip select
- FIFO data buffering (future)
- I/O
- Interrupt

Synchronous Mode:

- 8 data bits

Asynchronous Mode:

- -

6.1.4 Structural Overview

The GPIO is a Wishbone slave (asSlaveBPI) peripheral. This peripheral architecture aims for high bus performance by dividing peripherals into two clock domains. Figure 6.1 shows a simplified block diagram of the GPIO asSlaveBPI peripheral.

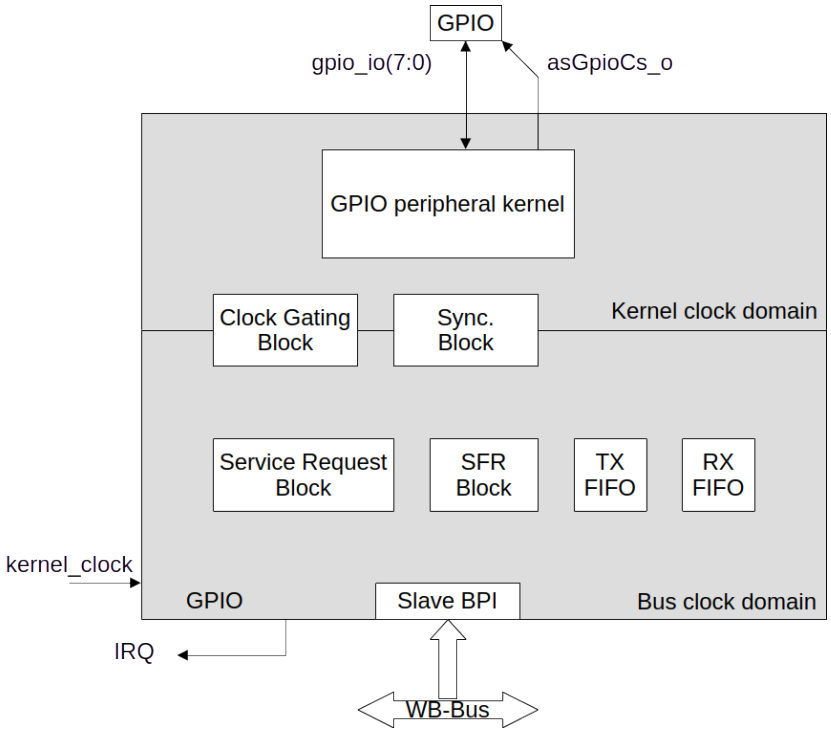


Figure 6.1: GPIO asSlaveBPI Peripheral

GPIO Peripheral Kernel: The GPIO kernel provides the specific functionality of the GPIO interface, that is described in Section 6.1.6. The peripheral kernel has its own kernel clock domain.

Bus Peripheral Interface BPI: The BPI is responsible for the interconnection of the GPIO registers with an on-chip bus. Also the peripheral clock gating is controlled by the BPI.

asSlaveBPI Block: The asSlaveBPI block lies within the bus clock domain and contains the following subblocks:

Special Function Register Block The Special Function Register Block contains all FIFO specific registers and all special function registers (SFR) of the peripheral kernel that are accessible by the SW via the system bus.

Service Request Block SRB The SRB is used to prepare the interrupt and data transfer requests for the Interrupt Control Unit (ICU) and the DMA Controller (DMAC), respectively (no DMA planned!).

TX FIFO The TX FIFO is used to buffer the transmission data from the bus, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the system bus.

RX FIFO The RX FIFO is used to buffer the received data from the kernel, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the bus system.

Clock Gating Block: The Clock Gating Block is used to derive the necessary clocks for the several blocks within the peripheral from the clocks provided by the system.

Synchronization Block: The Synchronization Block is used to synchronize the signals between the bus clock and kernel clock domains.

I/O of the GPIO Kernel

Table 6.7 shows the input and outputs of the GPIO kernel.

Table 6.7: GPIO Kernel I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
direction_i(7:0)	in	8	From GPIO-BPI (**GPIO0_DIR** register). Defines the direction of each GPIO. If the bit is 0, the GPIO is an output, if the bit is 1, the GPIO is an input.
addr_i(3:0)	in	4	From GPIO-BPI. Address for storing cs_s.
data_i(7:0)	in	8	From GPIO-BPI (**GPIO0_DATA** register). GPIO data in output direction. The elements of the register will be forwarded to a tristate buffer which is active when direction is output and 'Z' when direction is input.
data_o(7:0)	out	8	From pad to GPIO-BPI (**GPIO0_DATA** register). GPIO data in input direction. If the direction is input, the signals from the pads will go through a tristate buffer and will be forwarded to the register. Else, a '0' will be driven.
irq_o(7:0)	out	8	From kernel to GPIO-BPI. A change of an input will be detected and send to the IRQ register. Every possible GPIO input will be routed to an edge detector (both, rising and falling). Those detected edges will be forwarded to the IRQ structure in the BPI.
en_i	in	1	From GPIO-BPI. Enables a write to a GPIO block. It stores the chip select cs_o.
gpio_io	bidi	8	To inout pins. GPIO inout.
cs_o	out	1	To output pins. Chip select for outside GPIO block. Stored in an internal register (so, synchronized to the data).

I/O of the GPIO BPI

Table 6.8 shows the input and outputs of the GPIO BPI.

Table 6.8: GPIO BPI I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
addr_o(3:0)	out	4	To GPIO kernel. Address for external GPIO devices. addr_o = addr_i.
dat_from_core_i(7:0)	in	8	From GPIO kernel.
dat_to_core_o(7:0)	out	8	To GPIO kernel. GPIO outputs for one block. dat_to_core_o = dat_i
wr_o	out	1	To GPIO kernel. Enable for GPIO data and address, and chip select for outside GPIO blocks. wr_o = we_i AND stb_i.
addr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
dat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
dat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
we_i	in	1	From WB-Bus. Peripherals write enable.
sel_i(7:0)	in	8	From WB-Bus. Peripherals byte select. Not used here.
stb_i	in	1	From WB-Bus. Valid bus cycle. Combined (AND) with we_i for wr_o.
ack_o	out	1	To WB-Bus. Error free bus transaction. Here, ack_o = stb_i.
cyc_i	in	1	From WB-Bus. High for complete bus cycle. Not used here.

Internals of the GPIO BPI

Table 6.9 shows the internal signals and processes of the GPIO BPI.

Table 6.9: GPIO BPI Internals

Object	Type	Width	Explanation
any	logic	x	See BPI spec.

I/O of the GPIO Top

Table 6.10 shows the input and outputs of the GPIO Top.

Table 6.10: GPIO Top I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
wbdAddr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
wbdDat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
wbdDat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
wbdWe_i	in	1	From WB-Bus. Peripherals write enable.
wbdSel_i(7:0)	in	8	From WB-Bus. Peripherals byte select.
wbdStb_i	in	1	From WB-Bus. Valid bus cycle.
wbdAck_o	out	1	To WB-Bus. Error free bus transaction.
wbdCyc_i	in	1	From WB-Bus. High for complete bus cycle.
gpio_irq_o	out	1	GPIO interrupt.
gpio_o(7:0)	out	8	To chip pins. GPIO outputs for one block.
cs_o	out	1	To chip pins. Chip select for outside GPIO blocks.

Table 6.11 shows the internals of the GPIO Top.

Table 6.11: GPIO Top Signals

Signal	Width	Explanation
irq_comb_s	1	An ORed signal of all interrupts comming from the GPIO kernel.
irqsc_comb_s	1	An ORed signal of all interrupt clear bits from register IRQSC.
irqss_reg_s	64	GPIO Interrupt Request Source Status Register **GPIO0_IRQSS**. If irq_comb_s = 1, the interrupts from the kernel will be written. Else, if irqsc_comb_s = 1, irqss_reg_s will be cleared and irqsc_reg_s will be cleared.
irqsc_reg_s	64	GPIO Interrupt Request Source Clear Register **GPIO0_IRQSC**. One interrupt request source clear bit is provided for each interrupt request source: 0 - No change, 1 - Clear Interrupt source.
irqsm_reg_s	64	GPIO Interrupt Request Source Mask Register **GPIO0_IRQSM**. One interrupt request source mask bit is provided for each interrupt request source: 0 - Interrupt Request Source Disabled, 1 - Interrupt Request Source Enabled.
irqsm_comb_s	1	An ORed signal of all masked single interrupts comming from the GPIO kernel.
ris_reg_s	64	GPIO Raw Interrupt Status Register **GPIO0_RIS**. One interrupt status bit is provided for each service request: 0 - No Interrupt, 1 - Interrupt Pending.
isr_reg_s	64	GPIO Interrupt Set Register **GPIO0_ISR**. One interrupt set bit is provided for each service request. The interrupt status bits in the **GPIO0_RIS** and the MIS (if IMSC = 1) registers are set by writing a '1' to the desired positions. 0 - No change, 1 - Set Interrupt. Reading the register returns 0.
imsc_reg_s	64	GPIO Interrupt Mask Control Register **GPIO0_IMSC**. One interrupt mask bit is provided for each service request: 0 - Interrupt Disabled, 1 - Interrupt Enabled.
mis_reg_s	64	GPIO Masked Interrupt Status Register **GPIO0_MIS**. One interrupt status bit is provided for each service request: 0 - No Interrupt, 1 - Interrupt Pending.
id_reg_s	64	GPIO ID Register **GPIO0_ID**.
dir_reg_s	64	GPIO Direction Register **GPIO0_DIR**.
data_reg_s	64	GPIO Data Register **GPIO0_DATA**. If the direction is input, the values from the pad will be written to the register. If the direction is output, the values from the bus will be written to the register.

6.1.5 Service Requests of the GPIO

Table 6.12 shows the service requests of the GPIO.

Table 6.12: GPIO Service Requests

Service Request	IRQ	Explanation
gpio_io (0)	GPIO_INT	The input on gpio_io (0) changed.
gpio_io (1)	GPIO_INT	The input on gpio_io (1) changed.
gpio_io (2)	GPIO_INT	The input on gpio_io (2) changed.
gpio_io (3)	GPIO_INT	The input on gpio_io (3) changed.
gpio_io (4)	GPIO_INT	The input on gpio_io (4) changed.
gpio_io (5)	GPIO_INT	The input on gpio_io (5) changed.
gpio_io (6)	GPIO_INT	The input on gpio_io (6) changed.
gpio_io (7)	GPIO_INT	The input on gpio_io (7) changed.

6.1.6 The GPIO Peripheral Kernel

As common for asSlaveBPI peripherals there are also two main states of the GPIO's kernel.

Before it can actually start operation, it has to be configured. This configuration state is left by setting bit RUN in register RUN_CTRL. Thereby the configuration registers are locked for write accesses, the functional blocks of peripheral kernel and asSlaveBPI block are reset (e.g. the FIFOs are flushed) and working state is entered. The different functional blocks of the GPIO peripheral kernel start operating as described in the following.

Figure 6.2 shows the principal functionality of the GPIO kernel. The shown registers are part of the SFR block of the peripheral. When there is a '1' written to the direction register, the pad acts as an input and the captured value will be written to the data register. When there is a '0' written to the direction register, the pad acts as an output and the value from the data register will be written to the pad. The data register is rw from the bus side.

In case of an input setting, a signal change will be detected and written to the IRQ register inside the SRB.

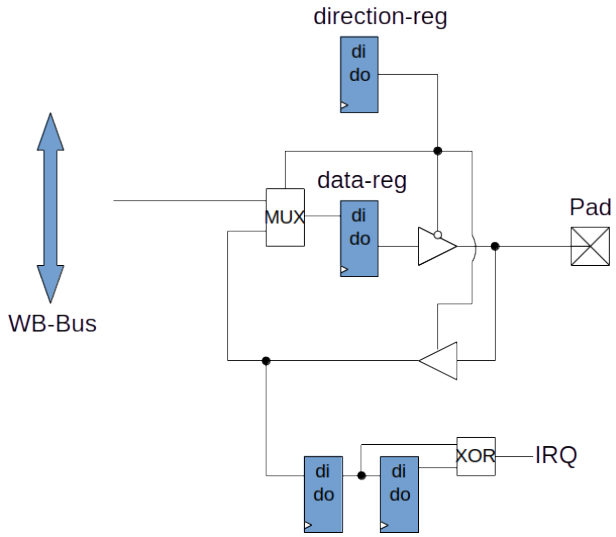


Figure 6.2: GPIO Kernel

6.1.7 Registers

In the following the registers of a GPIO peripheral are listed and described. Some registers' layout or availability depends on the generic feature set implemented for that GPIO's instantiation. Such registers and concerned bit fields are denoted by the use of italic style.

GPIO Register Overview

Table 6.13: GPIO Register List

Register Group	Register Name	Register Symbol
System Registers	(Clock Control Register) Identification Register	(**CLC**) **ID**
Special Function Registers	(Run Control Register) Direction Register Data Register	(**RUN_CTRL**) **GPIO0_DIR** **GPIO0_DATA**
Interrupt Registers	Interrupt Mask Control Register Raw Interrupt Status Register Masked Interrupt Status Register Interrupt Set Register Interrupt Source Status Register Interrupt Source Mask Register Interrupt Source Clear Register	**GPIO0_IMSC** **GPIO0_RIS** **GPIO0_MIS** **GPIO0_ISR** **GPIO0_IRQSS** **GPIO0_IRQSM** **GPIO0_IRQSC**

Read-Write Features

- w: writable bit, by SW
- r: readable bit, by SW
- h: bit will be set by HW only

System Registers

GPIO_ID -: Identification register of this peripheral. A write will have no effect, a read will return the fixed ID. The ID will be defined on chip level.

ID register (in BPI) [Reset value: $000100000000C001_H$]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
MOD_NUMBER															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	1	0	0	0	0	0	0	REV_NUMBER							
r	r	r	r	r	r	r	r	r							

Field	Bits	Type	Description
MOD_NUMBER	63:48	r	Module Identification Number.
-	47:16	r	Reserved
-	15:8	r	The value C0 indicates the peripheral as 64-bit module.
REV_NUMBER	7:0	r	Module Revision Number: Bits 7-0 are used for module revision numbering.

Special Function Registers

Interrupt Registers

6.2 Instruction Memory (I-Mem)

6.2.1 System Integration of I-Mem0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.14: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	November 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.15: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-11-27	Andreas Siggelkow	First draft (AS)

Functional Configuration of I-Mem0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a **I-Mem** peripheral are described in its functional description in Section 6.2.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for **I-Mem0** in Table 6.16.

Table 6.16: **I-Mem0** Feature Set

	Generic **I-Mem** Feature Set	Details
yes	Seperate Kernel Clock	`clk_core_s` (CGU ÷ 80); 1.25 MHz recommended, see Clock Concept Section 4.5.2
no	FIFO Data Buffering	no

HW Interfaces

The following table 6.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailled list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.17: HW Interfaces of Module **I-Mem0**

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	None	-
DMA	None	-
External Signals	addr_i data_o data_i wr_i	iBusAddr2_s iBusDataRd_s iBusDataWr_s iMemWr_s

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.18: Register List of Module **I-Mem0**

Register Name(s) in Chip	Name used in module register description	Comment Comment
None	-	-

Related Sections of this Spec

- **I-Mem** functional description starting with Section 6.2.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”
  - Peripheral ID Registers Section “Architecture Concepts”
  - Peripheral Clocking Concept Section “Architecture Concepts”
  - Clock Control Registers Section “Architecture Concepts”

- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.2.2 History of I-Mem

Table 6.19: History

History		
Target Spec.	Current version : 1.0, 2025-11-27 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-11-27		
6.2	6.2 / 2025-11-27	First draft (AS)

6.2.3 Functional Overview

The **I-Mem** peripheral is the internal instruction memory of the **RWU-RV64I** .

General Features:

- 32 kByte ROM (8192 words/instructions)
- Linear addressable
- 32 bit data width
- Wishbone bus slave

Synchronous Mode:

- 32 data bits (write; program download via JTAG)

Asynchronous Mode:

- 32 data bits (read; normal operation)

6.2.4 Structural Overview

The **I-Mem** is a Wishbone slave (asSlaveBPI) peripheral. It has one write port (for JTAG download) and one read port (for normal operation). The address is shared.

I/O of the I-Mem Kernel

Table 6.20 shows the input and outputs of the **I-Mem** kernel.

Table 6.20: **I-Mem** Kernel I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
addr_i	in	15	From program counter (PC). There is no byte access, so the lower two address bits will not be needed. In case of a program download, this is a part of the JTAG scan-chain (outside the block).
data_o	out	32	Instruction. Goes to the instruction decoder.
data_i	in	32	Data input port. From JTAG scan-chain (JTAG data register). This port is used for a program download.
wr_i	in	1	Write enable for data download. From JTAG scan-chain (JTAG data register). This port is used for a program download.

6.2.5 Service Requests of the I-Mem

None.

6.2.6 The I-Mem Peripheral Kernel

The memory is implemented as a synchronous write, asynchronous read memory. The write will be done on the rising edge of TCK and a write enable from a JTAG scan-chain.

6.2.7 Registers

None.

6.3 QSPI

6.3.1 System Integration of QSPI0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.21: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2026

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.22: History

Date	Name	Content Changes
Version of this document: V0.1		
2026-02-16	Andreas Siggelkow	First draft (AS, ChatGPT)
2026-04-13	A. Siggelkow, Claude	Completely revised HW
2026-05-12	A. Siggelkow, Claude	Add AXI4 data port (dual-bus architecture)

Functional Configuration of QSPI0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a QSPI peripheral are described in its functional description in Section 6.3.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for QSPI0 in Table 6.23.

Table 6.23: QSPI0 Feature Set

	Generic QSPI Feature Set	Details
yes	Separate Kernel Clock	`clk_core_s` (CGU ÷ 80 from external `clk_i`); target frequency TBD for `RV_PipelineCacheFwd` (QSPI0 exists in this variant only) – see the analogous `RV_NoPipeline` target in Clock Concept Section 4.5.2
yes	FIFO Data Buffering	RX and TX, each 16 entries × 64 bit = 128 bytes
yes	Programmable Baud Rate	$f_{SCK} = f_{kernel}/(2 \times (\text{CLKDIV} + 1))$, CLKDIV $\in [0 \ldots 255]$
yes	Single SPI Mode	1-1-1, cmd/adr/data on one wire
yes	Dual SPI Mode	1-1-2, data on two wires (DUAL=1, QUAD=0)
yes	Quad SPI Mode	1-1-4 and 1-4-4, data on four wires (QUAD=1)
yes	Configurable SPI Mode	CPOL and CPHA selectable (Mode 0 and Mode 3)
yes	3-/4-Byte Address	Selectable via ADDR_LEN bit
yes	Execute-In-Place (XIP)	Auto-read on bus access, fixed opcode from CMD register
yes	Double Data Rate (DDR)	DDR bit in CTRL register
yes	Interrupt Generation	7 interrupt sources, maskable via IMSC
yes	FIFO Flush	TX_FLUSH and RX_FLUSH bits in CTRL register
yes	Configurable Timeout	Timeout counter, threshold set via TIMEOUT register

HW Interfaces

The following table 6.24 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.24: HW Interfaces of Module QSPI0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus (config)	Wishbone slave	Wishbone B4 Classic
Bus (data)	AXI4 slave	AXI4 read-only (AR+R, 64-bit, 4-beat INCR burst)
Interrupt	qspi_irq_o	qspi_irq_s
DMA	None	-
External Signals	sck_o cs_o data_io[3:0]	qspiSck_o qspiCs_o qspi_data_io [3:0]

Bus Interface

The QSPI controller exposes two independent bus interfaces:

Wishbone Slave (configuration port) Provides software access to all configuration registers, the TX/RX FIFOs, and the interrupt registers. Implemented by `as_slave_bpi`. Protocol: Wishbone B4 Classic, zero-wait-state, 64-bit data width. See Section 4.3.6 (Architecture Concepts: Bus Concept).

AXI4 Slave (data port) Provides cache-refill read access for the Memory Arbiter (D-Cache has priority, I-Cache second). This port accepts 4-beat INCR bursts (ARLEN=3, ARSIZE=3, 64-bit data) and returns one 32-byte cache line per transaction. Only the AR and R channels are implemented; AW/W/B channels are absent. See Section 4.3.5 (Architecture Concepts: Bus Concept).

Registers

Table 6.25: Register List of Module QSPI0

offset	Register Name(s) in Chip	Name used in module register description	Comment
0x00	**QSPI0_ID**	id_reg_s	ID of QSPI0 peripheral
0x08	**QSPI0_CTRL**	ctrl_reg_s	Control register
0x10	**QSPI0_CMD**	cmd_reg_s	SPI command opcode
0x18	**QSPI0_ADDR**	addr_reg_s	Target flash address
0x20	**QSPI0_LEN**	len_reg_s	Transfer length in bytes
0x28	**QSPI0_DUMMY**	dummy_reg_s	Dummy cycles
0x30	**QSPI0_CLKDIV**	clkdiv_reg_s	SCK clock divider
0x38	**QSPI0_TIMEOUT**	timeout_reg_s	Timeout threshold register
0x40	**QSPI0_ISR**	isr_reg_s	Interrupt Set Register
0x48	**QSPI0_RIS**	ris_reg_s	Raw Interrupt Status Register
0x50	**QSPI0_IMSC**	imsc_reg_s	Interrupt Mask Control Register
0x58	**QSPI0_MIS**	mis_reg_s	Masked Interrupt Status Register
0x60	**QSPI0_ICR**	icr_reg_s	Interrupt Clear Register
0x68	**QSPI0_RXDATA**	rxdata_reg_s	RX FIFO read
0x70	**QSPI0_TXDATA**	txdata_reg_s	TX FIFO write
0x78	**QSPI0_FIFOSTAT**	fifostat_reg_s	FIFO fill levels
0x80	**QSPI0_XIPMODE**	xipmode_reg_s	XIP mode
0x88	**QSPI0_STATUS**	status_reg_s	Status flags

Related Sections of this Spec

- QSPI functional description starting with Section 6.3.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”

- AXI4 Cache Bus Section “Architecture Concepts” (Bus Concept)
- Peripheral ID Registers Section “Architecture Concepts”
- Peripheral Clocking Concept Section “Architecture Concepts”
- Clock Control Registers Section “Architecture Concepts”
- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.3.2 History of QSPI

Table 6.26: History

History		
Target Spec.	Current version : 1.0, 2026-02-17 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2026-02-17		
6.3	6.3 / 2026-02-17	First draft (AS)
6.3	6.3 / 2026-04-13	Complete rework (AS, Claude)

6.3.3 Functional Overview

The QSPI controller is a memory-mapped peripheral providing high-speed SPI and Quad-SPI access to external flash devices.

The controller exposes two bus interfaces: a Wishbone B4 Classic slave for software configuration and a read-only AXI4 slave for cache-refill data transfers (dummy citation [4]). Software accesses all registers via Wishbone; the Memory

Arbiter fetches 32-byte cache lines via the AXI4 port using 4-beat INCR bursts (ARLEN=3, ARSIZE=3, 64-bit).

General Features:

- Single, Dual, and Quad SPI modes
- Configurable dummy cycles
- Programmable baud rate
- TX and RX FIFOs
- Interrupt generation
- Execute-In-Place (XIP)

Synchronous Mode:

- 4 data bits

Asynchronous Mode:

- -

Reference Flash Devices

The QSPI controller is designed to be compatible with widely used NOR-Flash devices. Table 6.27 shows the relevant operating parameters for two representative devices. All register default values in this specification are set to match the Winbond W25Q128JV in standard (non-DDR) mode with CPOL=0, CPHA=0.

Table 6.27: Reference NOR-Flash Devices

Parameter	Winbond W25Q128JV	Micron MT25QL128	Comment
Capacity	128 Mbit (16 MByte)	128 Mbit (16 MByte)	
Supply Voltage	2.7 V…3.6 V	2.7 V…3.6 V	
SPI Modes supported	Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1)	Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1)	
Max. clock (STR)	133 MHz (Quad)	133 MHz (Quad)	
Default address width	3 Byte (24 bit)	3 Byte (24 bit)	4-Byte mode selectable
Quad enable	QE bit in Status Reg 2	via Enhanced Volatile Config Reg (0x61)	Must be set before QUAD=1
Relevant Read Opcodes:			
Read (slow)	0x03 (1-1-1, 0 dummy)	0x03 (1-1-1, 0 dummy)	$\leq$50 MHz
Fast Read	0x0B (1-1-1, 8 dummy)	0x0B (1-1-1, 8 dummy)	up to 133 MHz
Dual Output Read	0x3B (1-1-2, 8 dummy)	0x3B (1-1-2, 8 dummy)	DUAL=1
Quad Output Read	0x6B (1-1-4, 8 dummy)	0x6B (1-1-4, 8 dummy)	QUAD=1
Quad I/O Read	0xEB (1-4-4, 6+2 dummy)	0xEB (1-4-4, 10 dummy)	QUAD=1, fast
Relevant Write / Control Opcodes:			
Write Enable	0x06	0x06	Required before program/erase
Page Program	0x02 (1-1-1)	0x02 (1-1-1)	Up to 256 Byte
Sector Erase (4 KB)	0x20	0x20	
Block Erase (64 KB)	0xD8	0xD8	
Chip Erase	0xC7 / 0x60	0xC7 / 0x60	
JEDEC ID Read	0x9F	0x9F	Returns 3 Byte
JEDEC Manufacturer ID	0xEF (Winbond)	0x20 (Micron)	
Typical Dummy Cycles by Mode and Clock:			
1-1-1 Fast Read	8 cycles	8 cycles	DUMMY = 0x08
1-1-4 Quad Output	8 cycles	8 cycles	DUMMY = 0x08
1-4-4 Quad I/O	6 mode + 2 dummy cycles	10 cycles	DUMMY = 0x08 / 0x0A

Note on Quad Enable: Both reference devices require a Quad Enable (QE) bit to be set in the flash device's non-volatile status register before QUAD mode can be used. This is a one-time (or volatile) configuration step performed by the SW driver via a standard Write Enable (0x06) followed by a Write Status Register command. The QSPI controller itself does not perform this step automatically.

Functional Description OSPI-Kernel

The `as_qspi` kernel implements the SPI/QSPI state machine and clock generator. It is a fully state-driven Moore FSM. No FSM state depends on interrupt signals; all external behavior is driven by register state only.

The FSM traverses the states IDLE $\rightarrow$ CMD $\rightarrow$ ADDR $\rightarrow$ DUM $\rightarrow$ DAT $\rightarrow$ DONE $\rightarrow$ IDLE for a normal transaction. In XIP mode the CMD phase is omitted after the first transaction.

The SCK clock is generated internally from `clk_i` via a symmetric counter. The SCK frequency is:

$$f_{SCK} = \frac{f_{clk}}{2 \times (\text{CLKDIV} + 1)}$$

The kernel drives `data_io[3:0]` as output during CMD, ADDR, and TX-DAT phases and samples `data_io[0]` (single), `data_io[1:0]` (dual), or `data_io[3:0]` (quad) during RX-DAT phases.

Bits per Cycle (bpc):

- QUAD=1: bpc = 4
- DUAL=1, QUAD=0: bpc = 2
- DUAL=0, QUAD=0: bpc = 1 (default)

Start Semantics:

$$start_s = (start_i == 1) \ \wedge \ (stat_busy_o == 0)$$

The `start_i` signal must be a one-cycle pulse, generated by the BPI/Top from the CTRL.START write strobe.

XIP Mode: When `ctrl_reg_i.xip = 1` the CMD phase is skipped on subsequent transactions. The controller sets `xip_active_o` after the first DONE state. XIP is cleared by writing CTRL.XIP = 0; the current transfer completes before IDLE.

Error Latching:

- TX underflow: `tx_empty_i = 1` while in DAT phase and `tx_flag = 1` (TX direction)
- RX overflow: `rx_full_i = 1` while in DAT phase and `tx_flag = 0` (RX direction)
- Timeout: internal counter reaches zero while kernel is active

Kernel State Machine

Table 6.28 lists the states of the kernel FSM.

Table 6.28: `as_qspi` FSM States

State	`cs_o`	`sck_o`	Condition to leave
IDLE_ST	0	0	`start_i = 1`
CMD_ST	1	running	`cnt_cmd_r` has reached CMD_MAX (7); exit on `sck_drive_s`
ADDR_ST	1	running	`cnt_addr_r` has reached `addr_max_s`; exit on `sck_drive_s`
DUM_ST	1	running	`cnt_dum_r + 1` $\geq$ `dummy_reg_i`; exit on `sck_rise_s`
DAT_ST	1	running	`cnt_dat_r + bpc_s > dat_max_s`; exit on last sample (RX) or last drive (TX)
DONE_ST	0	0	Unconditional, one cycle

Figure 6.3 shows the state machine of the QSPI.

A simple Moore-FSM (Fig. 6.4): Additional to the FSM, a clock generator for sck_gen is required. This clock will be enabled or disabled by the FSM (clock enable cell). And more a few counters will be needed: a counter for the CMD-cycles (cnt_cmd), a counter for the ADDR-cycles (cnt_adr), a counter for the DUMMY-cycles (cnt_dum) and a counter for the DATA-cycles (cnt_dat). The counters end values will be either constants or loaded by a register. When writing to the TX-register a signal tx will be derived, this controls the direction of the data flow in the DATA state of the FSM.

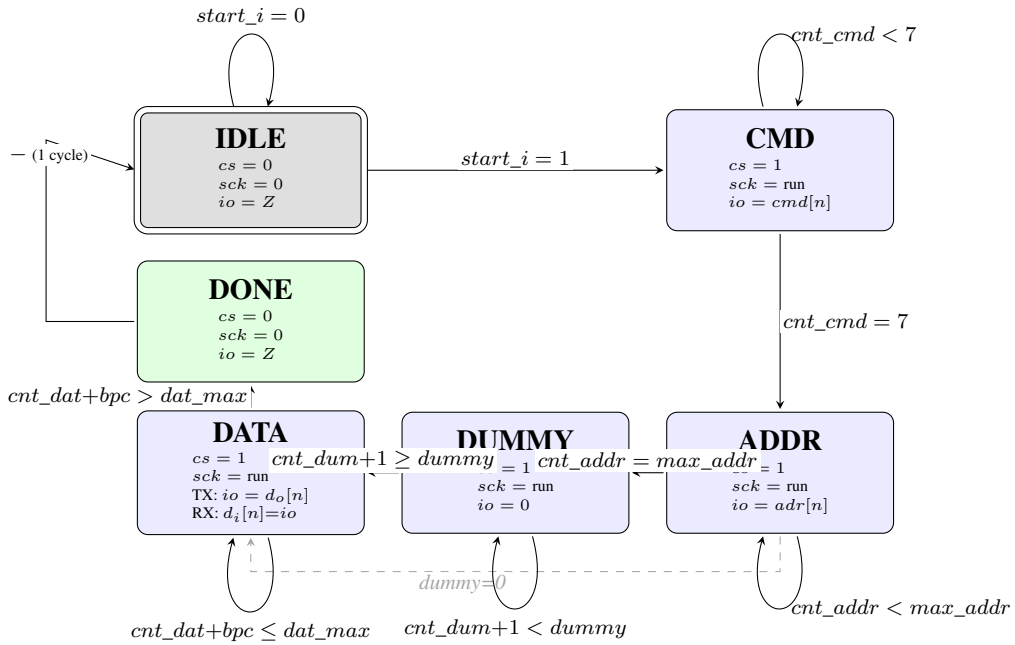


Figure 6.3: QSPI Kernel Moore-FSM. Doppelter Rahmen = Reset-Zustand IDLE (grau). Grüner Rahmen = DONE (1 Zyklus, bedingungslos zu IDLE). Gestrichelter Pfad: ADDR → DATA wenn `DUMMY_CYC` = 0.

Table 6.29 shows the kernel internel counters for the different parts of the SPI waveform.

Table 6.29: `as_qspi` Internal Counters

Counter	Width	Description
`sck_cnt_r`	8	SCK clock divider counter. Resets to `clkdiv_reg_i`.
`cnt_cmd_r`	3	CMD phase bit counter. Counts 0..7 (always 8 bits, single wire).
`cnt_addr_r`	5	ADDR phase bit counter. Max = 23 (3-byte) or 31 (4-byte).
`cnt_dum_r`	6	DUMMY cycle counter. Counts up to `dummy_reg_i`.
`cnt_dat_r`	20	DAT phase bit counter. Counts in steps of `bpc_s`. Max = `len_reg_i*8 - 1`.

Timing Constraints

- All configuration registers (cmd_reg_i, addr_reg_i, len_reg_i, etc.) must be stable before start_i is asserted and must remain stable for the duration of the transfer (BUSY=1).
- clkdiv_reg_i must not change while BUSY=1.
- tx_data_i must be valid when tx_rd_o fires. The BPI preloads the TX shift register one cycle before DAT_ST.
- rx_data_o is valid two clock cycles after rx_wr_o (NBA settling time in simulation; in synthesis: same cycle is valid).

Functional Description QSPI-BPI

The as_slave_bpi module implements a Wishbone Classic Compliant Slave BPI. It provides zero-wait-state single read/write transfers to the attached peripheral (QSPI top level). The BPI is fully combinatorial on the bus side; ACK is asserted in the same clock cycle as STB.

Protocol: A valid Wishbone transfer requires CYC_I = 1 and STB_I = 1 simultaneously. All SEL bits must be asserted (SEL_I = 0xFF) for a transfer to be acknowledged. The ACK is combinatorial:

$$ACK_O = CYC_I \ \wedge \ STB_I \ \wedge \ (\&SEL_I)$$

Write: When WE_I = 1, WR_O is asserted to the peripheral for the duration of the bus cycle. The address decoder in the Top level routes the write to the appropriate register.

Read: When WE_I = 0, RD_O is asserted. The peripheral must provide read data combinatorially on DAT_FROM_CORE_I; this is passed directly to WB_S_DAT_O.

Functional Description QSPI-Top-Level

as_qspi_top integrates the Wishbone BPI (as_slave_bpi), the QSPI kernel (as_qspi), the TX FIFO, and the RX FIFO into a complete memory-mapped QSPI peripheral. It decodes the register offset address, routes read/write strobes to the appropriate registers, and implements the interrupt logic.

Register Address Decoding: The BPI passes the raw bus address directly as a byte offset. The Top level compares this offset against the constants defined in Table 6.38 using combinatorial equality. There is no base-address stripping; the bus master must provide the byte offset from the peripheral's base address.

Start Pulse Generation: Writing `CTRL.START (bit 3) = 1` causes the Top level to register the write strobe for one clock cycle:

$$start_r \leftarrow wr_ctrl \wedge data_wr_s[3]$$

$$start_s = start_r \quad \text{(one-cycle pulse, registered)}$$

FIFO Flush: `CTRL.TX_FLUSH (bit 1)` and `CTRL.RX_FLUSH (bit 2)` are combinatorial and active only during the write cycle:

$$tx_flush_s = wr_ctrl \wedge data_wr_s[1]$$

$$rx_flush_s = wr_ctrl \wedge data_wr_s[2]$$

Flushing while BUSY is asserted is implementation-defined; in this implementation the flush takes effect immediately.

RX FIFO Write Pipeline: The kernel outputs `rx_wr_o` and `rx_data_o` (`rx_shift_r`) as NBA assignments in the same clock cycle. To avoid a simulation race condition (and to provide a registered interface for synthesis), a two-cycle pipeline is used:

- Cycle N: `rx_wr_kernel_s = 1`, `rx_shift_r` NBA pending.
- Cycle N+1: `rx_wr_d1 = 1`, `rx_data_kernel_s` captured into `rx_data_snap` (shift register now settled).
- Cycle N+2: `rx_wr_d2 = 1`, `rx_data_snap` written to RX FIFO.

RX FIFO Pop (read-before-pop): Reading `RXDATA` presents `mem[rd_ptr_r]` combinatorially and schedules the pointer advance one cycle later via `rx_pop_r`. This separates data presentation from pointer advancement and is required for correct operation with the asynchronous-read `as_fifo`.

6.3.4 Structural Overview

The QSPI controller is a dual-port peripheral: it is accessed from the Wishbone bus for configuration (register file, FIFOs, interrupts) and from the AXI4 bus for cache-refill data transfers. Both clock domains are synchronous to `clk_i`. Figure 6.4 shows the block diagram.

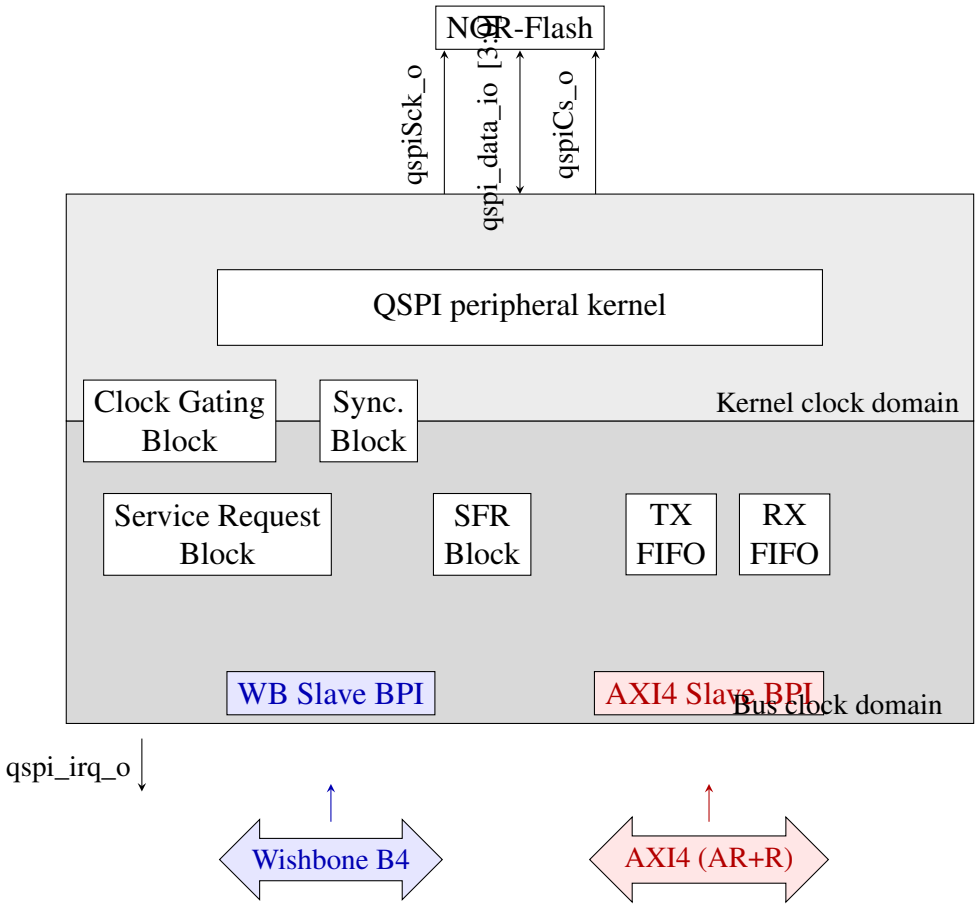


Figure 6.4: QSPI Dual-Port Peripheral (Wishbone config + AXI4 data)

QSPI Peripheral Kernel: The QSPI kernel provides the specific functionality of the QSPI interface, described in Section 6.3.6. It runs in the system clock domain.

Wishbone Slave BPI: Connects the QSPI configuration registers to the system Wishbone bus. Zero-wait-state, 64-bit. Handles all SW register accesses, FIFO reads/writes, and interrupt register updates.

AXI4 Slave BPI: Accepts cache-refill read requests from the Memory Arbiter via the AXI4 read channels (AR+R). Executes a Quad Fast Read (opcode 0x6B) on the NOR-Flash for each ARLEN=3 burst and returns 4 $\times$ 64-bit beats (32 bytes = one cache line) with `RLAST` asserted on the fourth beat.

asSlaveBPI Block: The Wishbone BPI block lies within the bus clock domain and contains the following subblocks:

Special Function Register Block The Special Function Register Block contains all FIFO specific registers and all special function registers (SFR) of the peripheral kernel that are accessible by the SW via the system bus.

Service Request Block SRB The SRB is used to prepare the interrupt and data transfer requests for the Interrupt Control Unit (ICU) and the DMA Controller (DMAC), respectively (no DMA planned!).

TX FIFO The TX FIFO is used to buffer the transmission data from the bus, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the system bus.

RX FIFO The RX FIFO is used to buffer the received data from the kernel, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the bus system.

Clock Gating Block: The Clock Gating Block is used to derive the necessary clocks for the several blocks within the peripheral from the clocks provided by the system.

Synchronization Block: The Synchronization Block is used to synchronize the signals between the bus clock and kernel clock domains.

I/O of the QSPI Kernel

Table 6.30, 6.31 shows the input and outputs of the QSPI kernel.

Table 6.30: as_qspi Ports – System and Configuration

Port	Dir.	Width	Description
clk_i	in	1	Kernel clock, active rising edge
rst_i	in	1	Asynchronous reset, active high
start_i	in	1	Start pulse (one cycle wide). Initiates FSM if not busy.
ctrl_reg_i	in	8	Packed qspi_ctrl_t: {addr_len, cpol, cpha, dual, cs_hold, xip, ddr, quad}
cmd_reg_i	in	8	SPI opcode byte
addr_reg_i	in	32	Flash target address
len_reg_i	in	16	Transfer length in bytes
dummy_reg_i	in	6	Number of dummy cycles
clkdiv_reg_i	in	8	SCK clock divider
timeout_reg_i	in	32	Timeout threshold in clock cycles; 0 = disabled
xip_mode_bits_i	in	8	XIP mode byte, driven on IO[3:0] during XIP address phase

Table 6.31: as_qspi Ports – Status, FIFO and SPI

Port	Dir.	Width	Description
stat_busy_o	out	1	FSM is active (not IDLE, not DONE). Level signal.
stat_done_o	out	1	FSM is in DONE state. High for exactly one clock cycle.
stat_error_o	out	1	Error latched. Remains until ICR clear.
stat_timeout_o	out	1	Timeout occurred. Remains until ICR clear.
xip_active_o	out	1	XIP mode is active.
tx_empty_i	in	1	TX FIFO empty indication (from TX FIFO or BPI)
tx_rd_o	out	1	TX FIFO read strobe. One cycle pulse on last TX drive.
tx_data_i	in	64	TX data word from TX FIFO
rx_full_i	in	1	RX FIFO full indication
rx_wr_o	out	1	RX FIFO write strobe. One cycle pulse on last RX sample.
rx_data_o	out	64	RX shift register output. Valid after rx_wr_o.
sck_o	out	1	SPI clock output
cs_o	out	1	Chip select (active high). Deasserted in IDLE and DONE.
data_io	inout	4	SPI data bus (tri-state). Driven in CMD/ADDR/TX-DAT, sampled in RX-DAT.

I/O of the QSPI BPI

Table 6.32 shows the input and outputs of the QSPI BPI.

Table 6.32: as_slave_bpi Ports

Port	Dir.	Width	Description
rst_i	in	1	System reset, active high (unused internally; for consistency)
clk_i	in	1	System clock (unused internally; for consistency)
Peripheral (kernel) side:			
addr_o	out	64	Decoded byte offset address, directly from wb_s_addr_i
dat_from_core_i	in	64	Read data from peripheral, presented combinatorially
dat_to_core_o	out	64	Write data to peripheral, directly from wb_s_dat_i
wr_o	out	1	Write enable to peripheral. Combinatorial.
rd_o	out	1	Read enable to peripheral. Combinatorial.
Wishbone slave side:			
wb_s_addr_i	in	64	Wishbone address (byte offset from peripheral base)
wb_s_dat_i	in	64	Wishbone write data
wb_s_dat_o	out	64	Wishbone read data
wb_s_we_i	in	1	Wishbone write enable
wb_s_sel_i	in	8	Wishbone byte select (all bytes must be selected)
wb_s_stb_i	in	1	Wishbone strobe (valid cycle)
wb_s_ack_o	out	1	Wishbone acknowledge (combinatorial, zero wait state)
wb_s_cyc_i	in	1	Wishbone cycle (high for complete bus transaction)

Internals of the QSPI BPI

Table 6.33 shows the internal signals and processes of the QSPI BPI.

Table 6.33: QSPI BPI Internals

Object	Type	Width	Explanation
any	logic	x	See BPI spec.

BPI Transfer Timing

Table 6.34: Wishbone Classic Zero-Wait-State Timing

Cycle	Description
T0	Master asserts CYC, STB, ADR, DAT (write) or ADR (read), WE, SEL
T0	Slave combinatorially asserts ACK. Read data appears on DAT_O.
T1	Master samples ACK and DAT_O on rising clock edge.
T1	Master deasserts STB (and CYC for single transfers).

Note: The peripheral register file is clocked (`always_ff`), so a write takes effect on the posedge of the clock during which `WR_O` is asserted. Read data must be available combinatorially before the posedge.

I/O of the QSPI AXI4 Slave Port

Table 6.35 lists the AXI4 slave signals of the QSPI data port. Only the read channels (AR and R) are implemented; write channels (AW, W, B) are absent. All signals are synchronous to `clk_i`.

Table 6.35: as_qspi_top AXI4 Slave Port Signals (AR + R channels only)

Signal	Dir.	Width	Description
AR channel (Address Read):			
axi_s_arvalid_i	in	1	Master presents a valid read-address transaction
axi_s_arready_o	out	1	Slave ready to accept address. Asserted when not busy.
axi_s_arid_i	in	4	Transaction ID. Echoed back on R channel.
axi_s_araddr_i	in	32	Byte address of first beat (must be 32-byte aligned)
axi_s_arlen_i	in	4	Burst length minus 1 (always 3; 4 beats per transaction)
axi_s_arsize_i	in	3	Beat size code (always 3 = 8 bytes = 64 bit)
axi_s_arburst_i	in	2	Burst type (always INCR = 2'b01)
R channel (Read Data):			
axi_s_rvalid_o	out	1	Slave presents valid read data
axi_s_rready_i	in	1	Master ready to accept data
axi_s_rid_o	out	4	Transaction ID (echo of axi_s_arid_i)
axi_s_rdata_o	out	64	Read data (one beat = 8 bytes of flash data)
axi_s_rresp_o	out	2	Response: OKAY (2'b00) or SLVERR (2'b10) on flash error
axi_s_rlast_o	out	1	Asserted on the 4th and final beat of each burst

Transaction Sequence:

1. Master (Memory Arbiter) asserts ARVALID with address, ID, ARLEN=3, ARSIZE=3, ARBURST=INCR.

2. Slave asserts ARREADY when idle; address is latched on the handshake cycle.

3. Slave initiates a Quad Fast Read (opcode 0x6B) on the NOR-Flash.

4. Slave returns 4 data beats on the R channel, asserting RVALID per beat and RLAST on beat 4.

5. On a flash error (timeout or kernel error), `RRESP` = SLVERR on the affected beat; the cache controller maps this to a RISC-V access-fault exception (mcause = 1 for I-Cache, 5 for D-Cache).

I/O of the QSPI Top

Table 6.36 shows the input and outputs of the QSPI Top.

Table 6.36: as_qspi_top Ports – System, Wishbone, and SPI

Port	Dir.	Width	Description
rst_i	in	1	Asynchronous reset, active high
clk_i	in	1	System and kernel clock
Wishbone slave port (configuration):			
wbdAddr_i	in	64	Wishbone address (byte offset from QSPI base)
wbdDat_i	in	64	Wishbone write data
wbdDat_o	out	64	Wishbone read data
wbdWe_i	in	1	Wishbone write enable
wbdSel_i	in	8	Wishbone byte select
wbdStb_i	in	1	Wishbone strobe
wbdAck_o	out	1	Wishbone acknowledge
wbdCyc_i	in	1	Wishbone cycle
AXI4 slave port (cache-refill data, AR+R channels only):			
axi_s_arvalid_i	in	1	AR valid
axi_s_arready_o	out	1	AR ready
axi_s_arid_i	in	4	AR transaction ID
axi_s_araddr_i	in	32	AR byte address (32-byte aligned)
axi_s_arlen_i	in	4	AR burst length (fixed 3 = 4 beats)
axi_s_arsize_i	in	3	AR beat size (fixed 3 = 8 bytes)
axi_s_arburst_i	in	2	AR burst type (fixed INCR = 2'b01)
axi_s_rvalid_o	out	1	R valid
axi_s_rready_i	in	1	R ready
axi_s_rid_o	out	4	R transaction ID (echo of arid)
axi_s_rdata_o	out	64	R data
axi_s_rresp_o	out	2	R response (OKAY / SLVERR)
axi_s_rlast_o	out	1	R last beat
SPI / Flash and interrupt:			
sck_o	out	1	SPI clock to NOR-Flash
cs_o	out	1	Chip select to NOR-Flash (active high)
data_io	inout	4	QSPI data bus to/from NOR-Flash
qspi_irq_o	out	1	Interrupt request to CPU interrupt controller

Table 6.37 shows the internals of the QSPI Top.

Table 6.37: QSPI Top Signals

Signal	Width	Explanation
tbd	1	tbd

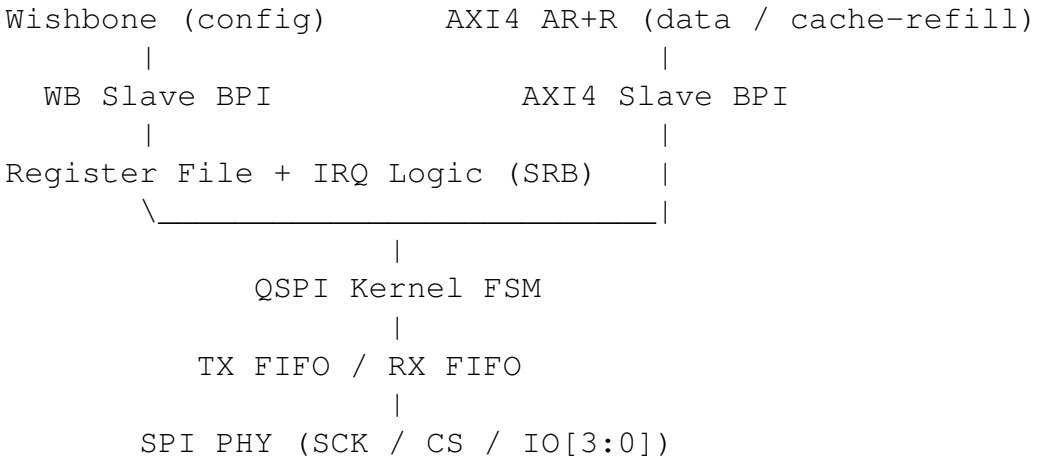
6.3.5 Service Requests of the QSPI

Interrupt sources are level-based and derived directly from kernel status or FIFO fill levels.

The details will be shown in section 6.3.7.

6.3.6 The QSPI Peripheral Kernel

Block Architecture -: The block architecture is like:



The kernel is fully state-driven and does not depend on external events such as interrupts or bus timing.

Writing bit 3 of **QSPI0_CTRL .START** initiates a transaction if not busy.

Design Rules -:

- The kernel FSM is fully state-driven.
- No FSM state depends on interrupt signals.
- Interrupts never control hardware behavior.
- All external behavior is driven by register state only.

6.3.7 Registers

In the following the registers of a QSPI peripheral are listed and described. Some registers' layout or availability depends on the generic feature set implemented for that QSPI's instantiation. Such registers and concerned bit fields are denoted by the use of italic style.

QSPI Register Overview

Table 6.38: `as_qspi_top` Register Map

Offset	Symbol	Reset	Access	Description
0x00 (0)	ID	0x00000000_00000010	r	Peripheral identification register
0x08 (8)	CTRL	0x00000000_00000000	rw	Control register (START, FLUSH, mode bits)
0x10 (16)	CMD	0x00000000_00000000	rw	SPI opcode
0x18 (24)	ADDR	0x00000000_00000000	rw	Flash target address
0x20 (32)	LEN	0x00000000_00000000	rw	Transfer length in bytes
0x28 (40)	DUMMY	0x00000000_00000000	rw	Dummy cycles
0x30 (48)	CLKDIV	0x00000000_00000000	rw	SCK clock divider
0x38 (56)	TIMEOUT	0x00000000_00000000	rw	Timeout threshold
0x40 (64)	ISR	0x00000000_00000000	w	Interrupt Set Register (w1 $\rightarrow$ set RIS)
0x48 (72)	RIS	0x00000000_00000000	rh	Raw Interrupt Status Register
0x50 (80)	IMSC	0xFFFFFFFF_FFFFFFFF	rw	Interrupt Mask Control Register
0x58 (88)	MIS	0x00000000_00000000	rh	Masked Interrupt Status (= RIS & IMSC)
0x60 (96)	ICR	0x00000000_00000000	w	Interrupt Clear Register (w1 $\rightarrow$ clear RIS)
0x68 (104)	RXDATA	0x00000000_00000000	r	RX FIFO pop (read-before-pop)
0x70 (112)	TXDATA	0x00000000_00000000	w	TX FIFO push
0x78 (120)	FIFOSTAT	0x00000000_00000000	rh	FIFO fill levels and flags
0x80 (128)	XIPMODE	0x00000000_000000A0	rw	XIP mode byte (Winbond default: 0xA0)
0x88 (136)	STATUS	0x00000000_00000000	rh	Kernel status flags

Read-Write Features

- **r** – read-only; writes have no effect
- **w** – write-only; reads return 0
- **rw** – read/write; value is retained after reset
- **rh** – read-only; value is set by hardware
- **X0** – read always returns 0; write-only access

System Registers

QSPI0_ID -: Identification register of the QSPI peripheral. Read-only; writes have no effect.

ID register [Reset value: 0000000000000010_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								PERIPH_ID							
r								r							

Field	Bits	Type	Description
-	63:8	r	Reserved, reads as 0
PERIPH_ID	7:0	r	Peripheral identification number. Fixed value `0x10`.

Special Function Registers

QSPI0_CTRL -: Main control register. Controls SPI mode, address length, clock polarity/phase, XIP, DDR, CS-hold, FIFO flush, and transfer start. All configuration bits must be set before asserting START.

Control register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	AL	CP	CH	DU	CS	XI	DD	QU	ST	RF	TF	0
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	w	w	w	r

Field	Bits	Type	Description
-	63:12	r	Reserved
AL	11	rw	ADDR_LEN. 0 = 3-byte address (24 bit); 1 = 4-byte (32 bit).
CP	10	rw	CPOL. Clock polarity. 0 = idle low (Mode 0); 1 = idle high (Mode 3).
CH	9	rw	CPHA. Clock phase. 0 = sample on leading edge; 1 = sample on trailing edge.
DU	8	rw	DUAL. Enable dual-wire data (1-1-2). Ignored when QUAD=1.
CS	7	rw	CS_HOLD. Keep CS asserted between consecutive transactions.
XI	6	rw	XIP. Enable execute-in-place mode.
DD	5	rw	DDR. Enable double data rate.
QU	4	rw	QUAD. Enable quad-wire data (1-1-4 / 1-4-4). Overrides DUAL.
ST	3	w	START. Write 1 to initiate a transfer when BUSY=0. Self-clearing.
RF	2	w	RX_FLUSH. Write 1 to flush RX FIFO. Self-clearing.
TF	1	w	TX_FLUSH. Write 1 to flush TX FIFO. Self-clearing.
-	0	r	Reserved.

QSPI0_CMD -: The SPI/QSPI command opcode to be sent in the CMD phase.

Command register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	OPCODE							
r	r	r	r	r	r	r	r	rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
OPCODE	7:0	rw	SPI command opcode. Sent MSB-first in single-wire CMD phase.

QSPI0_ADDR -: Target byte address in the NOR-Flash device.

Address register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADDR[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
ADDR[15:0]															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
ADDR	31:0	rw	Flash byte address. Sent MSB-first. Number of address bytes determined by CTRL.AL.

QSPI0_LEN -: Transfer data length in bytes.

Length register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
LEN															
rw															

Field	Bits	Type	Description
-	63:16	r	Reserved
LEN	15:0	rw	Data transfer length in bytes. Must be ≥ 1.

QSPI0_DUMMY -: Number of dummy clock cycles between ADDR and DATA phases.

Dummy register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	DUMMY_CYC					
r	r	r	r	r	r	r	r	r	r	rw					

Field	Bits	Type	Description
-	63:6	r	Reserved
DUMMY_CYC	5:0	rw	Dummy cycle count. 0 = no dummy phase (ADDR directly followed by DATA). Maximum 63.

QSPI0_CLKDIV -: SCK clock divider. Determines the SPI clock frequency.

$$f_{SCK} = \frac{f_{clk}}{2 \times (\text{CLKDIV} + 1)}$$

Must not be changed while BUSY=1.

Clock divider register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								CLKDIV							
r								rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
CLKDIV	7:0	rw	SCK divider. Reset 0x00 = $f_{clk}/2$ (maximum SCK).

QSPI0_TIMEOUT -: Timeout threshold in kernel clock cycles. When the FSM is active and the down-counter reaches zero, a timeout error is raised. Set to 0 to disable.

Timeout register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIMEOUT_THR[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
TIMEOUT_THR[15:0]															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
TIMEOUT_THR	31:0	rw	Timeout threshold in clock cycles. 0x00000000 = disabled.

QSPI0_TXDATA -: Writing to this register pushes data into the TX FIFO. If the TX FIFO is full, the write is silently discarded; the Wishbone ACK is still asserted. Reading returns 0.

TX data register (write pushes to TX FIFO) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
TX_DATA[63:48]															
w															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
TX_DATA[47:32]															
w															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TX_DATA[31:16]															
w															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
TX_DATA[15:0]															
w															

Field	Bits	Type	Description
TX_DATA	63:0	w	Write data word pushed into TX FIFO. MSB transmitted first. Reads return 0.

QSPI0_RXDATA -: Reading this register pops one 64-bit word from the RX FIFO and returns it. The pop (pointer advance) is delayed by one clock cycle (read-before-pop semantics). If the RX FIFO is empty, the read returns 0x0000000000000000 and no pop occurs. Writing has no effect.

RX data register (read pops from RX FIFO) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
RX_DATA[63:48]															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
RX_DATA[47:32]															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RX_DATA[31:16]															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
RX_DATA[15:0]															
r															

Field	Bits	Type	Description
RX_DATA	63:0	r	Read data word popped from RX FIFO. MSB received first. Writes have no effect.

QSPI0_FIFOSTAT -: FIFO fill level and status flags. Read-only. Updated combinatorially.

FIFO status register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0	0	0	0	0	0	RF	RH	0	0	0	RX_LEV[4:0]				
r	r	r	r	r	r	rh	rh	r	r	r	rh				
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	TF	TH	0	0	0	TX_LEV[4:0]				
r	r	r	r	r	r	rh	rh	r	r	r	rh				

Field	Bits	Type	Description
-	63:26	r	Reserved
RF	25	rh	RX FIFO full
RH	24	rh	RX FIFO $\geq$ half full (level $\geq$ FIFO_DEPTH/2)
-	23:21	r	Reserved
RX_LEV	20:16	rh	RX FIFO fill level (0…16)
-	15:10	r	Reserved
TF	9	rh	TX FIFO full
TH	8	rh	TX FIFO $<$ half full (level $<$ FIFO_DEPTH/2)
-	7:5	r	Reserved
TX_LEV	4:0	rh	TX FIFO fill level (0…16)

QSPI0_XIPMODE -: XIP mode byte driven on IO[7:4] (upper nibble on IO[3:0]) during the XIP address phase to terminate the mode register entry sequence (e.g. Winbond W25Q128JV: 0xA0).

XIP mode byte register [Reset value: $00000000000000A0_H$]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								XIP_BYTE							
r								rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
XIP_BYTE	7:0	rw	Mode byte for XIP address phase. Default 0xA0 (Winbond W25Q128JV).

QSPI0_STATUS -: Kernel status register. All bits are read-only and updated combinatorially.

Note on DONE: `STATUS.DONE` reflects the combinatorial `stat_done_o` signal which is high only while the kernel is in DONE_ST (one clock cycle). It is

not suitable for polling. Use `RIS.DO` instead, which latches the DONE event and holds it until cleared via ICR.

Status register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	XI	DO	BU
r	r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	rh	TIMEOUT. Timeout occurred; latched until ICR.TIMEOUT is written.
ER	3	rh	ERROR. Error latched; latched until ICR.ERROR is written.
XI	2	rh	XIP_ACTIVE. XIP mode is currently active.
DO	1	rh	DONE. High for exactly one clock cycle when FSM leaves DAT_ST. For polling use RIS.DO.
BU	0	rh	BUSY. FSM is active (not IDLE).

Interrupt Registers

Interrupt sources are level-based and derived directly from kernel status.

Flow of Interrupt Requests -: Figure 6.5 shows the flow of a single interrupt request, which comes from the Peripheral Kernel or from the FIFO Controller.

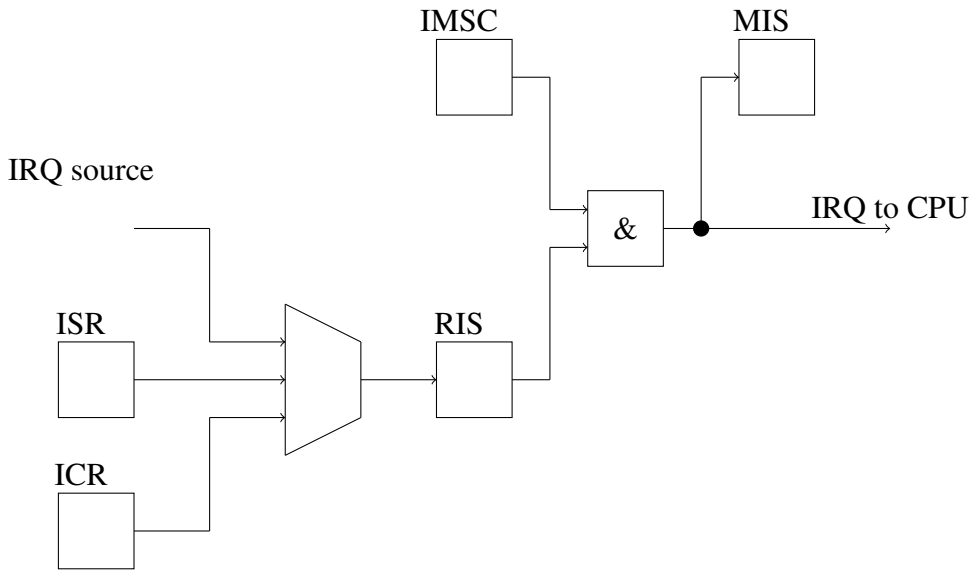


Figure 6.5: Interrupt Request Flow

A pulse on an interrupt request <xyz>_IRQ sets the corresponding interrupt status bit in the Raw Interrupt Status Register RIS. The interrupt status bits can be set with the Interrupt Set Register ISR or can be cleared with the Interrupt Clear Register ICR.

The Interrupt Mask Control Register IMSC enables or disables the level sensitive interrupts to the ICU. The Masked Interrupt Status Register MIS gives the current masked status value of the corresponding interrupt.

If an IMSC bit is disabled while its interrupt is active, then the interrupt for the ICU will be removed, but only until this IMSC bit is enabled again, unless the interrupt has been cleared in the mean time.

If the IMSC bit is not enabled when an interrupt source becomes active, then the interrupt bit will only be set in the RIS register. If the corresponding IMSC bit is later enabled, the interrupt will consequently become active in the MIS register.

Before enabling an interrupt in the MIS bit, it is good practice to always first clear the corresponding interrupt in the RIS via ICR.

In QSPI:

$$MIS = RIS\&IMSC$$

and then the external interrupt output is:

$$IRQ = |MIS$$

Interrupt Sources -:

Table 6.39: QSPI Interrupt Sources

RIS Bit	Name	Source	Latch Semantics
0	DONE	Rising edge of `stat_done_s` (FSM enters DONE_ST)	Set on edge; hold until ICR
1	RXHALF	Rising edge of `rx_half_s` (RX FIFO $\geq$ half full)	Set on edge; hold until ICR
2	TXHALF	Rising edge of `tx_half_s` (TX FIFO $<$ half full)	Set on edge; hold until ICR
3	ERROR	Rising edge of `stat_error_s`	Set on edge; hold until ICR
4	TIMEOUT	Rising edge of `stat_timeout_s`	Set on edge; hold until ICR
63:5	–	–	Reserved

QSPI0_RIS -: Raw Interrupt Status Register. Read-only. Reflects latched interrupt conditions independent of masking. Set by hardware edge events or by ISR write. Cleared by ICR write. Writing has no effect (use ISR/ICR instead).

Raw Interrupt Status Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	rh	TIMEOUT. Timeout threshold reached.
ER	3	rh	ERROR. Error latched in kernel.
TH	2	rh	TXHALF. TX FIFO below half-full threshold.
HA	1	rh	RXHALF. RX FIFO at or above half-full threshold.
DO	0	rh	DONE. Transfer completed. Latches on rising edge of `stat_done_s`.

QSPI0_IMSC -: Interrupt Mask Control Register. Each bit enables (1) or disables (0) the corresponding interrupt source. Reset value `0xFFFFFFFFFFFFFFFF` means all interrupts are enabled after reset.

Interrupt Mask Control Register [Reset value: $FFFFFFFFFFFFFFFF_H$]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
IMSC[63:48]															
rw															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
IMSC[47:32]															
rw															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IMSC[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Field	Bits	Type	Description
-	63:5	rw	Reserved (kept for forward compatibility; writes ignored by hardware)
TI	4	rw	TIMEOUT mask. 1 = interrupt enabled.
ER	3	rw	ERROR mask.
TH	2	rw	TXHALF mask.
HA	1	rw	RXHALF mask.
DO	0	rw	DONE mask.

QSPI0_MIS -: Masked Interrupt Status Register. Read-only. Returns the effective interrupt status as seen by the CPU. MIS $=$ RIS $\&$ IMSC.

Masked Interrupt Status Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	rh	TIMEOUT masked
ER	3	rh	ERROR masked
TH	2	rh	TXHALF masked
HA	1	rh	RXHALF masked
DO	0	rh	DONE masked

QSPI0_ICR -: Interrupt Clear Register. Write-only. Writing 1 to a bit clears the corresponding RIS bit. Writing 0 has no effect. Reading returns 0. The interrupt should be cleared only after the condition has been handled (FIFO drained / refilled) to avoid re-triggering.

Interrupt Clear Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	w	w	w	w	w

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	w	Write 1 to clear RIS.TIMEOUT
ER	3	w	Write 1 to clear RIS.ERROR
TH	2	w	Write 1 to clear RIS.TXHALF
HA	1	w	Write 1 to clear RIS.RXHALF
DO	0	w	Write 1 to clear RIS.DONE

QSPI0_ISR -: Interrupt Set Register. Write-only. Provided for software testing of the interrupt path. Writing 1 forces the corresponding RIS bit to 1. Writing 0 has no effect. Reading returns 0.

Interrupt Set Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	w	w	w	w	w

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	w	Write 1 to force-set RIS.TIMEOUT (test only)
ER	3	w	Write 1 to force-set RIS.ERROR (test only)
TH	2	w	Write 1 to force-set RIS.TXHALF (test only)
HA	1	w	Write 1 to force-set RIS.RXHALF (test only)
DO	0	w	Write 1 to force-set RIS.DONE (test only)

6.3.8 Software Programming Sequence

Normal Single Read (e.g. 0x0B Fast Read, 8 B)

1. Write `CLKDIV` to set the desired SCK frequency.
2. Write `CMD` = `0x0B`.
3. Write `ADDR` = target flash address.
4. Write `LEN` = `8`.
5. Write `DUMMY` = `8`.
6. Optionally write `IMSC` to unmask DONE interrupt (bit 0).
7. Write `CTRL` = `0x08` (START = 1, all mode bits = 0 for single mode).
8. Poll `RIS.DO = 1` or wait for `qspi_irq_o`.
9. Write `ICR` = `0x01` to acknowledge DONE.
10. Read `RXDATA` to retrieve received bytes (one 64-bit read per 8 bytes).

Quad Read (e.g. 0x6B Quad Output Fast Read, 8 B)

Same as above, with `CTRL` = `0x18` (QUAD=1, START=1, bit 4 and bit 3 set) and `DUMMY` = `8`.

TX Write (e.g. 0x02 Page Program, 8 B)

1. Write `TXDATA` with up to 8 bytes (pushed to TX FIFO).
2. Program `CMD`, `ADDR`, `LEN`, `DUMMY` as needed.
3. Write `CTRL` = `0x08` (START = 1).
4. Poll `RIS.DO` or wait for IRQ. Clear with ICR.

XIP Enable

1. Configure `CLKDIV`, `DUMMY`, `CMD` (e.g. 0xEB).

2. Write `CTRL` = `0x50` (QUAD=1, XIP=1, START=0).

3. XIP becomes active after the next DONE. `STATUS.XI` = 1.

4. CPU reads to the flash address range are now automatically serviced.

5. To exit XIP: write `CTRL.XIP = 0`.

6.4 UART

6.4.1 Requirements

This section defines the functional requirements for the **UART0** peripheral (module `as_uart_top`) in the **RWU-RV64I** system. Requirements follow IEEE 29148 terminology: *shall* = mandatory, *should* = recommended.

Each requirement identifier has the form **UART-*CAT*-*NNN***, where *CAT* is the category code (see Table 6.40) and *NNN* is a three-digit sequence number.

Table 6.40: UART Requirements – Category Codes

Category Code	Scope
FR	Frame Format
BR	Baud Rate and Oversampling
TX	Transmitter Path
RX	Receiver Path
FI	FIFOs
ER	Error Detection
IR	Interrupts
SW	Software Interface (Register Map)
HW	Hardware Interface (Pins)

Frame Format Requirements (FR)

Table 6.41: UART Requirements – Frame Format

ID	Requirement	Notes / Rationale
UART-FR-001	The UART *shall* support data word lengths of 5, 6, 7, and 8 bits per frame, configurable via **UART0_LCR**[1:0]: 00 = 5 bit, 01 = 6 bit, 10 = 7 bit, 11 = 8 bit (reset).	Covers the full range of the standard asynchronous serial frame (RS-232 / UART).
UART-FR-002	The UART *shall* support three parity modes configurable via **UART0_LCR**[3:2]: 00 = none (reset), 01 = odd, 10 = even. Encoding 11 is reserved (no parity).	Standard 8N1, 7E1, 7O1, etc. coverage. The parity bit is inserted between the last data bit and the first stop bit.
UART-FR-003	The UART *shall* support 1 stop bit (reset) and 2 stop bits, selectable via **UART0_LCR**[4]: 0 = 1 stop bit, 1 = 2 stop bits.	1.5 stop bits are omitted; they are only defined for 5-bit data frames and not required in this application.
UART-FR-004	The UART *shall* support forced break transmission: when **UART0_LCR**[5]=1 the TX output *shall* be driven continuously to logic 0, independent of FIFO content and frame boundaries.	Required by POSIX `tcsendbreak()` and Linux `termios`. Break ends when LCR[5] is cleared.

Baud Rate and Oversampling Requirements (BR)

Table 6.42: UART Requirements – Baud Rate and Oversampling

ID	Requirement	Notes / Rationale
UART-BR-001	The UART *shall* derive the baud-rate clock from the system clock f_{clk} using a 16-bit integer divisor stored in **UART0_CLKDIV**[15:0]: $$f_{baud} = \frac{f_{clk}}{16 \times \text{CLKDIV}}$$ CLKDIV = 0 is illegal; the hardware reset value is CLKDIV = 8 (RST_CLKDIV in as_uart_top.sv).	16× oversampling is the industry standard (16550 compatible). CLKDIV = 8 targets 9600 Baud at the actual $f_{clk} = 1.25$ MHz clk_core_s (Section 4.5.2); actual rate ≈9765.6 Baud (+1.72 % error) – see the note below.
UART-BR-002	The valid range of CLKDIV *shall* be 1…65535. Writing CLKDIV = 0 *shall* have no effect (value rejected, previous value retained).	CLKDIV = 0 would cause division-by-zero in the baud-rate counter.
UART-BR-003	The oversampling ratio *shall* be 16. Each received bit *shall* be sampled at sub-clock 8 of its 16-sub-clock period (centre-of-bit sampling).	Centre sampling maximises noise margin relative to both edges.

Note on the actual f_{clk} in `RV_NoPipeline` (fixed 2026-07-12): earlier revisions of this document assumed a 125 MHz kernel clock (a legacy Zybo-FPGA-prototype figure predating the CGU-based clock architecture, Section 4.5.1) and set RST_CLKDIV = 68 accordingly (115 200 Baud at 125 MHz). UART0's clk_i is actually clk_core_s, recommended at 1.25 MHz for RV_NoPipeline (Section 4.5.2) – 100× slower than 125 MHz, and too slow to reach 115 200 Baud at all ($f_{clk}/16 = 78.125$ kBaud is the fastest this clock can drive, even at CLKDIV = 1). RST_CLKDIV has therefore been changed to **8**, targeting the traditional safe-default 9600 Baud (actual rate ≈9765.6 Baud, +1.72 % error, well within standard UART tolerance) at the real 1.25 MHz clk_core_s. This does not affect any existing firmware or testbench: every UART0 user in this repo reprograms **UART0_CLKDIV** explicitly before use (the verified UART0 simulation tests in

Section 7.1.1 use `CLKDIV` = 4, chosen for simulation speed, not for a standard baud rate) and never relies on the reset default.

Transmitter Requirements (TX)

Table 6.43: UART Requirements – Transmitter

ID	Requirement	Notes / Rationale
UART-TX-001	The transmitter *shall* serialize data frames from the TX FIFO: start bit (0), data bits LSB first, parity bit (if enabled), stop bit(s) (1).	Standard UART bit ordering.
UART-TX-002	The TX output (`tx_o`) *shall* be logic 1 (mark state) when idle and between frames.	Required so that the receiver can detect the start-bit falling edge.
UART-TX-003	The transmitter *shall* fetch the next frame from the TX FIFO only when the FIFO is non-empty and the shift register is idle. If the TX FIFO is empty while the shift register finishes the current frame, `tx_o` *shall* return to idle (logic 1).	Prevents underrun: partial frames are never transmitted.
UART-TX-004	When **UART0_LCR**[5] = 1 (break), the transmitter *shall* drive `tx_o` = 0 continuously until LCR[5] is cleared, overriding normal FIFO operation.	See UART-FR-004.

Receiver Requirements (RX)

Table 6.44: UART Requirements – Receiver

ID	Requirement	Notes / Rationale
UART-RX-001	The `rx_i` input *shall* be synchronised through a 2-stage flip-flop synchroniser operating in the peripheral clock domain before any logic (edge detection, sampling) is applied to it.	Mandatory metastability protection for any asynchronous input on an FPGA; omitting it can cause the entire system to malfunction.
UART-RX-002	Start-bit detection *shall* be performed by detecting a high-to-low transition (falling edge) on the synchronised `rx_i` signal.	Marks the beginning of a new frame.
UART-RX-003	After start-bit detection, each data bit and (if enabled) the parity bit *shall* be sampled at sub-clock 8 of the corresponding bit period (centre sampling, see UART-BR-003).	The start-bit centre is used to align the oversampling counter so that subsequent bits are sampled at their centres.
UART-RX-004	On successful frame completion (valid stop bit detected), the received data word *shall* be written into the RX FIFO.	Frames with a missing stop bit generate a frame error (see UART-ER-001) and the data word is still pushed to the FIFO so that SW can inspect the erroneous byte.
UART-RX-005	When **UART0_CTRL**[0] = 1 (loopback), the receiver input *shall* be internally connected to the TX shift-register output, bypassing the external `rx_i` pin.	Enables SW self-test without external hardware.

FIFO Requirements (FI)

Table 6.45: UART Requirements – FIFOs

ID	Requirement	Notes / Rationale
UART-FI-001	The peripheral *shall* provide a TX FIFO and an RX FIFO. FIFO depth *shall* be set by the SystemVerilog parameter `FIFO_DEPTH` (default: 16 entries, 8 bits per entry).	Decouples bus transaction timing from baud-rate timing; prevents CPU stalls on every byte.
UART-FI-002	Each FIFO *shall* provide the following status signals to the register block: full, empty, half-full (level > FIFO_DEPTH/2), and current fill level as a $\lceil \log_2(\text{FIFO_DEPTH} + 1) \rceil$-bit integer.	Required for FIFOSTAT register (UART-SW-001).
UART-FI-003	FIFO status *shall* be readable via **UART0_FIFOSTAT**.	Allows software to perform polled I/O without relying on interrupts.
UART-FI-004	Writing 1 to **UART0_CTRL**[1] (tx_flush) *shall* synchronously reset the TX FIFO to the empty state. The bit *shall* auto-clear on the next clock cycle.	Used to abort a pending transmission without a full peripheral reset.
UART-FI-005	Writing 1 to **UART0_CTRL**[2] (rx_flush) *shall* synchronously reset the RX FIFO to the empty state. The bit *shall* auto-clear on the next clock cycle.	Used to discard stale receive data after a configuration change.
UART-FI-006	The RX interrupt threshold *shall* be configurable via **UART0_RXTHRES**[4:0] (valid range: 1…FIFO_DEPTH; reset: FIFO_DEPTH/2 = 8). The rx_ready interrupt source (RIS[0]) *shall* be asserted when the RX FIFO level is $\geq$ RXTHRES.	Allows SW to tune the interrupt rate vs. latency trade-off without code changes in the driver.

Error Detection Requirements (ER)

Table 6.46: UART Requirements – Error Detection

ID	Requirement	Notes / Rationale
UART-ER-001	The UART *shall* detect a **frame error**: the expected stop bit is sampled as logic 0. On detection, STATUS[4] (frame_err) *shall* be set.	Indicates baud-rate mismatch or line noise.
UART-ER-002	The UART *shall* detect a **parity error** when parity is enabled (LCR[3:2] $\neq 00$): the received parity bit differs from the locally computed parity of the received data word. On detection, STATUS[5] (parity_err) *shall* be set.	Indicates single-bit data corruption.
UART-ER-003	The UART *shall* detect an **overrun error**: the RX FIFO is full when the receiver completes a new frame. The new byte *shall* be discarded, and STATUS[6] (overrun_err) *shall* be set.	Signals that the driver is not reading fast enough.
UART-ER-004	The UART *shall* detect a **break condition**: `rx_i` (synchronised) is held continuously at logic 0 for ≥ 1 complete character time (start + data + parity + stop bits) after the initial falling edge. On detection, STATUS[7] (break_det) *shall* be set.	Required for Linux `tty` break handling and RS-232 modem signalling.
UART-ER-005	The error flags STATUS[7:4] *shall* be sticky (latched): they *shall* remain set until explicitly cleared by writing 1 to the corresponding bit in **UART0_ICR**.	Prevents loss of error information if the ISR runs after a delay.
UART-ER-006	Each of the four error types (frame, parity, overrun, break) *shall* generate an individual interrupt source in RIS (see UART-IR-004).	Allows driver to classify the error without reading STATUS separately.

Interrupt Requirements (IR)

Table 6.47: UART Requirements – Interrupts

ID	Requirement	Notes / Rationale
UART-IR-001	The UART *shall* provide an **rx_ready** interrupt source (RIS[0]) asserted when the RX FIFO fill level is $\geq$ **UART0_RXTHRES**.	Primary trigger for the receive interrupt service routine.
UART-IR-002	The UART *shall* provide a **tx_ready** interrupt source (RIS[1]) asserted when the TX FIFO fill level falls below **UART0_RXTHRES** (space available for writing).	Allows SW to refill the TX FIFO in an interrupt-driven transmit loop without polling.
UART-IR-003	The UART *shall* provide an **rx_timeout** interrupt source (RIS[2]) asserted when the RX FIFO contains at least one byte and no new byte has been received for ≥ 4 character times.	Prevents data stranded in the RX FIFO below the threshold from being silently ignored when a frame does not exactly fill the FIFO to RXTHRES.
UART-IR-004	The UART *shall* provide individual interrupt sources: frame_err (RIS[3]), parity_err (RIS[4]), overrun_err (RIS[5]), break_det (RIS[6]).	See UART-ER-006.
UART-IR-005	Each interrupt source *shall* be individually maskable via the corresponding bit in **UART0_IMSC**. The interrupt output signal (`uart_irq_o`) *shall* equal the bitwise OR of all (RIS & IMSC) bits: `uart_irq_o` = | (RIS & IMSC).	One-IRQ-line model consistent with GPIO0 and QSPI0.
UART-IR-006	Any bit in RIS *shall* be clearable by writing 1 to the corresponding bit of **UART0_ICR** (write-1-to-clear).	Consistent with QSPI0 interrupt model.

Software Interface Requirements (SW)

Table 6.48: UART Requirements – Software Interface

ID	Requirement	Notes / Rationale
UART-SW-001	All configuration, status, FIFO access, and interrupt control *shall* be accessible through exactly 12 memory-mapped, 64-bit registers on a Wishbone B4 Classic slave interface implemented by `as_slave_bpi`.	Consistent with GPIO0 and QSPI0.
UART-SW-002	Registers *shall* be 8-byte aligned and occupy the address range UART0_BASE + 0x00 to UART0_BASE + 0x5F (12×8 bytes).	Fits within the reserved address range 0x000000010000-0120 – 0x000000010000-017F.
UART-SW-003	The base address of UART0 *shall* be `0x0000_0001_0000_0120`	As defined in the system memory map (`as_pack.sv`: `uart0_start_address_c`).
UART-SW-004	Write accesses to read-only register fields *shall* have no effect; the field value *shall* not change.	Standard HW register protection.
UART-SW-005	Read accesses to write-only registers (ICR, and DATA when read returns RX FIFO data) *shall* return the correct value: ICR reads as 0x0; DATA read pops the RX FIFO.	DATA register is bi-directional: write pushes TX FIFO, read pops RX FIFO (standard UART DATA register model).
UART-SW-006	The Wishbone interface *shall* respond with zero wait states (ACK asserted one clock after STB & CYC).	Consistent with all other Wishbone slaves in the system.

Hardware Interface Requirements (HW)

Table 6.49: UART Requirements – Hardware Interface

ID	Requirement	Notes / Rationale
UART-HW-001	The peripheral *shall* expose exactly two external serial I/O signals: `tx_o` (output) and `rx_i` (input).	Standard asynchronous UART interface; no modem control signals (RTS/CTS) required for this application.
UART-HW-002	`rx_i` *shall* be treated as an asynchronous input and synchronised by a 2-stage flip-flop synchroniser in the peripheral clock domain before use (see UART-RX-001).	Mandatory for FPGA synthesis tool to infer proper placement of the input register chain and to suppress metastability.
UART-HW-003	The peripheral clock (`clk_i`) is `clk_core_s`, the CGU-derived core clock (fixed `clk_i_ext`/80 divider; see the Clock Concept, Section 4.5.1) – **not** a direct 125 MHz board clock as in earlier (Zybo FPGA prototype) revisions of this document. For `RV_NoPipeline`, the recommended operating point is 100 MHz external / 1.25 MHz core (Section 4.5.2).	Determines the achievable baud rate resolution (UART-BR-001). The 125 MHz assumption previously in this requirement was stale and has been corrected; `RST_CLKDIV` was updated to match – see the note after Table 6.42.

6.4.2 System Integration of UART0

History

Table 6.50: UART0 Authors

Author	Department	From
Andreas Siggelkow	Fac. E	June 2026

Table 6.51: UART0 Section History

Date	Name	Content Changes
Version of this document: V0.1		
2026-06-05	A. Siggelkow, Claude	First draft: Requirements section
Version of this document: V0.2		
2026-06-05	A. Siggelkow, Claude	Simulation testbenches (uart_tx, uart_rx); RX FIFO pop bug fix (`rd_data_r` in `as_uart_top.sv`)

Functional Configuration of UART0

This section should describe the product-specific instantiation of this module's feature set.

The standard features of the UART peripheral are described in its functional description in Section 6.4.4. Generic features for UART0 are listed in Table 6.52.

Table 6.52: UART0 Feature Set

	Generic UART Feature Set	Details
yes	Separate Kernel Clock	`clk_core_s` (CGU ÷ 80); 1.25 MHz recommended for `RV_NoPipeline` (Section 4.5.2)
yes	Configurable Data Bits	5, 6, 7, 8 bits via LCR
yes	Configurable Parity	None (reset), odd, even via LCR
yes	Configurable Stop Bits	1 (reset) or 2 via LCR
yes	Programmable Baud Rate	$f_{baud} = f_{clk}/(16 \times \text{CLKDIV})$; see the note after Table 6.42 regarding the reset value
yes	TX FIFO	16 entries × 8 bit
yes	RX FIFO	16 entries × 8 bit
yes	RX FIFO Threshold	Configurable 1…16, reset: 8
yes	Loopback Mode	CTRL[0]
yes	Break Generation	LCR[5]
yes	RX Input Synchroniser	2-stage FF, mandatory metastability protection
yes	Frame Error Detection	STATUS[4], RIS[3]
yes	Parity Error Detection	STATUS[5], RIS[4]
yes	Overrun Error Detection	STATUS[6], RIS[5]
yes	Break Detect	STATUS[7], RIS[6]
yes	RX Timeout Interrupt	RIS[2], after 4 character times of FIFO inactivity
yes	Interrupt Generation	7 sources, maskable via IMSC
yes	FIFO Flush	tx_flush / rx_flush in CTRL
no	9-bit Data Words	Not required
no	1.5 Stop Bits	Not required
no	Hardware Flow Control (RTS/CTS)	Not required
no	Modem Control Signals	Not required

HW Interfaces

The following table lists all hardware interfaces relevant for normal operation.

Table 6.53: HW Interfaces of Module UART0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	Wishbone slave	Wishbone B4 Classic
Interrupt	uart_irq_o	uart0_irq_s
DMA	None	-
External Signals	tx_o rx_i	uart0_tx_o uart0_rx_i

Bus Interface

The UART controller exposes one bus interface:

Wishbone Slave (configuration port) Provides software access to all configuration registers, the TX/RX FIFOs, and the interrupt registers. Implemented by `as_slave_bpi`. Protocol: Wishbone B4 Classic, zero-wait-state, 64-bit data width. See Section 4.3.6 (Architecture Concepts: Bus Concept).

Registers

Table 6.54: Register List of Module UART0

Offset	External Name	Internal Name	Mode	Description
0x00	**UART0_ID**	id_reg_s	ro	Peripheral identification
0x08	**UART0_LCR**	lcr_reg_s	rw	Line Control: data bits, parity, stop bits, break
0x10	**UART0_CLKDIV**	clkdiv_reg_s	rw	Baud rate divisor (16-bit)
0x18	**UART0_CTRL**	ctrl_reg_s	rw	Control: loopback, tx_flush, rx_flush
0x20	**UART0_STATUS**	status_reg_s	ro	TX/RX busy flags; sticky error flags
0x28	**UART0_DATA**	data_reg_s	rw	Write: push TX FIFO; Read: pop RX FIFO
0x30	**UART0_FIFOSTAT**	fifostat_reg_s	ro	TX/RX FIFO level, full, half, empty
0x38	**UART0_RXTHRES**	rxthres_reg_s	rw	RX FIFO interrupt threshold (1…16)
0x40	**UART0_RIS**	ris_reg_s	ro	Raw Interrupt Status
0x48	**UART0_IMSC**	imsc_reg_s	rw	Interrupt Mask Control
0x50	**UART0_MIS**	mis_reg_s	ro	Masked Interrupt Status (= RIS & IMSC)
0x58	**UART0_ICR**	icr_reg_s	wo	Interrupt Clear (write-1-to-clear RIS bits)

Related Sections of this Spec

- UART functional description starting with Section 6.4.4.
- Interrupt and DMA Concept in Chapter “Architecture Concepts”.
- Peripheral Architecture Concept in Chapter “Architecture Concepts”:
  - Slave WB (Wishbone) BPI.

  - Peripheral ID Registers.
  - Peripheral Clocking Concept.
  - Horizontal Interrupt and DMA Register Structure.
  - Basic Interrupt and DMA Register Set.
  - Synchronization Block.
  - Standardized Flip-Flop based FIFO.
  - TX-FIFO Registers.
  - RX-FIFO Registers.

- Requirements: Section 6.4.1.

6.4.3 History of UART

Table 6.55: History of UART Section

History		
Target Spec.	Current version: 0.2, 2026-06-05 Previous version: 0.1, 2026-06-05	Author: A. Siggelkow
Paragraph (prev.)	Paragraph (current) / date	Changes
Chapter Version: C0.1; Date: 2026-06-05		
—	6.4 / 2026-06-05	First draft: Requirements + System Integration stubs (AS, Claude)
Chapter Version: C0.2; Date: 2026-06-05		
—	7.1.1 / 2026-06-05	Simulation testbenches (uart_tx, uart_rx); RX FIFO pop bug fix (AS, Claude)

6.4.4 Functional Overview

To be written.

The UART peripheral provides standard asynchronous serial communication in half- or full-duplex mode. It interfaces to the system via a Wishbone B4 Classic slave bus and forwards received bytes to software via an RX FIFO and interrupts.

General Features:

- Configurable frame format (data bits, parity, stop bits)
- Programmable baud rate
- TX and RX FIFOs
- RX input synchroniser (metastability protection)
- Frame, parity, overrun, and break error detection
- RX timeout interrupt
- Loopback mode for self-test
- Interrupt generation (7 sources, maskable)

6.4.5 Structural Overview

To be written.

6.4.6 Service Requests of the UART

Table 6.56 shows the interrupt service requests of UART0.

Table 6.56: UART0 Service Requests

Service Request	RIS Bit	IRQ	Explanation
rx_ready	RIS[0]	uart0_irq_s	RX FIFO level $\geq$ RXTHRES.
tx_ready	RIS[1]	uart0_irq_s	TX FIFO level $<$ RXTHRES (space available).
rx_timeout	RIS[2]	uart0_irq_s	Data in RX FIFO; no new byte for $\geq$ 4 character times.
frame_err	RIS[3]	uart0_irq_s	Stop bit received as logic 0.
parity_err	RIS[4]	uart0_irq_s	Received parity bit differs from computed parity.
overrun_err	RIS[5]	uart0_irq_s	RX FIFO full; incoming byte discarded.
break_det	RIS[6]	uart0_irq_s	rx_i held low for $\geq$ 1 full character time.

6.4.7 The UART Peripheral Kernel

To be written.

6.4.8 Registers

To be written – detailed bit-field descriptions.

UART Register Overview

Table 6.57: UART Register List

Register Group	Register Name	Symbol	Offset
System Registers	Identification Register	**UART0_ID**	0x00
Configuration	Line Control Register	**UART0_LCR**	0x08
	Baud Rate Divisor Register	**UART0_CLKDIV**	0x10
	Control Register	**UART0_CTRL**	0x18
Status	Status Register	**UART0_STATUS**	0x20
	FIFO Status Register	**UART0_FIFOSTAT**	0x30
Data	Data Register (TX/RX)	**UART0_DATA**	0x28
Threshold	RX Threshold Register	**UART0_RXTHRES**	0x38
Interrupt Registers	Raw Interrupt Status	**UART0_RIS**	0x40
	Interrupt Mask Control	**UART0_IMSC**	0x48
	Masked Interrupt Status	**UART0_MIS**	0x50
	Interrupt Clear Register	**UART0_ICR**	0x58

Read-Write Features

- **w**: writable by SW
- **r**: readable by SW
- **h**: set by HW only
- **w1c**: write-1-to-clear

LCR – Line Control Register (offset 0x08, rw)

Table 6.58: UART0_LCR Bit Fields

Bits	Name	Mode	Description
[1:0]	DATABITS	rw	Data word length: 00 = 5 bit, 01 = 6 bit, 10 = 7 bit, 11 = 8 bit (reset).
[3:2]	PARITY	rw	Parity mode: 00 = none (reset), 01 = odd, 10 = even, 11 = reserved.
[4]	STOPBITS	rw	Stop bit count: 0 = 1 stop bit (reset), 1 = 2 stop bits.
[5]	BREAK	rw	Force break: 0 = normal (reset), 1 = TX held low.
[63:6]	–	r	Reserved; reads as 0.

CTRL – Control Register (offset 0x18, rw)

Table 6.59: UART0_CTRL Bit Fields

Bits	Name	Mode	Description
[0]	LOOPBACK	rw	0 = normal (reset), 1 = loopback (TX connected internally to RX).
[1]	TX_FLUSH	w	Write 1 to flush TX FIFO; auto-clears after one clock cycle.
[2]	RX_FLUSH	w	Write 1 to flush RX FIFO; auto-clears after one clock cycle.
[63:3]	–	r	Reserved; reads as 0.

STATUS – Status Register (offset 0x20, ro)

Table 6.60: UART0_STATUS Bit Fields

Bits	Name	Mode	Description
[0]	TX_BUSY	rh	1 while TX shift register is active.
[1]	RX_BUSY	rh	1 while RX shift register is active (frame in progress).
[3:2]	–	r	Reserved.
[4]	FRAME_ERR	rh	Sticky: frame error detected. Cleared by ICR[3].
[5]	PARITY_ERR	rh	Sticky: parity error detected. Cleared by ICR[4].
[6]	OVERRUN_ERR	rh	Sticky: overrun error. Cleared by ICR[5].
[7]	BREAK_DET	rh	Sticky: break condition detected. Cleared by ICR[6].
[63:8]	–	r	Reserved; reads as 0.

FIFOSTAT – FIFO Status Register (offset 0x30, ro)

Table 6.61: UART0_FIFOSTAT Bit Fields

Bits	Name	Mode	Description
[4:0]	TX_LEVEL	rh	TX FIFO fill level (0…FIFO_DEPTH).
[5]	TX_EMPTY	rh	1 when TX FIFO contains no entries.
[6]	TX_HALF	rh	1 when TX FIFO level $\leq$ FIFO_DEPTH/2.
[7]	TX_FULL	rh	1 when TX FIFO is full.
[15:8]	–	r	Reserved.
[20:16]	RX_LEVEL	rh	RX FIFO fill level (0…FIFO_DEPTH).
[21]	RX_EMPTY	rh	1 when RX FIFO contains no entries.
[22]	RX_HALF	rh	1 when RX FIFO level $\geq$ FIFO_DEPTH/2.
[23]	RX_FULL	rh	1 when RX FIFO is full.
[63:24]	–	r	Reserved; reads as 0.

RIS – Raw Interrupt Status (offset 0x40, ro)

Table 6.62: UART0_RIS Bit Fields

Bits	Name	Mode	Description
[0]	RX_READY	rh	RX FIFO level $\geq$ RXTHRES.
[1]	TX_READY	rh	TX FIFO level $<$ RXTHRES (space available).
[2]	RX_TIMEOUT	rh	Data in RX FIFO; no new byte for ≥ 4 character times.
[3]	FRAME_ERR	rh	Frame error (see UART-ER-001). Cleared by ICR[3].
[4]	PARITY_ERR	rh	Parity error (see UART-ER-002). Cleared by ICR[4].
[5]	OVERRUN_ERR	rh	Overrun error (see UART-ER-003). Cleared by ICR[5].
[6]	BREAK_DET	rh	Break detected (see UART-ER-004). Cleared by ICR[6].
[63:7]	–	r	Reserved; reads as 0.

ICR – Interrupt Clear Register (offset 0x58, wo)

Table 6.63: UART0_ICR Bit Fields

Bits	Name	Mode	Description
[0]	CLR_RX_READY	w1c	Write 1 clears RIS[0] (rx_ready).
[1]	CLR_TX_READY	w1c	Write 1 clears RIS[1] (tx_ready).
[2]	CLR_RX_TIMEOUT	w1c	Write 1 clears RIS[2] (rx_timeout).
[3]	CLR_FRAME_ERR	w1c	Write 1 clears RIS[3] and STATUS[4].
[4]	CLR_PARITY_ERR	w1c	Write 1 clears RIS[4] and STATUS[5].
[5]	CLR_OVERRUN_ERR	w1c	Write 1 clears RIS[5] and STATUS[6].
[6]	CLR_BREAK_DET	w1c	Write 1 clears RIS[6] and STATUS[7].
[63:7]	–	–	Reserved; writes ignored.

6.5 ICU

6.5.1 Requirements

This section defines the functional requirements for the **ICU0** peripheral (module `cc_icu_top`, kernel `cc_icu`) in the **RWU-RV64I** system. Requirements follow IEEE 29148 terminology: *shall* = mandatory, *should* = recommended. ICU0 is identical on both chip variants (Section 4.2, Table 4.29); differences are called out explicitly where they exist.

Each requirement identifier has the form **ICU-*CAT*-*NNN***, where *CAT* is the category code (Table 6.64) and *NNN* is a three-digit sequence number. The full requirement derivation (including the requester's original wording and every gap identified during implementation) is kept as the authoritative source in `ip_icu/README_REQ.txt`; this section is the spec-formatted summary of the same content.

Table 6.64: ICU Requirements – Category Codes

Category Code	Scope
SR	Source Management (count, assignment, spares)
PR	Priority Resolution
MA	Masking
PE	Pending/Clear Semantics
ID	Cause Identification
SW	Software Interface (Register Map)
HW	Hardware Interface (Pins, Integration)

Source Management Requirements (SR)

Table 6.65: ICU Requirements – Source Management

ID	Requirement	Notes / Rationale
ICU-SR-001	The ICU *shall* reserve input slots for peripherals that do not exist yet: at least 2 spare sources.	Explicitly requested; allows new interrupt-capable peripherals to be connected without widening the register map again.
ICU-SR-002	The ICU *shall* accept one interrupt input per existing interrupt-capable peripheral, plus the spare sources from ICU-SR-001.	GPIO0 and UART0 on both variants; QSPI0 additionally on *RV_PipelineCacheFwd*.
ICU-SR-003	The register file *shall* provision exactly 8 source slots (a full byte per bitmap register), of which 3 are wired to real peripherals, 2 are the requested spares, and 3 remain reserved.	Both chips get an identical register map with unused slots tied to `1'b0`; a single firmware driver works unmodified on either chip (ICU-HW-003).

Priority Resolution Requirements (PR)

Table 6.66: ICU Requirements – Priority Resolution

ID	Requirement	Notes / Rationale
ICU-PR-001	The ICU *shall* resolve priority in hardware across all simultaneously pending, unmasked sources, via a software-writable 4-bit priority value per source (**ICU0_PRIO**).	Explicitly requested (“Priorität ist wichtig”); unlike a fixed source-index priority, this is reconfigurable at runtime.
ICU-PR-002	On equal priority between two or more pending, unmasked sources, the *lowest* source index *shall* win.	Not specified by the requester; left undefined this would be simulator/synthesis-dependent – a real correctness bug class.

Masking Requirements (MA)

Table 6.67: ICU Requirements – Masking

ID	Requirement	Notes / Rationale
ICU-MA-001	The ICU *shall* provide an individually maskable enable bit per source (**ICU0_IMSC**).	Explicitly requested (“Masking ist wichtig”).
ICU-MA-002	The ICU *shall* additionally provide one global interrupt enable bit (**ICU0_CTRL**.GIE) that gates the aggregated output line independent of per-source masking.	Cheap “disable everything” critical-section primitive; per-source IMSC alone cannot do this atomically for an arbitrary/changing set of enabled sources.

Pending/Clear Semantics Requirements (PE)

Table 6.68: ICU Requirements – Pending/Clear Semantics

ID	Requirement	Notes / Rationale
ICU-PE-001	A source’s pending bit (**ICU0_RIS**) *shall* be edge-triggered and sticky: it latches the source line’s 0→1 transition and remains set until explicitly cleared by software, independent of the source line’s subsequent level.	Matches GPIO0’s IRQSS / UART0’s RIS convention.
ICU-PE-002	A source’s pending bit *shall not* be silently cleared by a software CLEAR (**ICU0_ICR**) that coincides, in the same clock cycle, with a new hardware edge (or software-forced SET via **ICU0_ISR**) on that same bit: SET *shall* take priority over CLEAR.	Root-caused against a documented defect in the graded reference submission this peripheral replaces (`ip_icu/DUMMY_ICU/asInterruptCont` a same-cycle set+clear on the same bit was resolved implicitly, in the clear’s favour, silently dropping a real interrupt occurrence.

Cause Identification Requirements (ID)

Table 6.69: ICU Requirements – Cause Identification

ID	Requirement	Notes / Rationale
ICU-ID-001	The ICU *shall* expose a register (**ICU0_IRQID**) identifying the current highest-priority pending, unmasked source, so that firmware does not have to poll every peripheral's own MIS register to determine the interrupt cause.	Not named explicitly by the requester, but the single biggest practical benefit of a real ICU over the OR-combine placeholder it replaces; see the "Open items" consequence already recorded in Section 3.3.8 prior to this peripheral's introduction.
ICU-ID-002	**ICU0_IRQID** *shall* carry an explicit VALID bit, separate from the source-id field.	Source index 0 (GPIO0) is a real, valid id; collapsing "no interrupt pending" onto id = 0 would make the register ambiguous – the same class of 0-as-sentinel bug as ICU-PE-002.
ICU-ID-003	**ICU0_IRQID** *shall* reflect pending/priority state independent of **ICU0_CTRL**.GIE.	Keeps IRQID a useful diagnostic register even while interrupts are globally disabled (see ICU-MA-002); only the CPU-facing output line is gated by GIE.

Software Interface Requirements (SW)

Table 6.70: ICU Requirements – Software Interface

ID	Requirement	Notes / Rationale
ICU-SW-001	The ICU *shall* be controllable exclusively through memory-mapped, 64-bit registers on a Wishbone B4 Classic slave interface implemented by `as_slave_bpi`.	Consistent with GPIO0, QSPI0, UART0, CGU.
ICU-SW-002	Registers *shall* be 8-byte aligned.	Project-wide convention (Section 9.1).
ICU-SW-003	The base address of ICU0 *shall* be `0x0000_0001_0000_0800` (cs[5]), the first free 512 B slot after UART0.	As defined in `as_pack.sv` (`icu0_start_address_c`, `chipsel`) and `as_decode.sv`.
ICU-SW-004	The ICU *shall* expose a read-only peripheral ID register at offset 0x00.	Consistent with every other peripheral in this SoC.
ICU-SW-005	Reset values *shall* keep the system interrupt-functional without any firmware setup: **ICU0_IMSC** resets to all-enabled, **ICU0_CTRL**.GIE resets to 1.	Matches the fact that interrupts already worked (via the OR-combine placeholder / direct wiring this peripheral replaces) before firmware had any chance to configure a real ICU.
ICU-SW-006	The Wishbone interface *shall* respond with zero wait states (combinational ACK).	Consistent with all other Wishbone slaves in the system.

Hardware Interface Requirements (HW)

Table 6.71: ICU Requirements – Hardware Interface

ID	Requirement	Notes / Rationale
ICU-HW-001	The ICU *shall* expose exactly one aggregated interrupt output line (`icu_irq_o`) to the CPU boundary.	Explicitly requested. Both `as_cpux` and `cc_cpupipefwd` consume only `irq_ext_i[7]` (Section 3.3.8); `icu_irq_o` drives exactly that bit on both chips.
ICU-HW-002	`icu_irq_o` *shall* be a level output, asserted for as long as at least one unmasked, pending, globally-enabled source exists.	Matches `mip[11]`'s existing read-only, level-mirroring semantics (Section 3.3.8); no CPU changes are required to consume it.
ICU-HW-003	ICU0 *shall* be a single, shared design instantiated identically on both chip variants, with a fixed, chip-independent source-index assignment (Table 6.75).	A single firmware ICU driver must work unmodified on either chip.
ICU-HW-004	The ICU performs no cross-clock-domain synchronization; all sources connected to `irq_src_i` *shall* already be synchronous to `clk_i`.	True for all current sources (GPIO0, QSPI0, UART0 and the CPU all run on `clk_div_s` on both chips). A future source on a different clock domain must be synchronized into `clk_div_s` before reaching `irq_src_i`; the ICU itself provides no synchronizer.

6.5.2 System Integration of ICU0

History

Table 6.72: ICU0 Authors

Author	Department	From
Claude Sonnet 5 (AI-generated, `cc_` prefix)	–	July 2026

Table 6.73: ICU0 Section History

Date	Name	Content Changes
Version of this document: V0.1		
2026-07-14	Claude, A. Siggelkow	First draft: full ICU0 spec section, written from `ip_icu/README_REQ.txt` and `ip_icu/README_IMPL.txt` after standalone verification and integration into both chip variants

Functional Configuration of ICU0

The standard features of the ICU peripheral are described in its functional description in Section 6.5.4. Generic features for ICU0 are listed in Table 6.74.

Table 6.74: ICU0 Feature Set

	Generic ICU Feature Set	Details
yes	Separate Kernel Clock	`clk_div_s`, shared with all other peripherals and the CPU on both variants
yes	Configurable Source Count	8 slots, 3 wired + 2 spare + 3 reserved (Table 6.75)
yes	Software-Configurable Priority	4-bit/source, **ICU0_PRIO**
yes	Per-Source Masking	**ICU0_IMSC**
yes	Global Interrupt Enable	**ICU0_CTRL**.GIE
yes	Cause Identification	**ICU0_IRQID** (valid + id)
yes	Software Test/Force	**ICU0_ISR**
no	Vectored Dispatch	Not implemented; both CPUs support `mtvec` Direct mode only (Section 3.3.8)
no	Nested/Preemptive Interrupts	Not implemented; precluded at the CPU level (`mstatus.MIE` cleared on trap entry) independent of the ICU

Table 6.75: ICU0 Source Index Assignment (fixed, identical register map on both chips)

Index	Name	*RV_NoPipeline*	*RV_PipelineCacheFwd*
0	GPIO0	wired (`irq_gpiox_s`)	wired (`irq_gpiox_s`)
1	UART0	wired (`irq_uart_s`)	wired (`irq_uart_s`)
2	QSPI0	tied to `1'b0` (no QSPI0 in this variant)	wired (`qspi_irq_s`)
3	SPARE0	tied to `1'b0`	tied to `1'b0`
4	SPARE1	tied to `1'b0`	tied to `1'b0`
5–7	reserved	tied to `1'b0`	tied to `1'b0`

Behavioural change on *RV_NoPipeline*: before ICU0 existed, this variant wired UART0's interrupt output directly to `irq_external_s[6]`, a bit `as_cpux` never reads (only bit 7 is synchronized and consumed, Section 3.3.8) – a UART0 interrupt had no effect on program flow, by design gap rather than intent. Routed through ICU0, UART0 now reaches the CPU for the first time once unmasked in **ICU0_IMSC**[1]. Tied-off sources' RIS bits can still be forced via **ICU0_ISR** for software testing but will never set from real hardware.

HW Interfaces

The following table lists all hardware interfaces relevant for normal operation.

Table 6.76: HW Interfaces of Module ICU0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_div_s
Bus	Wishbone slave	Wishbone B4 Classic
Interrupt out	icu_irq_o	icu_irq_s → `irq_ext_i[7]`
Interrupt in	irq_src_i(7:0)	{5'b0, qspi_irq_s[(1)], irq_uart_s, irq_gpiox_s}
DMA	None	-
External Signals	None	– (register-mapped peripheral only)

[(1)] tied to `1'b0` on *RV_NoPipeline* (no QSPI0 instance in that variant).

Bus Interface

ICU0 exposes one bus interface:

Wishbone Slave (configuration port) Provides software access to all ICU0 registers. Implemented by `as_slave_bpi`. Protocol: Wishbone B4 Classic, zero-wait-state, 64-bit data width. See Section 4.3.6 (Architecture Concepts: Bus Concept).

Registers

Table 6.77: Register List of Module ICU0

Offset	External Name	Internal Name	Mode	Description
0x00	**ICU0_ID**	id_reg_s	ro	Peripheral identification (constant 0x40)
0x08	**ICU0_RIS**	ris_reg_s	ro	Raw Interrupt Status: sticky, edge-triggered, 1 bit/source
0x10	**ICU0_IMSC**	imsc_reg_s	rw	Interrupt Mask Control: 1 bit/source, 1 = enabled
0x18	**ICU0_MIS**	mis_reg_s	ro	Masked Interrupt Status = RIS & IMSC (registered)
0x20	**ICU0_ICR**	(n/a, wo strobe)	wo	Interrupt Clear: write-1-to-clear RIS bits
0x28	**ICU0_ISR**	(n/a, wo strobe)	wo	Interrupt Set (SW test/-force): write-1-to-set RIS bits
0x30	**ICU0_PRIO**	prio_reg_s	rw	8×4-bit priority nibbles, packed in bits[31:0]
0x38	**ICU0_IRQID**	irqid_s	ro	bit[3] = VALID, bits[2:0] = id of highest-priority pending source
0x40	**ICU0_CTRL**	ctrl_gie_r	rw	bit[0] = GIE, global interrupt enable

Related Sections of this Spec

- ICU functional description starting with Section 6.5.4.
- Interrupt Concept, Section 3.3.8 (Product Overview) and Section 4.2 area (Architecture Concepts): the CPU-side CSR/trap mechanism ICU0's output feeds into.
- External Interrupt Map, Table 9.6 (Memory Map and Registers chapter).
- Peripheral Architecture Concept in Chapter "Architecture Concepts":
  - Slave WB (Wishbone) BPI.
  - Peripheral ID Registers.

- Requirements: Section 6.5.1.
- Implementation source material: `ip_icu/README_REQ.txt`, `ip_icu/README_IMPL`

6.5.3 History of ICU

Table 6.78: History of ICU Section

History		
Target Spec.	Current version: 0.1, 2026-07-14 Previous version: —	Author: Claude, A. Siggelkow
Paragraph (prev.)	Paragraph (current) / date	Changes
Chapter Version: C0.1; Date: 2026-07-14		
—	6.5 / 2026-07-14	First draft: requirements, system integration, functional/structural overview, service requests, kernel description, full register map (AS, Claude)

6.5.4 Functional Overview

ICU0 aggregates up to 8 peripheral interrupt lines into the single external interrupt bit both CPU implementations consume (`irq_ext_i[7]`, Section 3.3.8), with hardware priority resolution, per-source and global masking, and a cause register

so firmware does not have to poll every peripheral's own MIS register to determine which one interrupted.

It replaces two previous, more limited mechanisms:

- On *RV_PipelineCacheFwd*, a placeholder that simply OR-combined GPIO0, QSPI0, and UART0 with no priority, masking, or cause identification.
- On *RV_NoPipeline*, direct wiring of GPIO0 to `irq_external_s[7]` and UART0 to `irq_external_s[6]` – a bit the CPU never reads, so UART0 interrupts had no effect on program flow.

General Features:

- 8 interrupt source slots, register-identical on both chip variants
- Software-configurable priority per source (4 bits, 16 levels)
- Per-source masking and one global enable bit
- Edge-triggered, sticky pending bits with race-free set/clear semantics (Section 6.5.7)
- Cause identification register (valid + highest-priority id), independent of the global enable
- Software test/force register for verification without real peripheral stimulus

6.5.5 Structural Overview

ICU0 follows the same kernel/BPI split as GPIO0 (`as_gpio`/`as_gpio_top`): a stateless kernel performs per-source edge detection and priority arbitration, while the top-level wrapper owns all Wishbone-addressable register state and instantiates the shared `as_slave_bpi` Wishbone-to-kernel translator.

Table 6.79: ICU Kernel (cc_icu) I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock (clk_div_s)
rst_i	in	1	System reset. Active high.
irq_src_i(7:0)	in	8	Raw peripheral IRQ lines, same clock domain.
irq_pulse_o(7:0)	out	8	1-cycle pulse on each rising edge of irq_src_i; feeds the RIS set logic in cc_icu_top.
mis_i(7:0)	in	8	Masked Interrupt Status (RIS & IMSC), from top.
prio_i(31:0)	in	32	Packed 4-bit priority per source, from top.
gie_i	in	1	Global interrupt enable, from top.
irq_valid_o	out	1	|mis_i, independent of gie_i.
irq_id_o(2:0)	out	3	Highest-priority index among mis_i; ties: lowest index wins.
irq_o	out	1	gie_i & irq_valid_o – to cc_icu_top.

Table 6.80: ICU Top (cc_icu_top) I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
wbdAddr_i(7:0)	in	8	From WB-Bus.
wbdDat_i(63:0)	in	64	From WB-Bus.
wbdDat_o(63:0)	out	64	To WB-Bus.
wbdWe_i	in	1	From WB-Bus.
wbdSel_i(7:0)	in	8	From WB-Bus.
wbdStb_i	in	1	From WB-Bus.
wbdAck_o	out	1	To WB-Bus.
wbdCyc_i	in	1	From WB-Bus.
irq_src_i(7:0)	in	8	Raw peripheral IRQ lines (see Table 6.75).
icu_irq_o	out	1	Aggregated interrupt, to irq_ext_i[7].

6.5.6 Service Requests of the ICU

ICU0 itself does not generate service requests toward the CPU beyond the single aggregated `icu_irq_o` line; Table 6.75 already lists the sources it aggregates. Table 6.81 restates them from the interrupt-cause perspective, i.e. what **ICU0_IRQID** reports once that source becomes the highest-priority pending, unmasked one.

Table 6.81: ICU0 Service Requests

Source (RIS/IRQID id)	Reset Priority	Explanation
0 – GPIO0	7 (highest of the 3 wired sources)	Any GPIO0 pin-change interrupt (`gpio_irq_o`, itself already the OR of GPIO0's own IRQSS/IMSC path).
1 – UART0	6	Any UART0 interrupt source (`uart_irq_o`, itself already the OR of UART0's RIS & IMSC).
2 – QSPI0	5	*RV_PipelineCacheFwd* only. Any QSPI0 interrupt source (`qspi_irq_o`).
3–7 – spares/reserved	0	Not wired to hardware; reachable only via **ICU0_ISR** for software test.

All priorities are software-reconfigurable at runtime via **ICU0_PRIO**; the values above are only the hardware reset defaults (ICU-SW-005), chosen so the system is fully interrupt-functional immediately out of reset without firmware setup.

6.5.7 The ICU Peripheral Kernel

Edge detection. Each of the 8 `irq_src_i` lines is fed through a 2-stage shift register (identical structure to GPIO0's own pin-change detector); `irq_pulse_o[i]` pulses for exactly one clock cycle on the source's 0→1 transition. A source line is treated as a level – it stays high until the peripheral's own cause is cleared (e.g. GPIO0's IRQSC or UART0's ICR) – while ICU0 only reacts to the transition, not the level, when setting its own sticky pending bit.

Pending storage (RIS). **ICU0_RIS**[i] is set by either a hardware edge (`irq_pulse_o[i]`) or a software-forced write to **ICU0_ISR**[i], and cleared by a software write to **ICU0_ICR**[i]. If a SET and a CLEAR of the same bit coincide in the same clock cycle, SET wins (ICU-PE-002): this is the design choice that avoids the race present in the graded reference submission this peripheral replaces, where a same-cycle set+clear silently favoured the clear and could drop a freshly arriving interrupt. Once **ICU0_RIS**[i] is set, it stays set regardless of further source activity until firmware explicitly clears it via **ICU0_ICR** – the two-level scheme (peripheral-level cause, then ICU-level pending) is deliberately decoupled: firmware normally clears the underlying peripheral's own cause first, then ICU0's copy, in either order.

Masking and MIS. **ICU0_MIS** is the bitwise AND of **ICU0_RIS** and **ICU0_IMSC**. The value used by the priority arbiter (`mis_comb_s` inside `cc_icu_top`) is purely combinational and always current; the register readback at offset 0x18 is a registered copy and therefore lags the arbiter's view by one clock cycle – the same split GPIO0 uses between its combinational `irq_mis_s` (drives the IRQ pin) and its registered `mis_reg_s` (drives the readable MIS register), so the CPU-facing interrupt path is never delayed by a register-readback latency.

Priority arbitration. Among all sources with **ICU0_MIS**[i] = 1, the kernel selects the one with the numerically highest **ICU0_PRIO** nibble. The comparison is strict (>, not >=), so an already-selected candidate is only displaced by a strictly higher priority – this is how the "lowest index wins on equal priority" rule (ICU-PR-002) is implemented without a separate tie-break step.

Cause output. **ICU0_IRQID** reports the arbiter's result (bit[3] = VALID, bits[2:0] = winning source id) independent of **ICU0_CTRL**.GIE (ICU-ID-003); `icu_irq_o` itself is `GIE & VALID`, so GIE gates only the pin that reaches the CPU, not the diagnostic register.

Timing summary.

- Wishbone access: zero-wait-state, combinational ACK (via the shared `as_slave_bpi`), identical to every other peripheral in this SoC.

- Edge-to-RIS latency: 2 clock edges from a source's 0→1 transition to the corresponding RIS bit being software-readable.

- No cross-clock-domain synchronization inside ICU0 (ICU-HW-004): all sources, ICU0, and the CPU share `clk_div_s` on both chips.

6.5.8 Registers

ICU Register Overview

Table 6.82: ICU Register List

Register Group	Register Name	Symbol	Offset
System Registers	Identification Register	**ICU0_ID**	0x00
Interrupt Registers	Raw Interrupt Status	**ICU0_RIS**	0x08
	Interrupt Mask Control	**ICU0_IMSC**	0x10
	Masked Interrupt Status	**ICU0_MIS**	0x18
	Interrupt Clear Register	**ICU0_ICR**	0x20
	Interrupt Set Register	**ICU0_ISR**	0x28
Arbitration Registers	Priority Register	**ICU0_PRIO**	0x30
	Cause / IRQ-ID Register	**ICU0_IRQID**	0x38
	Control Register (GIE)	**ICU0_CTRL**	0x40

Read-Write Features

- **w**: writable by SW

- **r**: readable by SW

- **h**: set by HW only

- **w1c**: write-1-to-clear

- **w1s**: write-1-to-set

ID – Identification Register (offset 0x00, ro)

Table 6.83: ICU0_ID Bit Fields

Bits	Name	Mode	Description
[63:0]	ID	r	Fixed peripheral identification, `0x40`. True read-only (no write branch) – unlike GPIO0's ID register, which is nominally rw in RTL despite being documented ro; not replicated here.

RIS – Raw Interrupt Status (offset 0x08, ro/rh)

Table 6.84: ICU0_RIS Bit Fields

Bits	Name	Mode	Description
[0]	GPIO0_PEND	rh	Sticky pending, source 0 (see Section 6.5.7).
[1]	UART0_PEND	rh	Sticky pending, source 1.
[2]	QSPI0_PEND	rh	Sticky pending, source 2 (*RV_PipelineCacheFwd* only).
[3]	SPARE0_PEND	rh	Sticky pending, source 3 (unwired, SW-test only).
[4]	SPARE1_PEND	rh	Sticky pending, source 4 (unwired, SW-test only).
[7:5]	RES_PEND	rh	Sticky pending, sources 5–7 (unwired, SW-test only).
[63:8]	–	r	Reserved; reads as 0.

IMSC – Interrupt Mask Control (offset 0x10, rw)

Table 6.85: ICU0_IMSC Bit Fields

Bits	Name	Mode	Description
[7:0]	MASK	rw	Per-source enable, bit[i] = 1 enables source i into MIS. Reset: `0xFF` (all enabled, ICU-SW-005).
[63:8]	–	r	Reserved; reads as 0.

MIS – Masked Interrupt Status (offset 0x18, ro/rh)

Table 6.86: ICU0_MIS Bit Fields

Bits	Name	Mode	Description
[7:0]	MIS	rh	`RIS & IMSC`, per source. Registered readback, 1 cycle behind the combinational value the arbiter uses (Section 6.5.7).
[63:8]	–	r	Reserved; reads as 0.

ICR – Interrupt Clear Register (offset 0x20, wo/w1c)

Table 6.87: ICU0_ICR Bit Fields

Bits	Name	Mode	Description
[7:0]	CLR	w1c	Write 1 to biti clears **ICU0_RIS**[i], unless a hardware/ISR SET on the same bit lands in the same cycle (SET wins, ICU-PE-002). Reads as 0.
[63:8]	–	–	Reserved; writes ignored.

ISR – Interrupt Set Register (offset 0x28, wo/w1s)

Table 6.88: ICU0_ISR Bit Fields

Bits	Name	Mode	Description
[7:0]	SET	w1s	Write 1 to biti sets **ICU0_RIS**[i] without a real source edge (software test/force). Reads as 0.
[63:8]	–	–	Reserved; writes ignored.

PRIO – Priority Register (offset 0x30, rw)

Table 6.89: ICU0_PRIO Bit Fields

Bits	Name	Mode	Description
[3:0]	PRIO0	rw	Priority of source 0 (GPIO0). Reset: 7.
[7:4]	PRIO1	rw	Priority of source 1 (UART0). Reset: 6.
[11:8]	PRIO2	rw	Priority of source 2 (QSPI0). Reset: 5.
[15:12]	PRIO3	rw	Priority of source 3 (spare). Reset: 0.
[19:16]	PRIO4	rw	Priority of source 4 (spare). Reset: 0.
[31:20]	PRIO5–7	rw	Priority of sources 5–7 (reserved). Reset: 0.
[63:32]	–	r	Reserved; reads as 0, ignored on write.

0 = lowest, 15 = highest priority. On equal priority, the lowest source index wins (ICU-PR-002).

IRQID – Cause Register (offset 0x38, ro)

Table 6.90: ICU0_IRQID Bit Fields

Bits	Name	Mode	Description
[2:0]	ID	r	Index of the highest-priority pending, unmasked source. Meaningful only when VALID = 1.
[3]	VALID	r	1 if at least one source is pending and unmasked (`|MIS`); 0 if no interrupt is pending. Independent of **ICU0_CTRL**.GIE (ICU-ID-003).
[63:4]	–	r	Reserved; reads as 0.

CTRL – Control Register (offset 0x40, rw)

Table 6.91: ICU0_CTRL Bit Fields

Bits	Name	Mode	Description
[0]	GIE	rw	Global interrupt enable. 0 = `icu_irq_o` forced low regardless of pending/masked state; 1 = normal operation (reset, ICU-SW-005).
[63:1]	–	r	Reserved; reads as 0.

Chapter 7

Test and Debug

7.1 Simulation Test Suite

The `RV_NoPipeline` and `RV_PipelineCacheFwd` projects include a regression test suite driven by CMake. Each test assembles a dedicated firmware program, runs it under xsim, and checks the result via GPIO output (0x55 = pass, 0xFF = fail) or a waveform monitor.

7.1.1 UART0 Simulation Tests

Two system-level tests verify UART0 end-to-end. Firmware uses `CLKDIV = 4`, giving a bit period of $16 \times 4 \times 800\,\text{ns} = 51\,200\,\text{ns}$ at the 1.25 MHz core clock.

uart_tx Firmware writes twelve ASCII bytes (*"Hello Rocker"*) to the UART TX FIFO one by one, polling STATUS[0] (tx_busy) before each write. The testbench monitors `uart0_tx_o` and decodes each 8N1 frame, comparing against the expected string. On success GPIO = 0x55 is written.

uart_rx The testbench injects a pre-built 8N1 serial stream (*"We will rock you."*, 17 bytes) onto `uart0_rx_i`. Firmware polls FIFOSTAT[21] (rx_empty) until cleared, then reads the DATA register; each received byte is compared against the expected ASCII value. On full match GPIO = 0x55; on any mismatch GPIO = 0xFF.

RX FIFO Pop Bug (Fixed 2026-06-05)

During development, UART RX reads returned 0x00 for every received byte. Root cause: the non-pipelined CPU (`as_cpux`) uses a two-cycle load sequence —

EXEC_ST (address and read-enable asserted) followed by EXECLD_ST (data captured by the register file). The Wishbone master (`asMBpi`) holds STB = CYC = 1 throughout both cycles, so the UART slave's `rd_data_s` signal was high during EXEC_ST, popping the show-ahead RX FIFO at the EXEC→EXECLD clock edge. EXECLD_ST then saw an empty FIFO and captured 0x00.

Fix: a one-cycle delay register `rd_data_r` was added in `ip_uart_v2/src/as_uart_top` and used as the FIFO `rd_en_i`. The pop now fires at EXECLD→FETCH0, leaving the FIFO head valid throughout EXECLD_ST. The same pattern must be applied to any future read-once (clear-on-read) register on this Wishbone bus.

7.1.2 `RV_PipelineCacheFwd` Simulation Tests

In addition to the shared instruction-set regression (58 tests common with `RV_NoPipeline`, plus `ecall`/`ebreak` and the hazard suite of Section 4.2.1, all run under xsim), `RV_PipelineCacheFwd` has test infrastructure the flat-SRAM `RV_NoPipeline` variant has no equivalent of, since it alone has a cache/AXI4/QSPI-Flash memory subsystem:

- **Cache-specific xsim tests:** `scratchpad_intensive` (RMW byte/halfword/word isolation on the scratchpad SRAM), `dcache_test` (multi-way fill, eviction, warm-cache hits, cold-miss-after-evict), `memory_sections` (scratchpad heap write/read-back across the full address range). Each is a dedicated testbench (`tb/tb_rv64i_*.sv`) with its own multi-phase GPIO checkpoints (`GPIO = 0x0N` per phase, `0x55` on final success), not the shared `tb_rv64i_instr.sv`.

- **cocotb ISS-scoreboard suite** (`cocotb/`, ported from `RV_NoPipeline`'s): re-runs the same 59 programs instruction-by-instruction against a Python instruction-set simulator (`iss.py`), comparing architectural state every cycle rather than only the final GPIO byte. Since this variant has no directly pokeable instruction RAM, `tb_flash_wrap.sv` wraps `as_top_mem` with a behavioural QSPI NOR-Flash model (copied from `tb/tb_rv64i_instr.sv`) so a test program can be loaded by poking `flash_mem_s` directly – bypassing JTAG, but exercising the real I-Cache/AXI4/QSPI path for every fetch.

- **AXI4 cache-error trap test:** `tb/tb_cpupipefwd_cachefault.sv` (xsim, standalone `CMakeLists.txt` in the same directory) verifies the `mcause`=1/5 trap path of Section 4.8.18 directly against `cc_cpupipefwd.sv`, with a scripted `icpu_if`/`dcpu_if` response queue standing in for the cache (whose own `ic_err`/`dc_err` generation is separately unit-tested in `ip_icache/tb/tb_asICache.sv` and `ip_dcache/tb/tb_asDCache.sv`)

– this isolates the CPU's own fault-flag threading and `mcause`/`mepc`/`mtval` logic from the cache/AXI4 timing it is triggered by.

Flush/Stall Priority Race (Fixed 2026-07-13)

During verification of the `ecall`/`ebreak` traps (a new cycle-accurate cocotb ISS scoreboard, ported from `RV_NoPipeline` for this purpose), a pre-existing, previously-hidden pipeline bug was found and confirmed by reproducing it against the unmodified RTL: the `id_ex` and `ex_mem` pipeline registers only processed `flush_s`-driven squashing (branch/trap/`mret` redirect) inside an `else if (!global_stall_s)` branch. When a redirect landed on the exact same cycle as an ordinary one-cycle icache-hit lookup stall, the flush was silently skipped: stale, wrong-path instructions already latched in `id_ex`/`ex_mem` survived and were committed once the stall cleared.

xsim's GPIO-only pass/fail check never caught this: the wrong path happened to reach the same final output byte by coincidence, on every one of the 68 tests then passing. It only surfaced once the ISS scoreboard started comparing architectural state cycle-by-cycle rather than only the final GPIO write.

Fix: `flush_s` (and, for `ex_mem`, `wb_trap_taken_s`/ `wb_mret_s`) now takes priority over the stall gate in both registers' clocking, matching how `if_id`/`ibuf` already handled the same situation. The general lesson – **any squash condition driven by a WB-stage or EX-stage event must never be nested inside a stall-gated branch, since the two can coincide** – was applied again, by construction, when the AXI4 cache-error fault flags were added later (Section 4.8.18): `icache_stall_s`/ `dcache_stall_s` release on `ic_err`/`dc_err` exactly like on `ic_rvalid`/`dc_rvalid`, so a fault is never left waiting behind a stall condition that will never clear.

Chapter 8

Electrical Characteristics

Not yet

Chapter 9

Memory Map and Registers

9.1 System Memory Map

Table 9.1 shows the complete address map of the **RWU-RV64I** system. The address space is 64-bit. Bit 32 of the address separates main memory (bit 32 = 0) from the peripheral bus (bit 32 = 1); the peripheral bridge in **RV_PipelineCacheFwd** uses this bit to route accesses. All peripheral registers are 64-bit wide and 8-byte aligned. The column *AW* gives the address-width parameter passed to the peripheral's Wishbone slave interface (lower *AW* bits of the full address; the base address is always aligned to 2^{AW} bytes so the lower *AW* bits equal the register offset directly).

Table 9.1: System Memory Map

Base Address	Size	Peripheral	AW	Notes
Main memory (bit 32 = 0)				
`0x0000_0000_0000_0000`	64 kB	D-Mem / Scratchpad	—	*RV_NoPipeline*: flat SRAM. *RV_PipelineCacheFwd*: scratchpad.
Peripheral bus (bit 32 = 1)				
`0x0000_0001_0000_0000`	512 B	**GPIO0**	7	cs[1]. Both variants.
`0x0000_0001_0000_0200`	512 B	**QSPI0**	8	cs[2]. *RV_PipelineCacheFwd* only.
`0x0000_0001_0000_0400`	512 B	**CGU**	4	cs[3]. Both variants.
`0x0000_0001_0000_0600`	512 B	**UART0**	8	cs[4]. Both variants.
`0x0000_0001_0000_0800`	512 B	**ICU0**	8	cs[5]. Both variants; aggregates GPIO0/QSPI0/UART0 IRQs into `irq_ext_i[7]`. See Section 6.5 for the register map.

Tables 9.2–9.5 repeat the register offsets of every peripheral (from its own base address, see Table 9.1) for quick reference; full bit-field descriptions are in the peripheral's own chapter/section as noted below each table. Each table states which chip variant(s) (Table 4.29) it applies to.

Table 9.2: GPIO0 Register Offsets (from base `0x0000_0001_0000_0000`) – *Both variants*

Offset	Symbol	Mode	Description
`0x00`	**GPIO0_ID**	rw	Peripheral ID
`0x08`	**GPIO0_DIR**	rw	Direction, per pin: 1 = input (pad → reg), 0 = output (reg → pad, SW-writable)
`0x10`	**GPIO0_DATA**	rw	Pin data: reads pad value (input pins) or driven value (output pins); writes set output pins
`0x18`	**GPIO0_IRQSS**	ro	Interrupt Request Source Status (sticky, one bit per pin; hardware-set on pin change)
`0x20`	**GPIO0_IRQSM**	rw	Interrupt Request Source Mask (per-pin; masks IRQSS into the single RIS[0] bit)
`0x28`	**GPIO0_IRQSC**	wo	Interrupt Request Source Clear (write-1-to-clear IRQSS)
`0x30`	**GPIO0_ISR**	wo	Interrupt Set Register (SW test: force RIS[0] without a pin change)
`0x38`	**GPIO0_RIS**	ro	Raw Interrupt Status (single combined bit, RIS[0] only)
`0x40`	**GPIO0_IMSC**	rw	Interrupt Mask Control (chip-level enable feeding `gpio_irq_o`)
`0x48`	**GPIO0_MIS**	ro	Masked Interrupt Status (= IMSC ∧ RIS)

See Section 6.1 (GPIO) for full bit-field descriptions.

Table 9.3: QSPI0 Register Offsets (from base `0x0000_0001_0000_0200`) – *RV_PipelineCacheFwd only*

Offset	Symbol	Mode	Description
`0x00`	**QSPI0_ID**	ro	Peripheral ID
`0x08`	**QSPI0_CTRL**	rw	[11:4] = `qspi_ctrl_t` (addr_len, cpol, cpha, dual, cs_hold, xip, ddr, quad), [3] = start (pulse), [2] = rx_flush, [1] = tx_flush
`0x10`	**QSPI0_CMD**	rw	[7:0] SPI opcode (reset = `0x6B`, Quad Fast Read)
`0x18`	**QSPI0_ADDR**	rw	[31:0] Flash address
`0x20`	**QSPI0_LEN**	rw	[15:0] Transfer length in bytes (reset = 8)
`0x28`	**QSPI0_DUMMY**	rw	[5:0] Dummy cycles (reset = 8)
`0x30`	**QSPI0_CLKDIV**	rw	[7:0] SCK divider (reset = 4)
`0x38`	**QSPI0_TIMEOUT**	rw	[31:0] Timeout counter (reset = 0, disabled)
`0x40`	**QSPI0_ISR**	wo	Interrupt Set Register (write-1 $\rightarrow$ set RIS bit)
`0x48`	**QSPI0_RIS**	ro	Raw Interrupt Status: [4] = TIMEOUT, [3] = ERR, [2] = TXHALF, [1] = RXHALF, [0] = DONE
`0x50`	**QSPI0_IMSC**	rw	Interrupt Mask Control
`0x58`	**QSPI0_MIS**	ro	Masked Interrupt Status (= RIS $\wedge$ IMSC)
`0x60`	**QSPI0_ICR**	wo	Interrupt Clear Register (write-1 $\rightarrow$ clear RIS bit)
`0x68`	**QSPI0_RXDATA**	ro	Pop from RX FIFO
`0x70`	**QSPI0_TXDATA**	wo	Push to TX FIFO
`0x78`	**QSPI0_FIFOSTAT**	ro	[28:24] = rx_level, [20] = rx_full, [19] = rx_half, [12:8] = tx_level, [4] = tx_full, [3] = tx_half
`0x80`	**QSPI0_XIPMODE**	rw	[7:0] XIP mode byte (reset = `0xA0`, Winbond mode)
`0x88`	**QSPI0_STATUS**	ro	[4] = timeout, [3] = error, [2] = xip_active, [1] = done, [0] = busy

QSPI0 does not exist in *RV_NoPipeline* (Table 4.29); see Section 5.3 (Memory System) for the AXI4 cache-refill path this peripheral also serves.

Table 9.4: CGU Register Offsets (from base `0x0000_0001_0000_0400`) – *Both variants*

Offset	Symbol	Mode	Description
`0x00`	**CGU_ID**	rw	Peripheral ID

The CGU's clock-divider ratios (`clk_core_div`, `clk_bus1_div`, `clk_bus2_div`, `clk_qspi_div`) are compile-time constants, not registers – see the Clock Concept, Section 4.5.1.

Table 9.5 repeats the UART0 register offsets (from the UART0 base address) for quick reference (*both variants*); full bit-field descriptions are in Section 6.4.

Table 9.5: UART0 Register Offsets (from base `0x0000_0001_0000_0600`)

Offset	Symbol	Mode	Description
`0x00`	**UART0_ID**	ro	Peripheral ID (`0x20`)
`0x08`	**UART0_LCR**	rw	Line Control (data bits, parity, stop bits, break)
`0x10`	**UART0_CLKDIV**	rw	Baud-rate divisor (16-bit)
`0x18`	**UART0_CTRL**	rw	Control (loopback, tx_flush, rx_flush)
`0x20`	**UART0_STATUS**	ro	TX/RX busy; sticky error flags
`0x28`	**UART0_DATA**	rw	Write: TX FIFO push; Read: RX FIFO pop
`0x30`	**UART0_FIFOSTAT**	ro	TX/RX FIFO levels, full, half, empty
`0x38`	**UART0_RXTHRES**	rw	RX interrupt threshold (default = 8)
`0x40`	**UART0_RIS**	ro	Raw Interrupt Status
`0x48`	**UART0_IMSC**	rw	Interrupt Mask Control
`0x50`	**UART0_MIS**	ro	Masked Interrupt Status
`0x58`	**UART0_ICR**	wo	Interrupt Clear (write-1-to-clear)

9.2 Interrupt Map

Table 9.6 shows how the peripheral interrupt sources map onto the 8-bit `irq_ext_i` vector at the CPU boundary, for each chip variant. As detailed in Section 3.3.8, **only bit 7 of `irq_ext_i` is synchronised and evaluated by the CPU** (identically in `as_cpux` and `cc_cpupipefwd`); bits `[6:0]` are declared `UNUSEDSIGNAL` in the CPU RTL and have no effect on program flow, regardless of what is wired to them at the top level.

Table 9.6: External Interrupt Map (`irq_ext_i`)

Bit	*RV_NoPipeline*	*RV_PipelineCacheFwd*	Consumed by CPU?
7	`icu_irq_s`: GPIO0 | UART0, arbitrated by ICU0 (`icu_irq_o`, Section 6.5); QSPI0 source tied to 0 (no QSPI0 in this variant)	`icu_irq_s`: GPIO0 | QSPI0 | UART0, arbitrated by ICU0 (`icu_irq_o`, Section 6.5)	**Yes** – drives `mip.MEIP`
6 – 0	tied to 0	tied to 0	**No** – reserved (5 of ICU0's 8 source slots remain unwired, Table 6.75)

Both GPIO0 and UART0 (and, on `RV_PipelineCacheFwd`, QSPI0) feed ICU0 directly, not `irq_ext_i`; ICU0's own arbitrated output `icu_irq_o` is the only signal driving bit 7. Firmware determines the interrupt cause by reading **ICU0_IRQID** (Section 6.5) instead of polling each peripheral's own `MIS` register – this was an open item prior to ICU0's introduction and is now resolved on both chip variants. On `RV_NoPipeline`, UART0's interrupt reaches the CPU for the

first time as a consequence of routing through ICU0 (see Table 6.75, “Behavioural change on `RV_NoPipeline`”); previously it was wired to a bit the CPU never read.

Bibliography

[1] The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2, https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf, Accessed: 2024-02-23, 2017.

[2] RISC-V Guide, https://marks.page/riscv/, Accessed: 2024-02-23, 2024.

[3] Five EmbedDev, https://five-embeddev.com/riscv-isa-manual/latest/instr-table.html, Accessed: 2024-02-23, 2024.

[4] G. Lehmann, B. Wunder, and M. Selz, Schaltungsdesign mit VHDL, Synthese, Simulation und Dokume Poing: Franzis, 1994, ISBN: 978-3772361630. [Online]. Available: https://www.itiv.kit.edu/downloads/Buch_gesamt.pdf.